Deployment Guide

24.0.0.1100

NEXACRO

TOBESOFT strives to maintain the accuracy of information provided in this document and will consistently modify and supplement it. However, we cannot guarantee accuracy of information in this document. TOBESOFT assumes no responsibility for errors and omissions. No liability is assumed for damages resulting from the use of information contained herein.

Users of this document implicitly agree that they understand the contents of this disclaimer.

Each trademark herein, including the product name of each company, is a solely registered trademark of each company who developed it, and is protected by patent laws and the copyright laws, and should be used for the purpose of reference only.

TOBESOFT Co.,Ltd.
2-5F INTOPS Building, 617 Bongeunsa-Ro, Gangnam-Gu,
Seoul 06083, Korea
www.tobesoft.com

Table of Contents

Copyright and Disclaimer ii

Part I. Outline 1

1. Nexacro Deployment 2

1.1 Deployment Overview 2

1.2 Deployed Files 3

1.2.1 Desktop NRE 3

Windows NRE 3

macOS NRE 4

1.2.2 Mobile NRE 4

Android NRE and iOS NRE 4

WRE 5

1.3 License 5

Part II. Windows App 7

2. Deploy Installation File and Install App 8

2.1 Deploy to User 8

2.1.1 Apply digital signature to created installation file and then deploy it 8

2.1.2 Deploy installation file by retaining default digital signature 10

2.1.3 Deploy engines without modification 11

2.2 Update App when Launched 12

2.3 Tips on Deployment 12

2.3.1 Change bootstrap URL of shortcut 12

2.3.2 Options for executing *nexacro.exe* 13

2.3.3 Deployment for each OS version 14

2.4 Install App (User) 15

2.5 Check NRE Log 16

Part III. App Builder (Android, iOS, macOS) 17

3. Create App 18

3.1 Enter default settings for the app 19

3.1.1 Basic Property 20

3.1.2 Nexacro Library 20

3.1.3 Target OS 22

3.2 Setting additional options 23

3.2.1 Basic Property Options 24

3.2.2 Target OS > Android Options 25

Android Permission 27

3.2.3 Target OS > iOS Options 28

3.2.4 Target OS > macOS Options 30

3.2.5 Generating image files using Target OS > Icon Generate 31

3.2.6 Target OS > Export and import project files 31

3.2.7 Target OS > User Library 33

3.3 Build App 34

4. App List 36

Part IV. iOS/Android/macOS App 38

5. App Development and Execution (iOS/iPadOS - Xcode) 39

5.1 Establish App Development Environment 39

5.2 Develop App Project 39

5.2.1 Create a project 39

5.2.2 Set Nexacro libraries 41

5.2.3 Set resources 42

Set images 42

Set messages 43

5.2.4 Set Config 45

5.2.5 Set build environment 47

AppDelegate.h 47

AppDelegate.m 48

main.m 49

ViewController.h 49

Other settings 50

5.2.6 Set your app stand-alone in disconnected environments 51

5.3 Test App 53

5.4 UIScene Configuration (New) 53

5.4.1 Configuring SceneDelegate 54

SceneDelegate.h 54

SceneDelegate.m 54
5.5 UIScene Configuration (Migration) 55
5.5.1 Framework Update 56
5.5.2 SceneDelegate Configuration 56
SceneDelegate.h 56
SceneDelegate.m 56
5.5.3 Add Scene Settings 57
5.5.4 Check Info.plist Configuration 58
6. App Development and Execution (Android - Android Studio) 59
6.1 Establish App Development Environment 59
6.2 Develop App Project 59
6.2.1 Create a project 60
6.2.2 Set Nexacro libraries 62
6.2.3 Set resources 65
Set images 65
Set strings 66
6.2.4 Set Config 67
6.2.5 Set build environment 70
MainActivity.java 70
plugin_config.xml 70
AndroidManifest.xml 71
AndroidManifest.xml > Additional privilege settings 73
AndroidManifest.xml > FCM settings 76
AndroidManifest.xml > ImagePicker, Camera, FileUpload, FileDailog settings 76
Dependency Management 78
6.2.6 Set your app stand-alone in disconnected environments 78
6.3 Build 80
6.3.1 App test 80
6.3.2 Create installation file 81
7. App Development and Execution (macOS - Xcode) 83
7.1 Preparation 83
7.1.1 Development environment 83
7.2 Develop App Project 83
7.2.1 Create a project 84
7.2.2 Set Nexacro libraries 85
7.2.3 Set resources 86
7.2.4 Set build environment 86
main.m 86
Other settings 87

Appendix A. Setting Path 89
A.1 Alias Path 89
A.2 nexacro.xml 90
A.3 Relative Path 90
A.3.1 Relative Path Support for Paths in Projects 90
Appendix B. Exceptions 92
B.1 Web Browser Option 92
B.1.1 JavaScript Activation 92
B.1.2 File Download Activation 92
Appendix C. Checking the NRE log 94
C.1 Create log file settings 94
C.1.1 Create log file settings in Environment 94
Setting log file creation in the tracemode property 94
Setting logging level and frequency 95
C.1.2 Config file settings 95
Setting -NC options 95
Setting nexacro_config.xml file 95
C.2 nexacro_config.xml file properties 96
C.3 Example of file creation by property 98
C.3.1 backupfilecount 98
C.3.2 traceduration 99
C.4 Reading log files 100
C.4.1 Reading log files script example 100
C.4.2 Restrictions in the Windows operating system 101
Appendix D. NRE remote debugging 102
D.1 NRE remote debugging 102
D.1.1 Project settings 102
D.1.2 Run settings 103
Windows 103
macOS 103
D.1.3 Run the Devices inspect feature 103
If the app is not visible in Remote Target 106
D.1.4 Option settings for Pause While Loading feature 106
Windows, macOS 107
macOS 107
Appendix E. Applying the Cordova Plugin 108
E.1 Nexacro Studio 108
E.2 Android - Android Studio 108
E.3 iOS/iPadOS - Xcode 109

Part I.

Outline

1.

Nexacro Deployment

To use apps developed with the Nexacro, an appropriate environment must be created on the user's devices. Certain files must be downloaded to the user environment. The process of establishing this environment and downloading files is called "deployment". Deployed files are managed automatically based upon the cache policy and are replaced with new files as necessary.

If you launch a Nexacro-based app (Nexacro app or Nexacro app for short) in web browsers running on the desktop computers or mobile devices, your device will need just HTML, JavaScript, CSS files from the web server to execute the app. In the Nexacro runtime environment (NRE), you need to download and install other modules. Detailed deployment processes may vary according to run environments.

1.1 Deployment Overview

To run Nexacro apps, you need several files. The deployed files for each execution environment are listed in the following table.

File	Contents	NRE	WRE
Runtime engine	Runtime engine used to process scripts, memories and rendering	O	X
Nexacro framework	Frameworks and component libraries (JavaScript)	O	O
Code	Code converted as the result of generation (JavaScript, CSS)	O	O
License	License file	O	O

The most important feature of the NRE is its runtime engine. Frameworks, components and code are directly applicable to the web browser environment. By contrast, the NRE requires the distribution of the runtime engine appropriate to each operating environment.

You may feel that the deployment of the runtime engine is inconvenient. In many cases, however, users want connectivity with existing systems or need extended features that web browsers cannot provide. In those cases, the Nexacro runtime environment can meet their needs.

Nexacro offers its basic features no matter which version you use. However, extended features, such as device APIs and connectivity with external devices, may vary according to run environments.

1.2 Deployed Files

This section describes each distribution in detail.

Please refer to Alias Pathfor alias paths used by the Nexacro to set paths on the user's computer.

The of name of files and libraries listed herein may change depending on distribution updates. The list of files generated may vary according to the deployment option.

1.2.1 Desktop NRE

Windows NRE

After distributing the Nexacro runtime engine, apps are called according to their execution types. Although the same contents are included, framework JavaScript files are not be downloaded from the server but are distributed by being included in library files of the DLL format.

Category	File	Description	Location
Runtime Engine	v8.dll	V8 Script Engine Library	
	XMemLib.dll	Memory Management Library	
	XBasicLib.dll	Base Libaray	
	nexacrolib.dll	Platform & Render Engine Library	
	nexacro.exe	Nexacro Executor	
Framework DLL	XFrameworkLib.dll	Framework API Library	
Extend DLL Files	protocol adaptor dll	Network Protocol Adaptor	%UPDATE%
	External DLL	External DLLs	%UPDATE%
Runtime Config Files	nexacro.xml	Runtime Execution Environment File & UserProfile	%USERAPP%
Log Files	nexacro_xxxxx.log	Log File	%USERAPP%\Log

Category	File	Description	Location
Launcher Download Files	splash.png	Splash Image File	%USERAPP%
	loadingimage.png	WaitCursor Image File	%USERAPP%
	nexacro.ico	Application Icon File	%USERAPP%
	globalvars.dat	Global Variable List File	%USERAPP%

File names can vary according to engine versions and operating systems. The above table lists important files only.

macOS NRE

The macOS NRE requires separate packaging process. Firstly, you need to build an archive file based on the code converted through generation process. Then, make deployment files based on the archive file.

Category	File	Description	Packing
macOS App Files	nexacroN.macOS.framework	macOS Library Framework	macOS App
	start_macos.json	macOS Archive Information	
	localizable.strings	macOS Locale String Information	
Execute HTML	index.html		Hybrid Run Archive
Framework JS			Hybrid Engine Archive
Component JS			Hybrid Engine Archive
Resource Files	loadingimage.png		Hybrid Engine Archive
Theme File	theme_*.css		Hybrid Theme Archive
Application Files			Hybrid Application Archive

1.2.2 Mobile NRE

Android NRE and iOS NRE

iOS/Android apps need separate packaging processes for the different operation systems. Archive files are created based on the code converted through generation process. Then, deployment files appropriate to each operating system are created.

Category	File	Description	Packing
Android App Files	Acceleration-release.aar ...	Android Archive	Android App
	start_android.json	Android Archive Information	
	strings.xml	Android Locale String Information	
iOS App Files	nexacro.framework	iOS Library Framework	iOS App

Category	File	Description	Packing
	start_ios.json	iOS Archive Information	
	localizable.strings	iOS Locale String Information	
Execute HTML	index.html		Run Archive
Framework JS			Engine Archive
Component JS			Engine Archive
Resource Files	loadingimage.png		Engine Archive
Theme File	theme_*.css		Theme Archive
Application Files			Application Archive

WRE

This type of deployment directly loads all of the required files on the web browsers. It performs differently depending on web browser in use.

Category	File	Description	Location
Execute HTML	index.html		%WEBDEPLOY PROJECT%
Framework JS	BasicObjs.js		%WEBDEPLOY FRAMEWORK%
	SystemBase.js	System Utility	%WEBDEPLOY FRAMEWORK%
	Platform.js	Platform Objects	%WEBDEPLOY FRAMEWORK%
	CssObjs.js	Style Object	%WEBDEPLOY FRAMEWORK%
	Error.js	Error Information	%WEBDEPLOY FRAMEWORK%
Component JS	CompBase.js	Component Base Module	%WEBDEPLOY COMPONENT%
	ComComp.js	Common Component Module	%WEBDEPLOY COMPONENT%
	Grid.js	Grid Component Module	%WEBDEPLOY COMPONENT%
	DeviceAPI.js	DeviceAPI Module	%WEBDEPLOY COMPONENT%
Resource Files	waitimage.gif	WaitCursor Image File	%WEBDEPLOY RESOURCE%
Theme File	theme_*.css	Deployed Theme Source	%WEBDEPLOY THEME%
Application Files		Deployed files that compose an Application	%WEBDEPLOY PROJECT%

1.3 License

The license information is required to distribute the Nexacro apps to users and to use them. A warning message is displayed at the top of the app if a proper license information is not available.

Status	Explanation	Action
No license	The licenseurl property value of the application	Error message displayed

Status	Explanation	Action
	object is not specified or the license file does not exist in that path	
Product information mistmatch	The product information specified in the license is different	Error message displayed
User information mismatch	The IP address specified in the license is different from the host IP address	Error message displayed
Feature information mismatch	Use of features not permitted under the license	Error message displayed
Execution environment mismatch	Use of execution environment not permitted under the license	Error message displayed
Duration mismatch	Use before the start date specified in the license or after the expiration date	Error message displayed
File tampered	License information changed without authorization	Error message displayed
Communication error	Communication error while accessing licenseurl	Error message displayed

The license is not checked in the following cases.

- When running QuickView in Nexacro Studio
- When the starting file path is a local file (file://)
- When running on localhost (127.0.0.1)

Part II.

Windows App

2.

Deploy Installation File and Install App

To provide your Nexacro-based application to the users who use Windows-powered PCs, the Nexacro execution environment must be established in their computers. This is just the same as other software products such as Microsoft Office. After you create an installation file by using the features in the menu [Deploy] in Nexacro Studio, you can deploy the file to users and guide them in the installation process.

2.1 Deploy to User

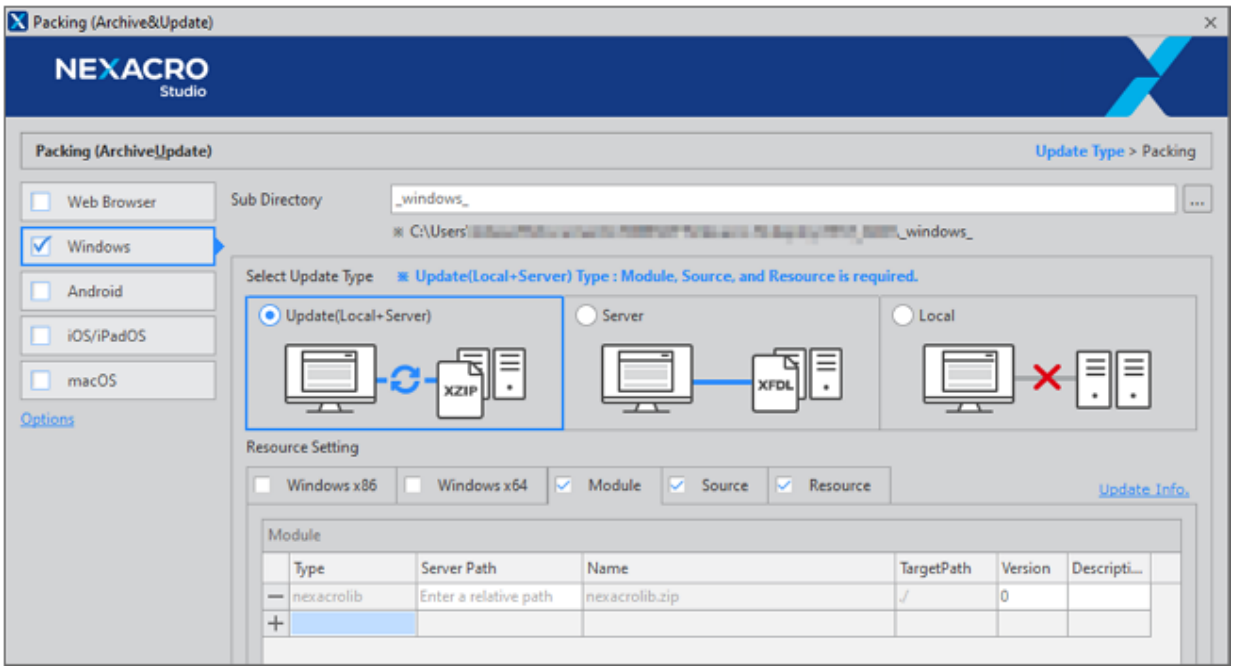
You can create archive files and installation files through the menu [Deploy] in Nexacro Studio. You can change the name of the file and the installation path. System settings serve as a factor for altering the series of steps from you creating and deploying an installation file to users installing your application.

2.1.1 Apply digital signature to created installation file and then deploy it

An installation file will be created without a digital signature if you equip the Nexacro setup engine file with information like an installation path and shortcut icon. If an installation file does not have a digital signature, the installation may not be allowed depending on execution environments.

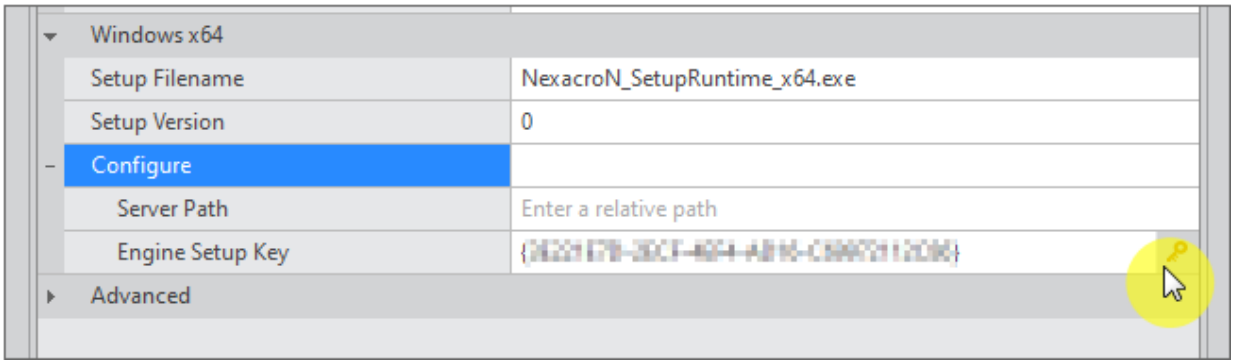
1. Launch the Packing wizard by accessing the menu [Deploy > Packing(Archive&Update)] in Nexacro Studio.

2. In the Packing wizard, select operating systems and then choose the modules, sources and resources that will be deployed.



3 After the packing process is completed, launch the Build App wizard by clicking the Build App button.

4 Configure options through the Build App wizard. If you launch the Build App wizard for the first time in a certain project, a default value will be automatically given as an engine setup key. You can change this setup key value to a new one by clicking a key-shaped icon on the right side.



5 Include a digital signature in the produced installation file.

> If an installation file is executed without a digital signature, the publisher of the app will be identified as an "Unknown Publisher".
> An app may not be allowed to be installed depending on system environments if its publisher is unknown.

> You can apply a digital signature by using a tool like *SignTool.exe*, which is distributed by Microsoft.
> https://msdn.microsoft.com/en-us/library/windows/desktop/aa387764(v=vs.85).aspx

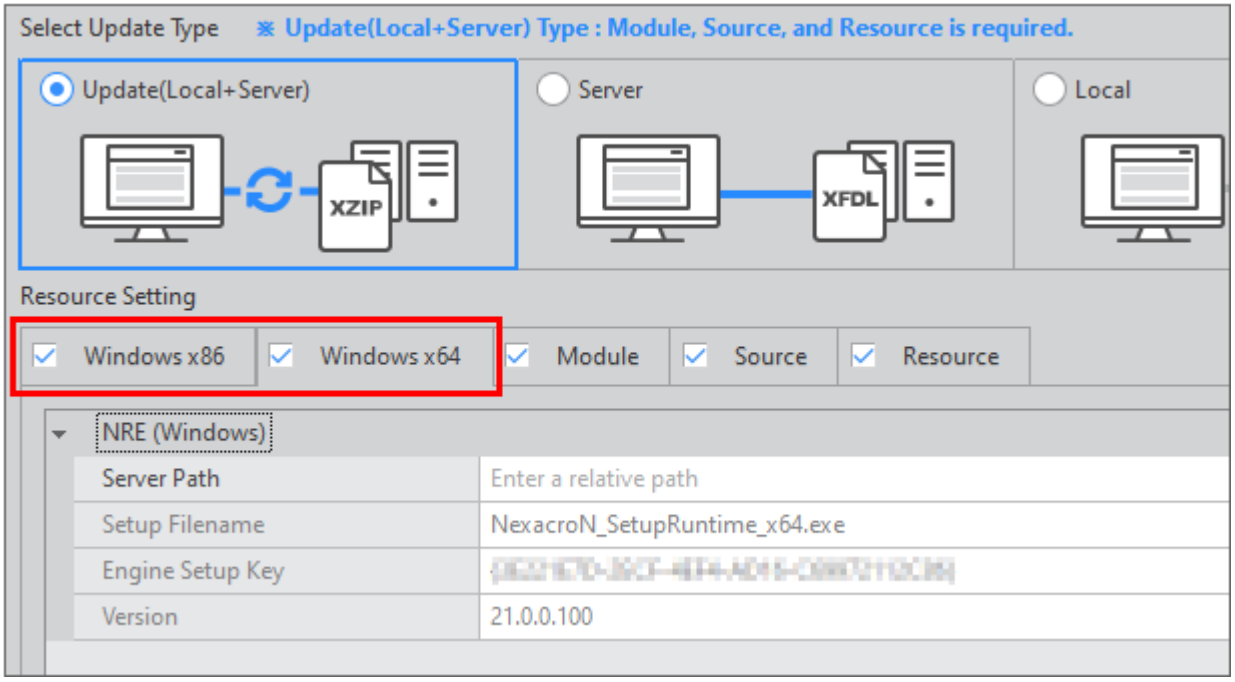
6 Distribute the installation file to users.

2.1.2 Deploy installation file by retaining default digital signature

An installation file will maintain its default digital signature if you equip Nexacro setup engine files only with information like an installation path and shortcut icon, without applying a separate digital signature.

1 Launch the Packing wizard by accessing the menu [Deploy > Packing(Archive&Update)] in Nexacro Studio.

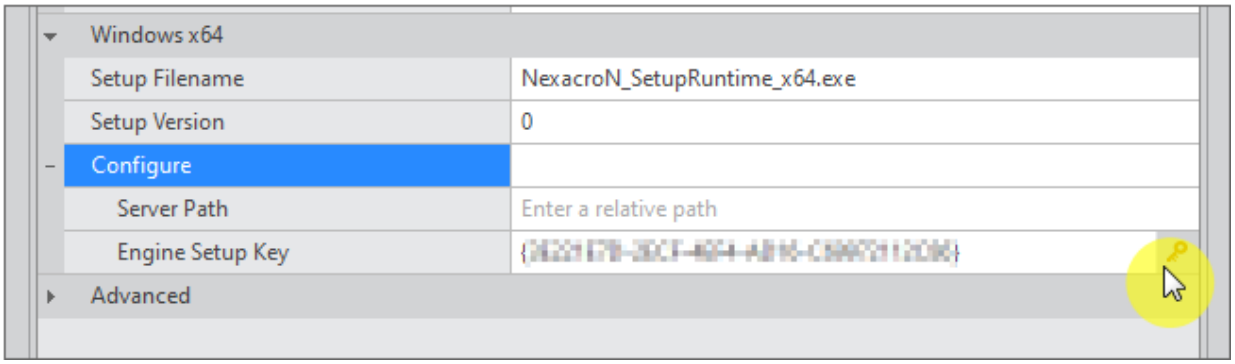
2 In the Packing Wizard, select an execution environment and select modules, sources, and resources to deploy. Check the NRE item according to the operating system.



3 After the packing process is completed, launch the Build App wizard by clicking the Build App button.

4 Configure options through the Build App wizard. If you launch the Build App wizard for the first time in a certain project, a default value will be automatically given as an engine setup key. You can change this

setup key value to a new one by clicking a key-shaped icon on the right side.



5 Distribute the installation file to users. You must download both the installation file produced by the Build App wizard and the *SetupResource.zip* file to the same path.

ProjectResource.zip will be produced if you set the update type to "Update" or "Local" in the Packing wizard.

If you use a launcher service, you need to download the *SetupResource.zip* file to a designated directory by using the *downloadresource* method. In this case, you should proceed with the installation by referring to *SetupResource.zip* regardless of the path where an installation file is downloaded and executed.

If you retain the default digital signature, the below restrictions will be imposed on a distributed installation file and created files as the result of the installation.
- You cannot change the icon of the *SetupRuntime.exe* file.
- You cannot change the icon of the executable file *nexacro.exe*.

2.1.3 Deploy engines without modification

If Nexacro setup engine files are used as they are, you can deploy the files produced as the result of operating the Packing wizard.

1 Launch the Packing wizard by accessing the menu [Deploy > Packing(Archive&Update)] in Nexacro Studio.

2 In the Packing Wizard, select an execution environment and select modules, sources, and resources to

deploy. Check the NRE item according to the operating system.

3 Distribute produced module, source and resource files through your deployment page, from which users can download those files for installation.

> You can find the setup files in the *SetupRuntime* folder, which is located under the directory where Nexacro Studio is installed.

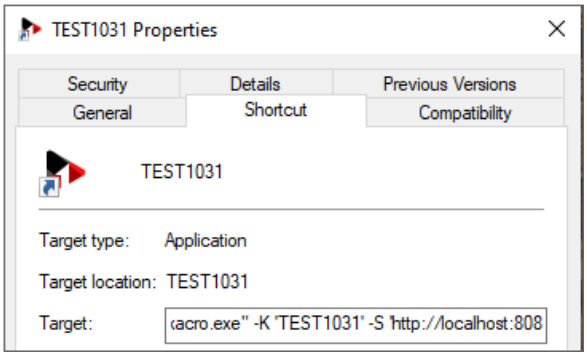
2.2 Update App when Launched

- An app will be updated according to the settings of the *start.json* file, which is located in the path specified as a bootstrap URL, a configuration item set in the Build App wizard. An update will be installed only when the update is a higher version than the current one.
- If you set the update type to “Local” in the Packing wizard, the app will not be updated when it is launched because the administrator of the app cannot deliver *start.json* to you

2.3 Tips on Deployment

2.3.1 Change bootstrap URL of shortcut

If you change the bootstrap URL set for a shortcut installed in your device, the shortcut may not function since the version of the installed app may differ from the deployed version. Therefore, it is not recommended to change the bootstrap URL at your discretion. Instead, if you deploy another app, it is recommended to create and distribute an installation file whose engine setup key and installation location are different.



2.3.2 Options for executing *nexacro.exe*

The below table presents the options for executing *nexacro.exe*.



The options presented here match the options of the *system.execNexacro* method.

Option	Description
-K	This specifies a string that represents the key of an app.
-S	This specifies a path to a bootstrap.
-C	This specifies the directory of components.
-THR	This specifies the number of threads executed in the communications inside the engine or in other manages. Threads amount to three at least, ten at most and three by default.
-SP	This specifies a path to a splash image.
-SM	This specifies "none", which prevent a system message from appearing in a splash image.
-LF	This specifies the coordinates of the left position of an app.
-TP	This specifies the coordinates of the top position of an app.
-WD	This specifies the width of an app in pixel.
-HT	This specifies the height of an app in pixel.
-LG	This specifies a path to a loading image.
-TW	This specifies the maximum waiting time for the threads of the managers inside the engine.
-OO	This specifies a Boolean value to determine whether to permit the execution of only one app if there are more than one app that have the same key.
-V	This specifies global variables with the format "variable name:type=value variable name:type=value". Variables are separated by whitespaces. If you specify "Type" as "ev", the variable will be added to the Variables field in Environment. If you specify "Type" as "ec", the variable will be added to the Cookies field in Environment. If you do not specify "Type" or specifies it as "av", the variable will be added to the Variables field in the Application.
-SIE	This specifies a Boolean value to determine whether to close an app with an error message opened if the error occurs while executing the app.
-CT	This specifies the maximum waiting time for checking whether a network is connected. The check-up is conducted before the launch of the app. If a network is not detected in the waiting time, the app will be closed.
-SCR	This specifies screen IDs used for the Layout Manager.
-CC	This option is designed to deactivate the code caching of V8. If you specify "NO", the code caching will be unavailable. If you specify "AL" or nothing, the code caching will be available.

Option	Description
-AU	This option determines whether the engine is updated when an app launches. If you specify "auto", the engine will be updated to the latest version. If you specify "manual", the engine will not be updated.
-NC	This specifies the location and name of a config file in the local path.

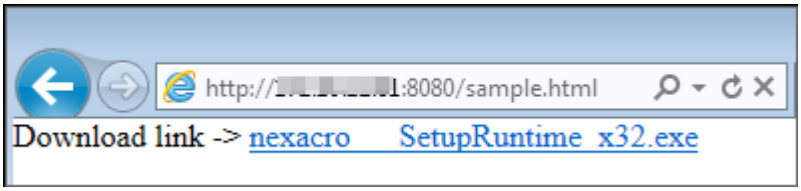
2.3.3 Deployment for each OS version

If you create installation files for each of 32-bit and 64-bit Windows versions, you can provide download information of all the files and then let users select among them. Otherwise, you can write a script designed to check OS versions first and then provide appropriate links.

```
<script type="text/javascript">
var SetupRuntimeName="";
function oninit()
{
    var userAgent = navigator.userAgent;
    if(userAgent.indexOf("WOW64") != -1
        || userAgent.indexOf("Win64") != -1)
        SetupRuntimeName="nexacroN_SetupRuntime_x64.exe";
    else
        SetupRuntimeName="nexacroN_SetupRuntime_x32.exe";

    document.getElementById("test").innerHTML = "<a href='"+SetupRuntimeName+"'
>"
        +SetupRuntimeName+"</a>"
}
</script>
<body onload="oninit()">
Download link -> <span id="test"></span>
</body>
```

If a download page is accessed by a user whose OS is 32-bit, the page can notice the OS version and provide an appropriate link.

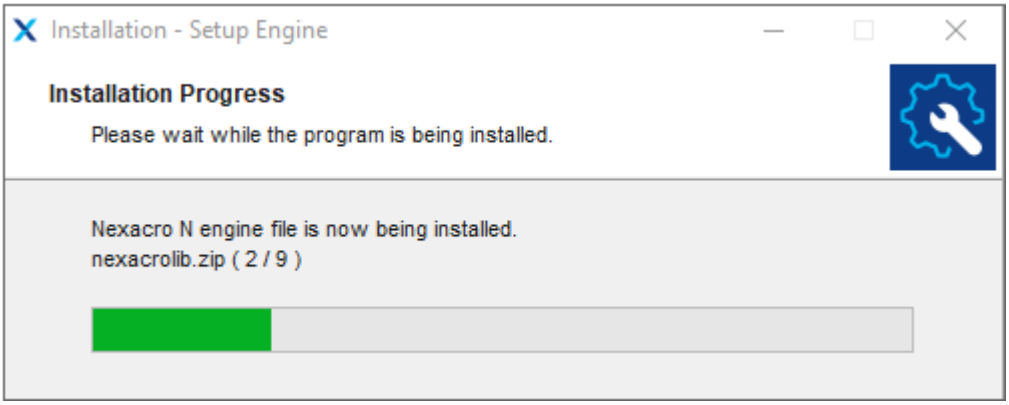




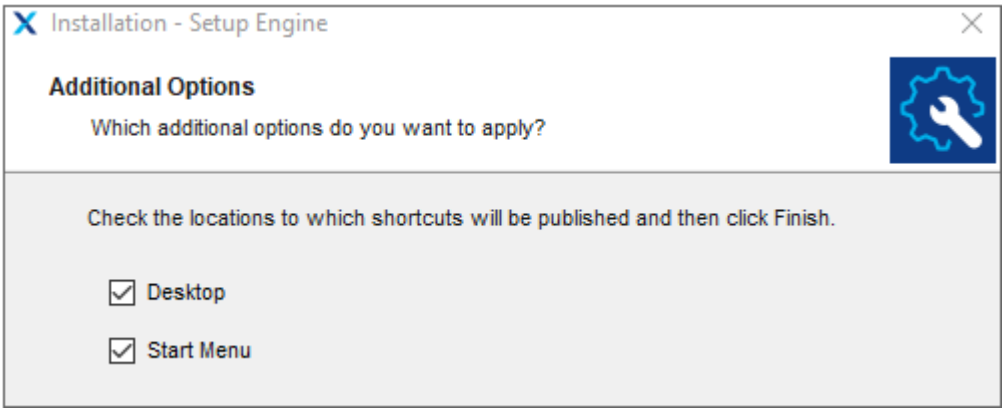
As the above script is written as an example, it can be adapted for actual operating systems. The script may fail if OSes are changed.

2.4 Install App (User)

The process of installing the Nexacro engine will begin when users execute the installation file of your app. You can change the names of the installation folder and installed program in the deployment stage.



After the installation is completed, users will be asked whether to create shortcuts, depending on the settings you configured in the deployment stage. If the installation program is set to create shortcuts automatically, it will do so without asking.



2.5 Check NRE Log

You can adjust settings to leave a log in preparation for some cases like one where an app abruptly terminates in the development stage. A log can also be used to analyze the errors that occur in an app in the user environment. After deploying an app with the *tracemode* property of the project's Environment object set to "new" or "append", you should select either of the below two options.

- Specify "-NC [configuration file]" as an execution option for *nexacro.exe* to create the config file.
- Create the *nexacro_config.xml* file in the directory %USERAPP%\config\[Key].



[Key] refers to the -K option for executing *nexacro.exe* or the app title defined in the Build App stage.



Errors that occur before the launch of the log manager will not be left as a log. Therefore, such errors are displayed to users in the form of an alert when the app is launched.

The configuration file (*nexacro_config.xml*) is defined as below.

```
<?xml version="1.0" encoding="UTF-8"?>
<nexacro-config os="windows">
    <application
        file-logging="true"
        loglevel="debug"
        tracemode="new"/>
    <log
        filepath="%USERAPP%\Log\nre_log_test.log"
        filesize="408960"
        backupfilecount="5"/>
</nexacro-config>
```

Part III.

App Builder (Android, iOS, macOS)

3.

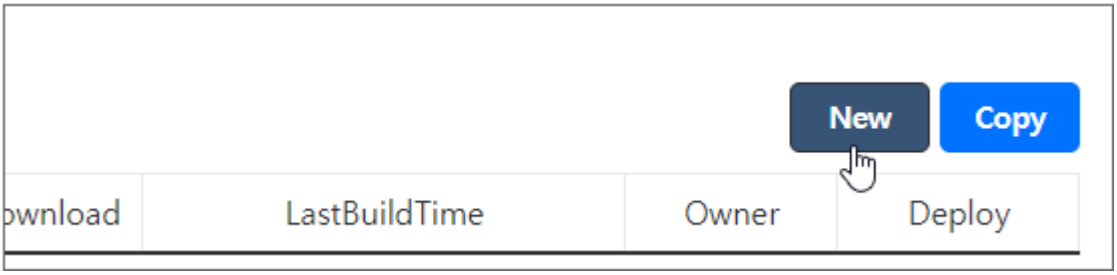
Create App

App Builder related articles

- **Server Setup Guide > App Builder**
  App Builder Installation
  Server Configuration
  Deploy Server Setting
  Signature Setting by OS
  User Library Settings
  Nexacro Library Settings
  User Settings
- **Deployment Guide > App Builder**
  Create App
  App List
- **Development Tools Guide > Build App- Create Installation Files**
  Build App- Create Installation Files

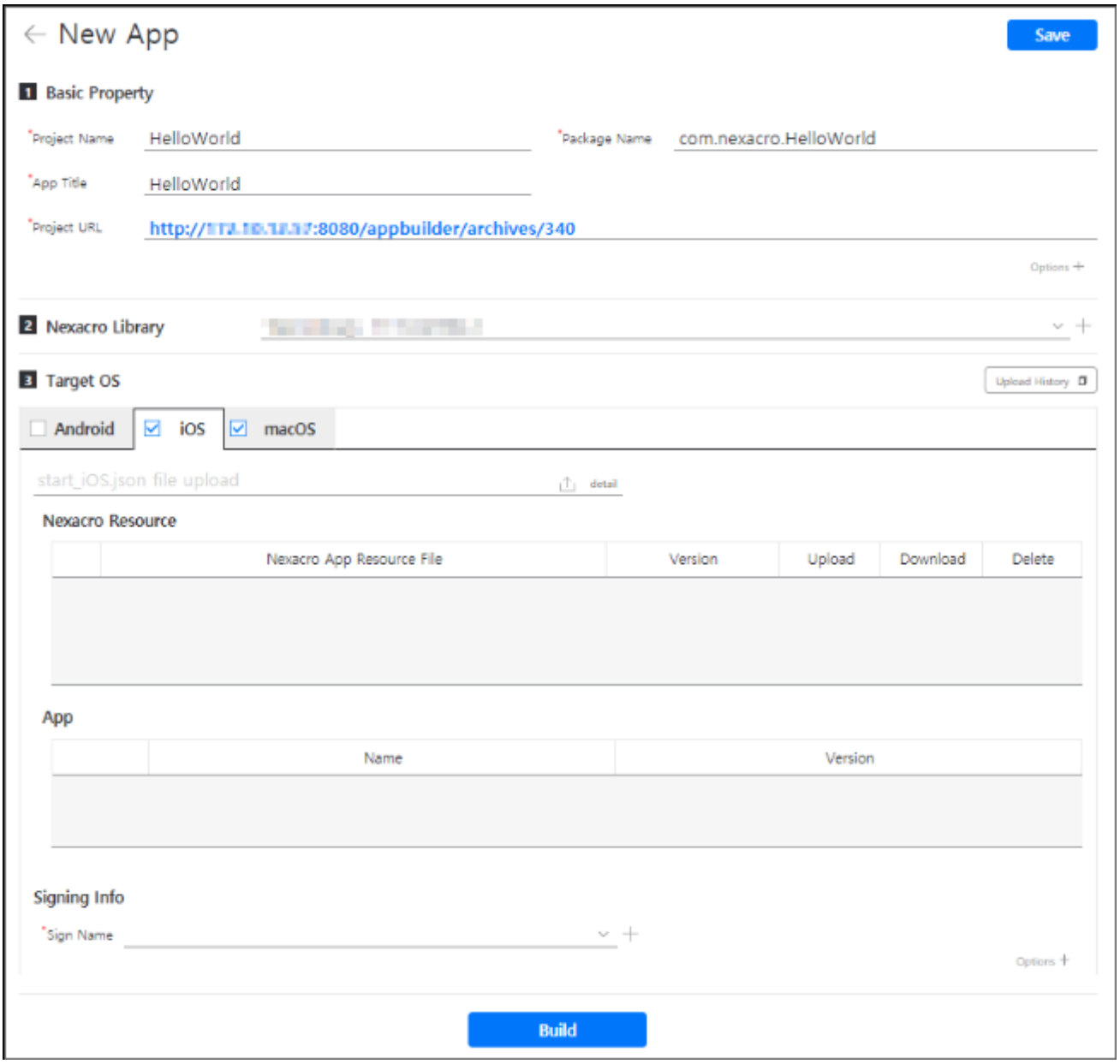
An "App" is a basic unit used in the App Builder for building apps. You should create an App to build new apps. Moreover, you can change or delete an App in the App Builder. You can build Android, iOS and macOS apps depending on how you configure an App.

Click the [New] button in the menu [BuildApp > App List] and a window for creating a new "App" will be opened.



3.1 Enter default settings for the app

The "App" will not be created until you enter the information and click the [Save] or [Build] button.



		Description
1	Basic Property	Specify the basic information of the app to be built and the settings required when building.
2	Nexacro Library	Specify the library as Nexacro.
3	Target OS	Choose the operating system you want to deploy your app on and specify the necessary settings for the building.

3.1.1 Basic Property



		Description
1	Project Name	Enter a name for the project.
2	Package Name	Name of the package
3	App Title	Name of apps to build
4	Project URL	This is the URL where the Nexacro-based app is located. The last slash in the URL must be omitted.

A package name is used as the identifier of apps to build. Normally, a package name is created by writing a domain name in the opposite way.
eg) com.nexacro

If you create a package name with only one word, an error may occur in the build process.

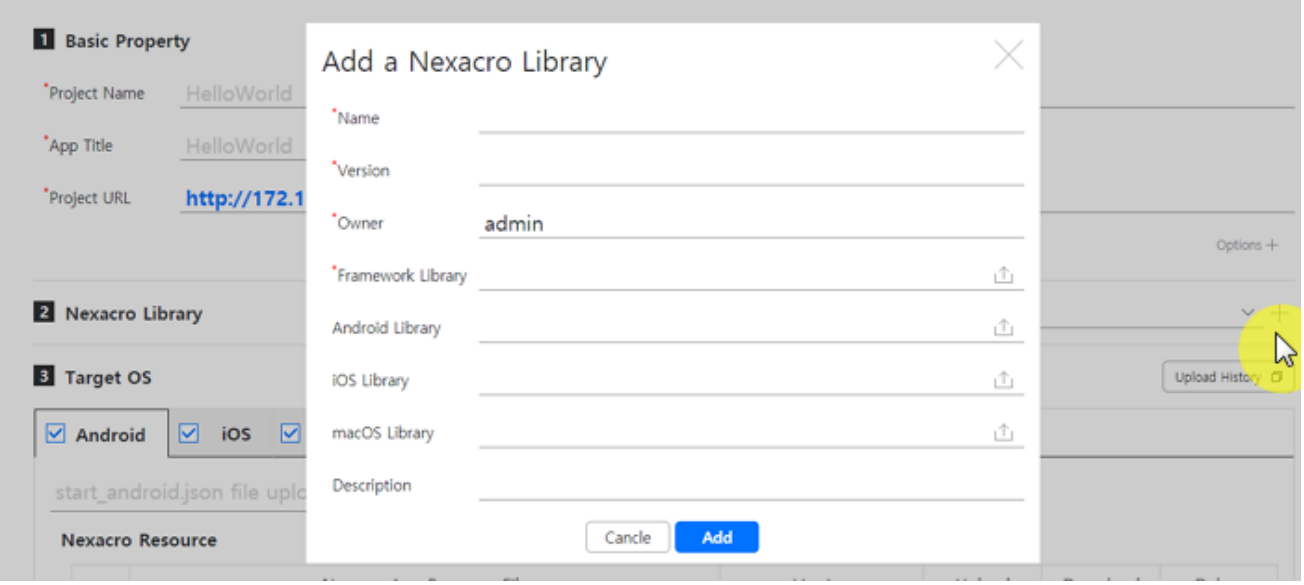
If you include iOS as a target OS, you must use the same package name as the one that you entered when requesting a provisioning profile.

3.1.2 Nexacro Library



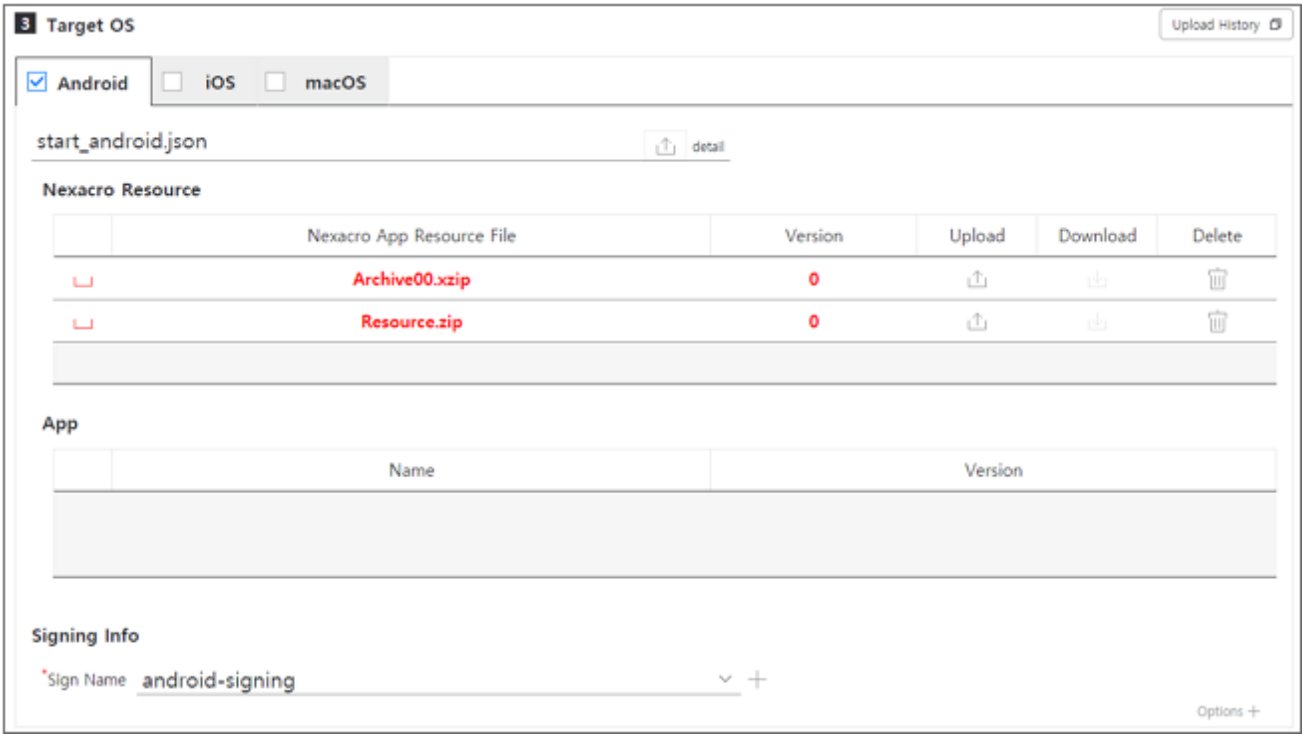
		Description
	Nexacro Library	Selects a Nexacro mobile library. You must choose the library of the same version as the one used in developing the app.

Click the [+] button to add a new library.



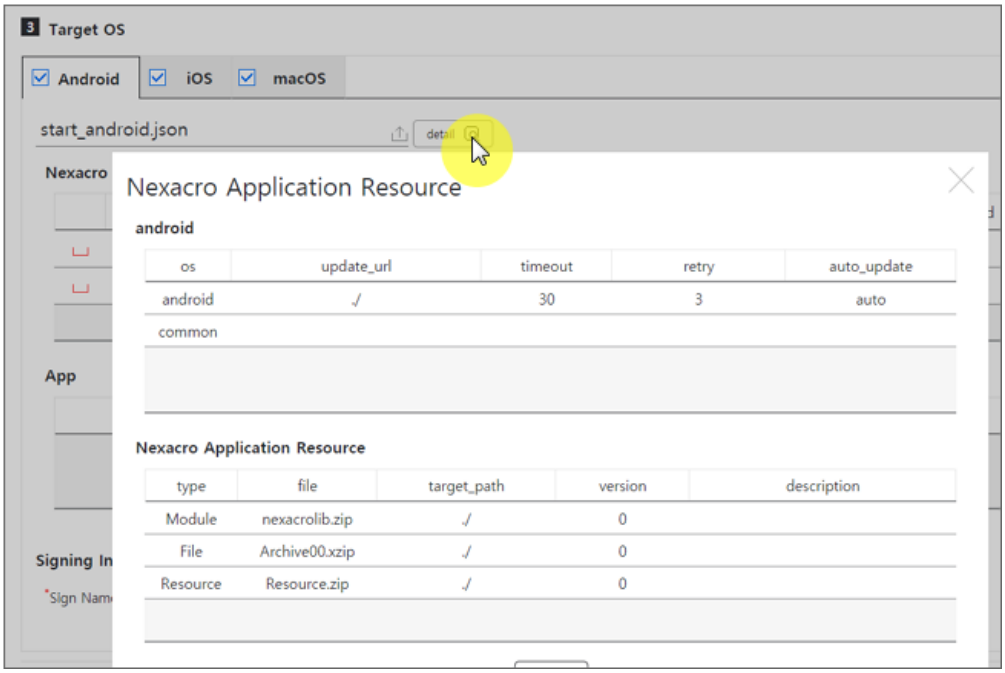
	Item	Description
1	Name	Enter the library name.
2	Version	Enter the library version.
3	Owner	Displays the owner information.
4	Nexacro Framework File	Uploads the nexacrolib.zip file to be deployed. The uploaded item can be downloaded by clicking with the mouse.
5	Version	This is the uploaded Nexacro Framework File version. Checks and displays the version information in the JSON file included in Nexacro Framework File.
6	Library File	Uploads the library file. ex) Android : nexacroN_Android_2021xxxx_1.zip iOS : nexacroN_iOS_2021xxxx_1.zip macOS : nexacroN_macOS_2021xxxx_1.zipThe uploaded item can be downloaded by clicking with the mouse. ● Version: Checks and displays the version information in the JSON file included in Library File. ● BuildProcessor: Selects and displays the build processor according to the library Version information.
7	Description	Enter the description of the library.

3.1.3 Target OS

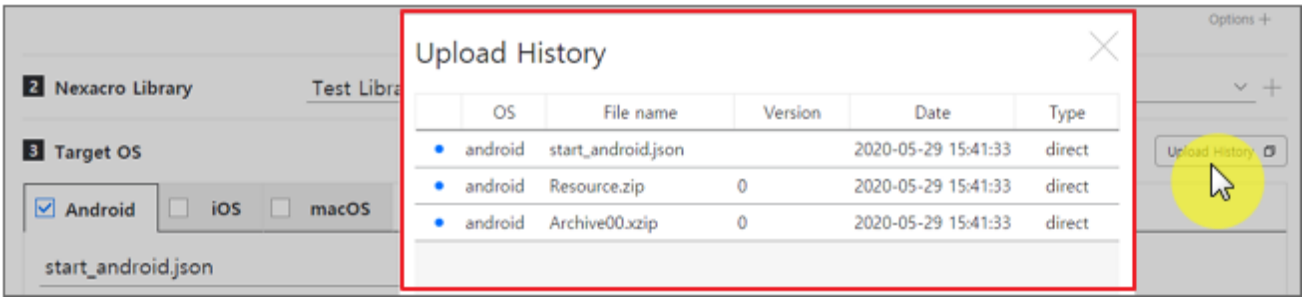


Select OSes for which apps will be built. You can configure detailed settings for each selected OS. If you attempt to upload resources directly from the management console rather than Nexacro Studio, you should register appropriate JSON files (*start_*[OS name]*.json*). Then the *Nexacro application resource* category will print necessary resources, allowing you to upload those resource files.

Clicking the *detail* button opens a table showing the contents of the JSON file.



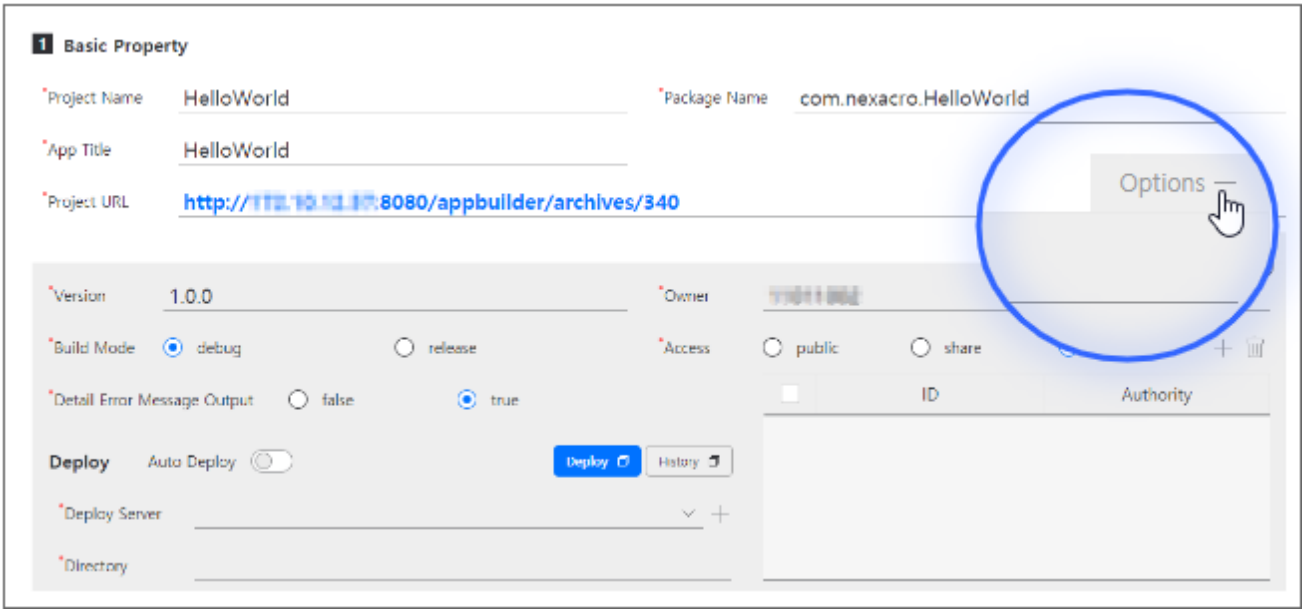
Click the [Upload History] button to check the upload history of the resource files.



On macOS, if you select "none" for the Signing Info information, the app build is processed without including the Signing Info. The build will work normally, but you will need to change the security settings when running the app after installing it.
https://support.apple.com/kb/PH14369?locale=en_US

3.2 Setting additional options

Just enter the basic settings and you can proceed to building the app. To set additional options for each item, click the [Option +] button at the bottom right, expand the option, and then change the settings.

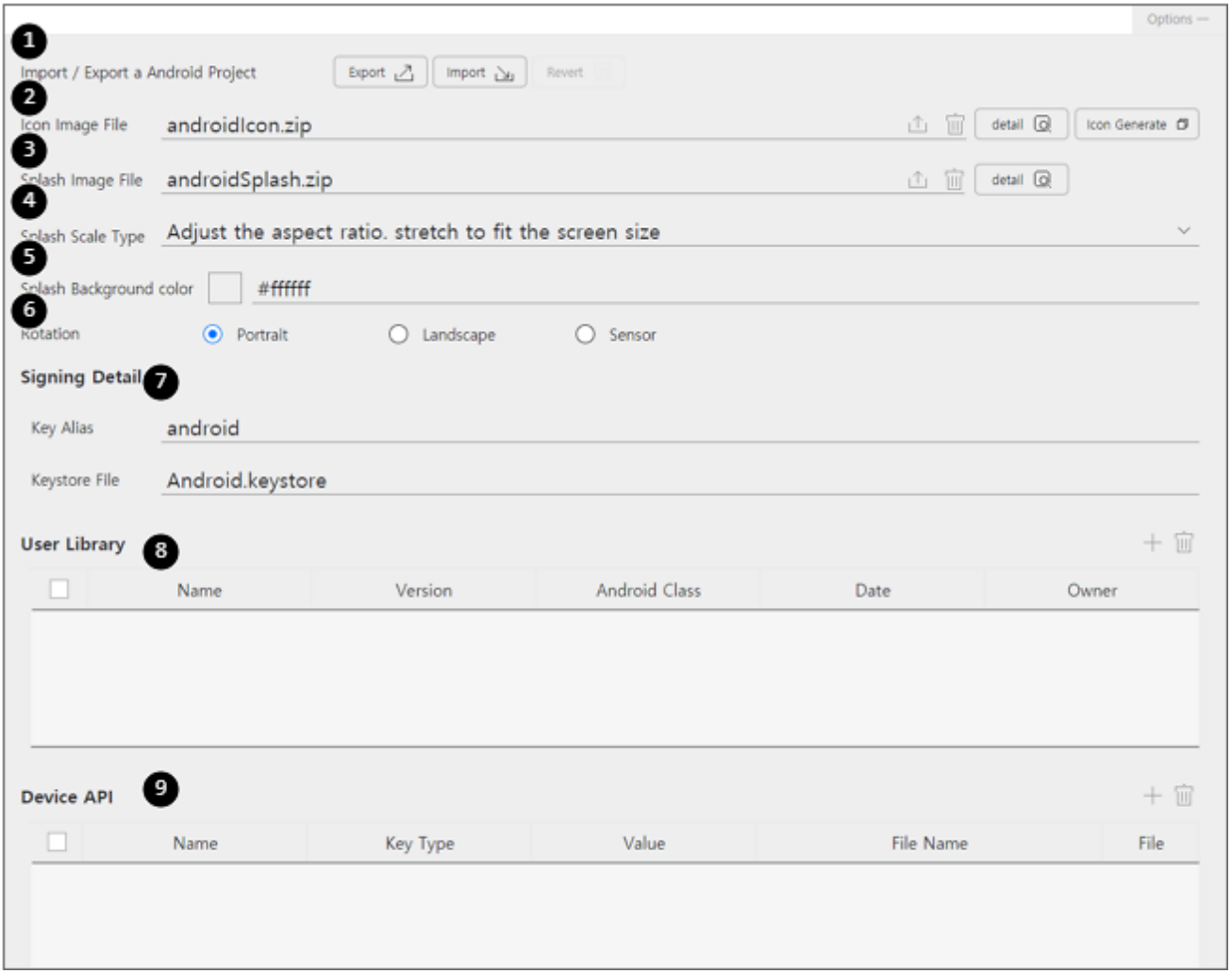


3.2.1 Basic Property Options



		Description
1	Version	Enter the app version to build.
2	Build Mode	elects the build mode. If you choose "debug", you can perform debugging by using Android Debug Bridge (adb).
3	Detail Error Message	Determines whether to expose the error messages that come out during the loading process
4	Deploy	Select or add a deployment server and a server path. Auto Deploy: When an item is selected, the app can be deployed to the deployment server immediately after building. Click the [Deploy] button to deploy the resources to the specified server. Click the [History] button to check the deployment history. Click the [+] button to add information on new deployment servers.
5	Owner	Designates the owner of the App (The account holder who has logged in the App builder will be designated as the owner automatically.)
6	Access	Authorization to use the App. The items of the App List varies according to the authorization. - share: designated users can use the App. - public: all users can use the App. - private: only the owner can use the App.

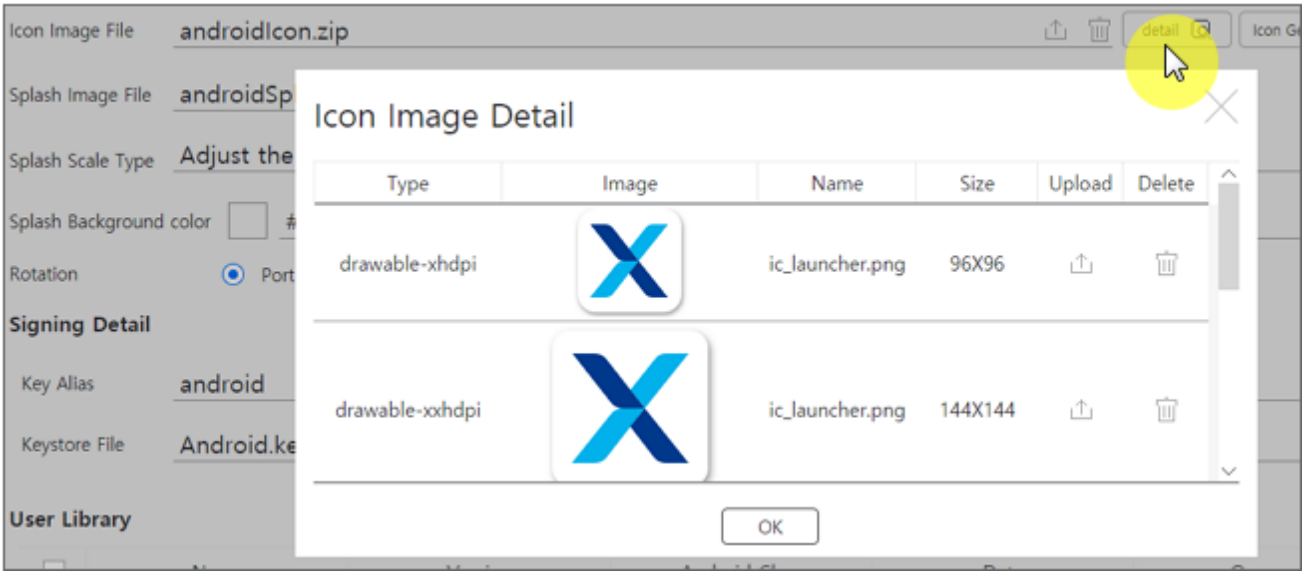
3.2.2 Target OS > Android Options



	Category	Description
1	Import / Export	Based on the created App, you can export a project that can be edited in Xcode. By contrast, you can import a project that has been edited through Xcode.
2	Icon Image File	Designates the image file of the app's icon. If you designate nothing, Nexacro's default image will be applied. You can designate either one compressed file (ZIP format) that contains multiple image files or each image file separately.
3	Splash Image File	Designates the file of app's splash image, which appears at the launch of the app. If you designate nothing, Nexacro's default image will be applied. You can designate either one compressed file (ZIP format) that contains multiple image files or each image file separately.
4	Splash Scale Type	Set the splash image size. One the following 4 options can be selected.

	Category	Description
		- Adjust the aspect ratio. stretch to fit the screen size - Maintain the aspect ratio. expand without blank area - Maintain the aspect ratio. expand to fit - Original size
5	Splash Background color	Specify the background color of the splash area.
6	Rotation	In the app, select an option to lock the screen to Portrait or Landscape, or to unlock the screen and activate the sensor.
7	Signing Detail	Selects a signature
8	User Library	Selects user libraries
9	Device API	Selects device API permissions

Click the [detail] button next to the Icon image and Splash image items to check the set image list.



If you designate a compressed file, you should name the directories and files in the compressed file as defined in the below table.
ic_launcher.png: icon image file
splashimage_phone_landscape.png: splash image file for the landscape orientation
splashimage_phone_portrait.png: splash image file for the portrait orientation

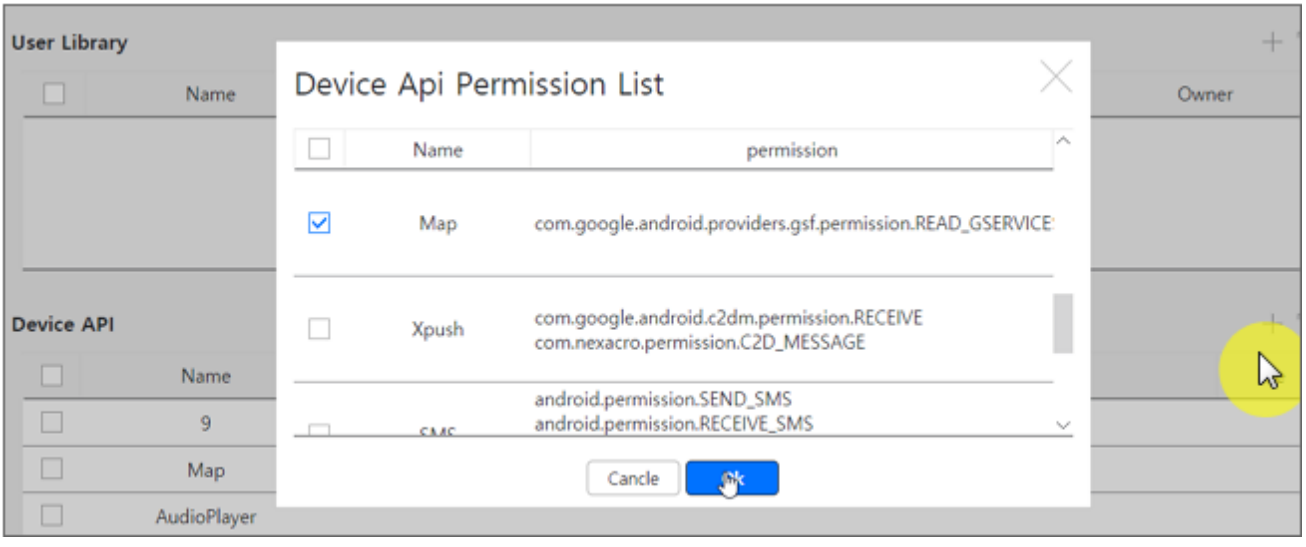
If directories and files are named falsely, the build process may fail. Even if the build process is completed, the app may fail to work properly.

Directory	file name	Image Size(px)
drawable-hdpi	ic_launcher.png	72x72

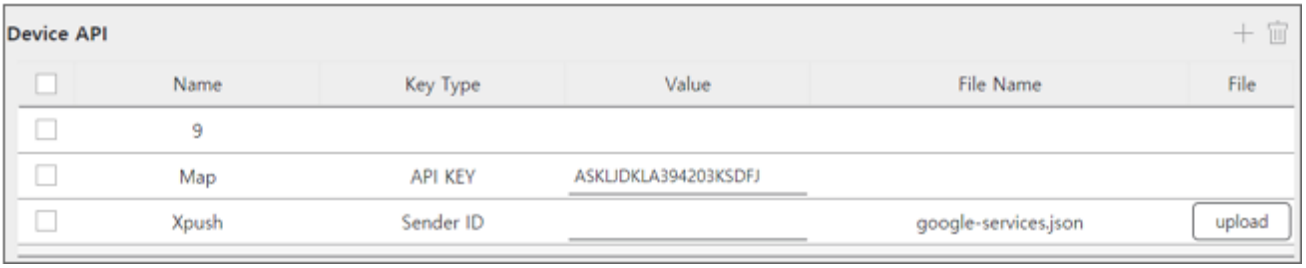
Directory	file name	Image Size(px)
	splashimage_phone_landscape.png	Changing in response to screen sizes
	splashimage_phone_portrait.png	
drawable-ldpi	ic_launcher.png	36x36
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-mdpi	ic_launcher.png	48x48
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xhdpi	ic_launcher.png	96x96
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xxhdpi	ic_launcher.png	144x144
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xxxhdpi	ic_launcher.png	192x192
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	

Android Permission

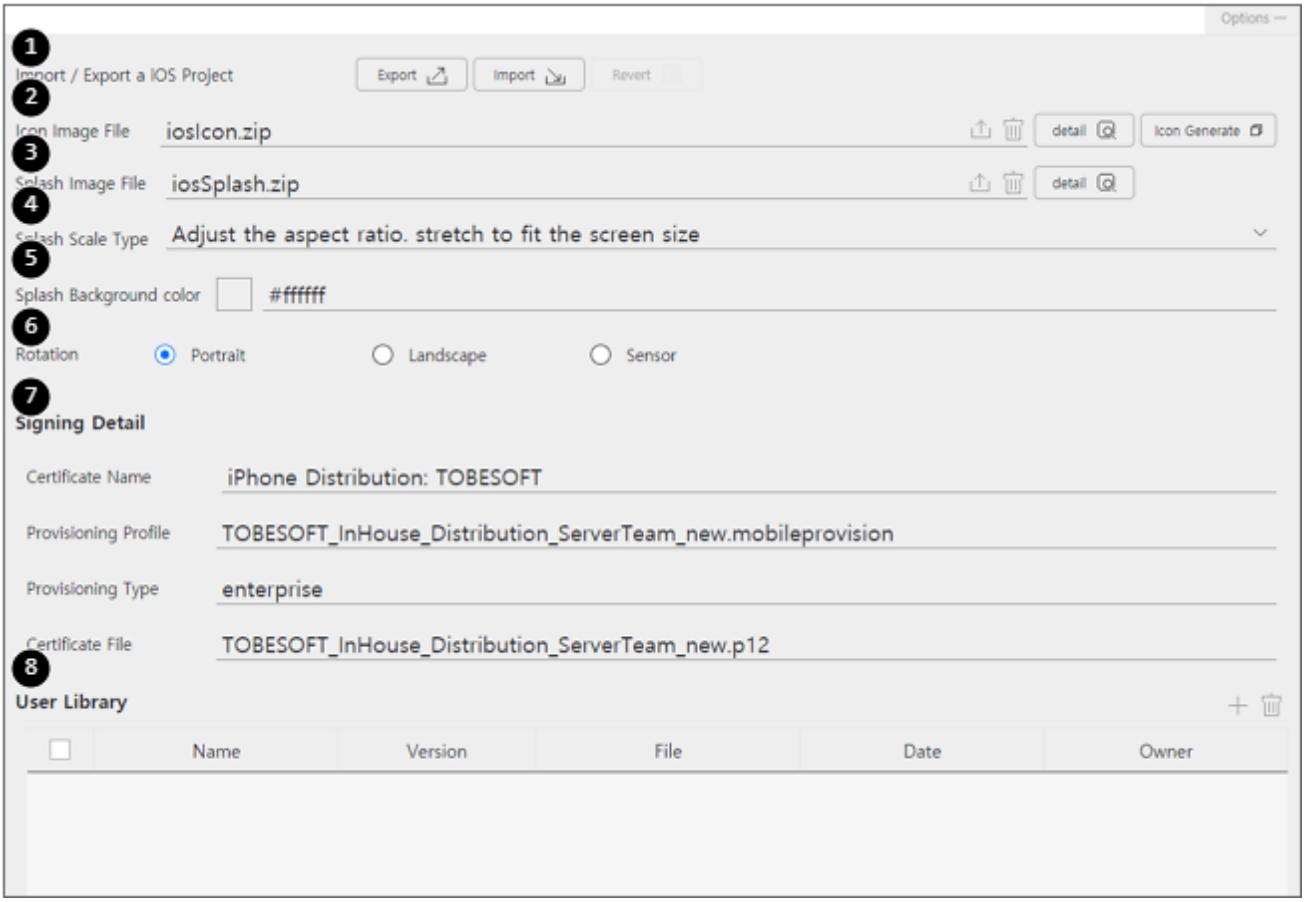
Select permissions for Nexacro device APIs. By clicking the + button, you can open the whole list of the device APIs. After selecting necessary permissions, click the OK button.



You must enter keys additionally when you use the Map and X-PUSH APIs. When you select the permissions for the X-PUSH APIs especially, you must attach a Firebase Android config file (*google-services.json*).



3.2.3 Target OS > iOS Options



	Category	Description
1	Import / Export	Based on the created App, you can export a project that can be edited in Xcode. By contrast, you can import a project that has been edited through Xcode.
2	Icon Image File	Designates the image file of the app's icon. If you designate nothing, Nexacro's default image will be applied. You can designate either one compressed file (ZIP format) that contains multiple image files or each image file separately.
3	Splash Image File	Designates the file of app's splash image, which appears at the launch of the app. If you designate nothing, Nexacro's default

	Category	Description
		image will be applied. You can designate either one compressed file (ZIP format) that contains multiple image files or each image file separately.
4	Splash Scale Type	Set the splash image size. One the following 4 options can be selected. - Adjust the aspect ratio. stretch to fit the screen size - Maintain the aspect ratio. expand without blank area - Maintain the aspect ratio. expand to fit - Original size
5	Splash Background color	Specify the background color of the splash area.
6	Rotation	In the app, select an option to lock the screen to Portrait or Landscape, or to unlock the screen and activate the sensor.
7	Signing Detail	Selects a signature
8	User Library	Selects user libraries

Click the [detail] button next to the Icon image and Splash image items to check the set image list.



You should name the icon and splash image files as defined in the below table.

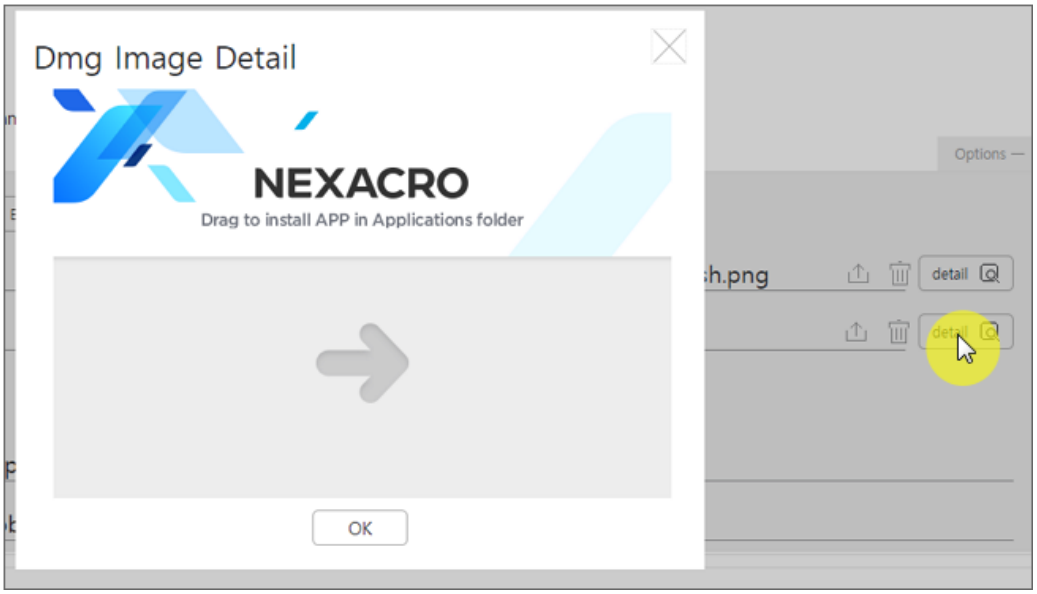
file name	Image Size(px)	Description
Icon.png	57x57	Home screen on iPhone/iPod touch (iOS 6.1 and earlier)
Icon@2x.png	114x114	Home screen on iPhone/iPod Touch with retina display (iOS 6.1 and earlier)
[size].png		57X57 (57.png) 76X76 (76.png) 120x120 (120.png) 152X152 (152.png) 167X167 (167.png)
splashimage_pad_landscape.png	1024x768	Splash image file for the landscape orientation in iPad.
splashimage_pad_portrait.png	768x1024	Splash image file for the portrait orientation in iPad.
splashimage_phone_landscape.png	960x640	Splash image file for the landscape orientation in iPhone.
splashimage_phone_portrait.png	640x960	Splash image file for the portrait orientation in iPhone.
splashimgae_phone5_landscape.png	1136x640	Splash image file for the landscape orientation in iPhone.
splashimgae_phone5_portrait.png	640x1136	Splash image file for the portrait orientation in iPhone.

3.2.4 Target OS > macOS Options



	Category	Description
1	Import / Export	Based on the created App, you can export a project that can be edited in Xcode. By contrast, you can import a project that has been edited through Xcode.
2	Icon Image File	Designates the image file of the app's icon. If you designate nothing, Nexacro's default image will be applied. The extension of the image file is ".icns".
3	Splash Image File	Designates the file of app's splash image, which appears at the launch of the app. If you designate nothing, Nexacro's default image will be applied.
4	Dmg Image File	Designates the file of an image that appears at the installation of the app. If you designate nothing, Nexacro's default image will be applied.
5	Signing Detail	Selects a signature

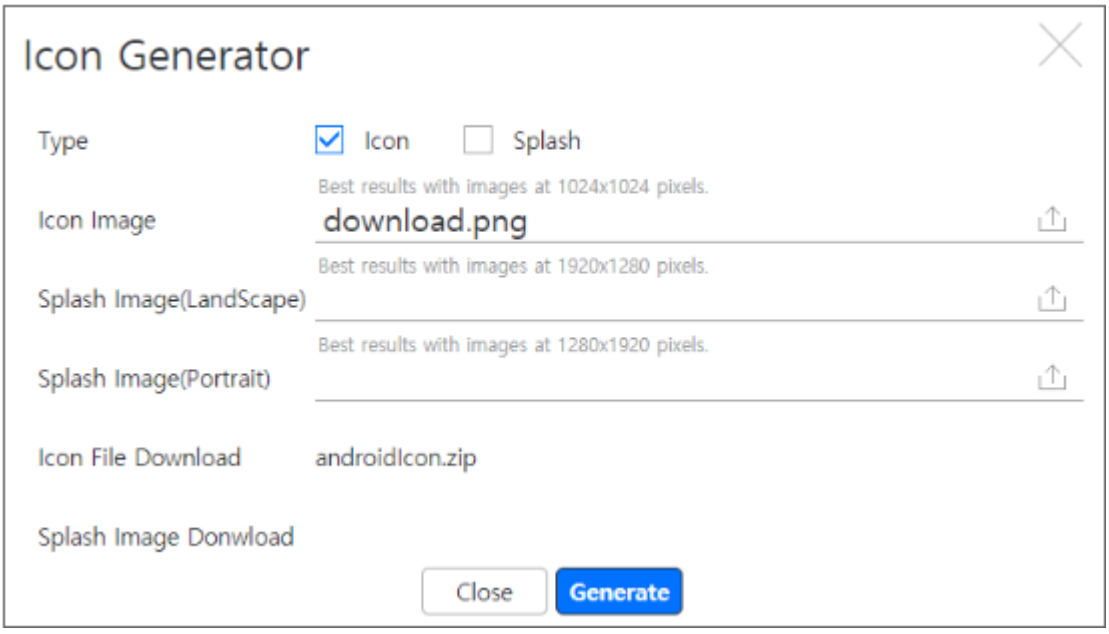
Click the [detail] button next to the Icon, Splash, and Dmg image items to check the set image list.



If the dots per inch (DPI) of the designated DMG file is not 72X72, the background image may appear distorted in macOS 10.7 or later.

3.2.5 Generating image files using Target OS > Icon Generate

Click the [Icon Generate] button next to the image file setting screen to run the Icon Generator screen. After selecting the image type to create and uploading the image file, click the [Generate] button to create a compressed file according to the selected target OS. The created file can be downloaded and uploaded for the image setting.



Only the png files are supported for the uploaded images.

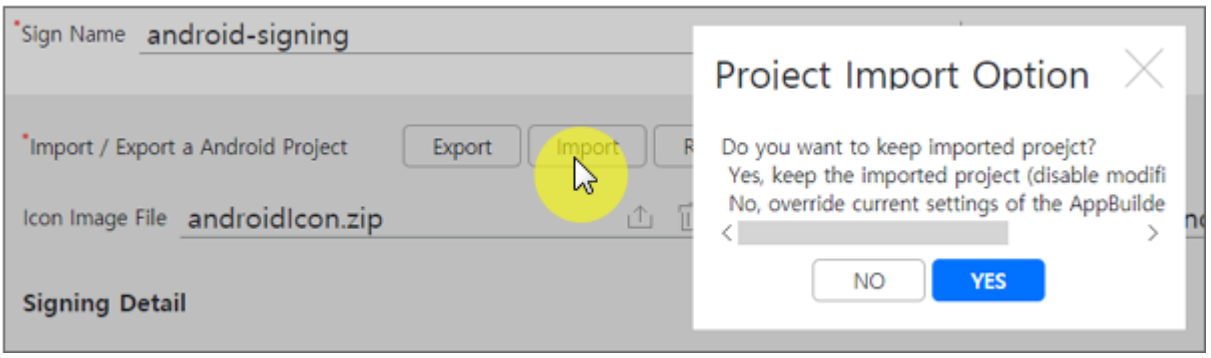
3.2.6 Target OS > Export and import project files

If you need functionality that is not supported by App Builder, you can export a created App in the form of a project so that you can add necessary features or edit the project through development tools of each OS. Access the menu [Project Modification] to export the current App as a compressed file or import a project that was edited through another development tool.



	Button	Description
1	Export	The current App project will be exported as a compressed file.
2	Import	A user-modified project will be imported and added to the current App project.
3	Revert	The state of the current App project will return to the data saved in App Builder.

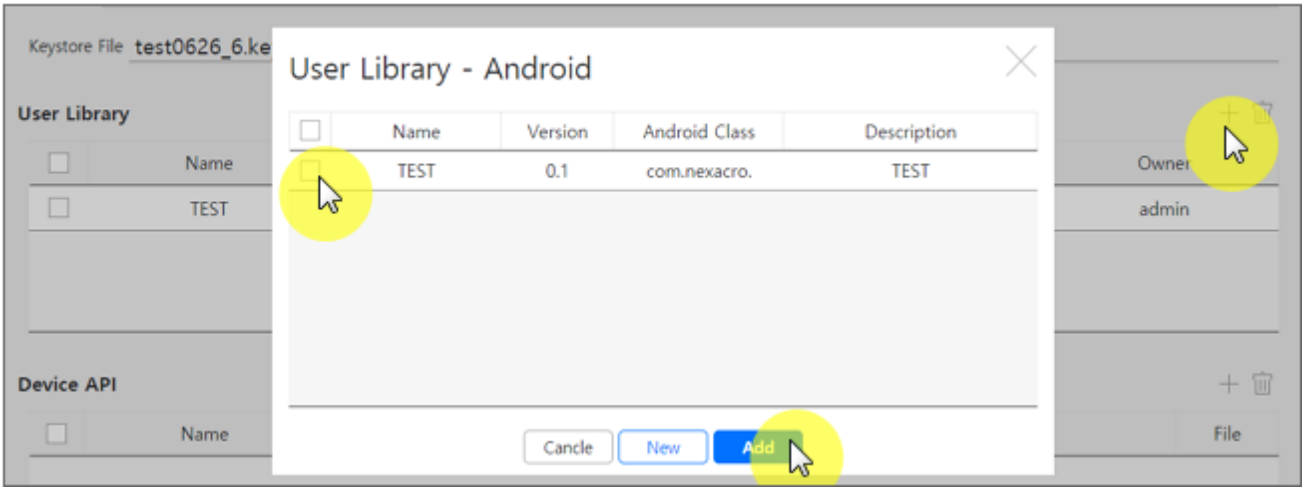
If you click the Import button, the below dialog will appear to prompt you to select either Yes or No. Your selection will decide how to import a modified project.



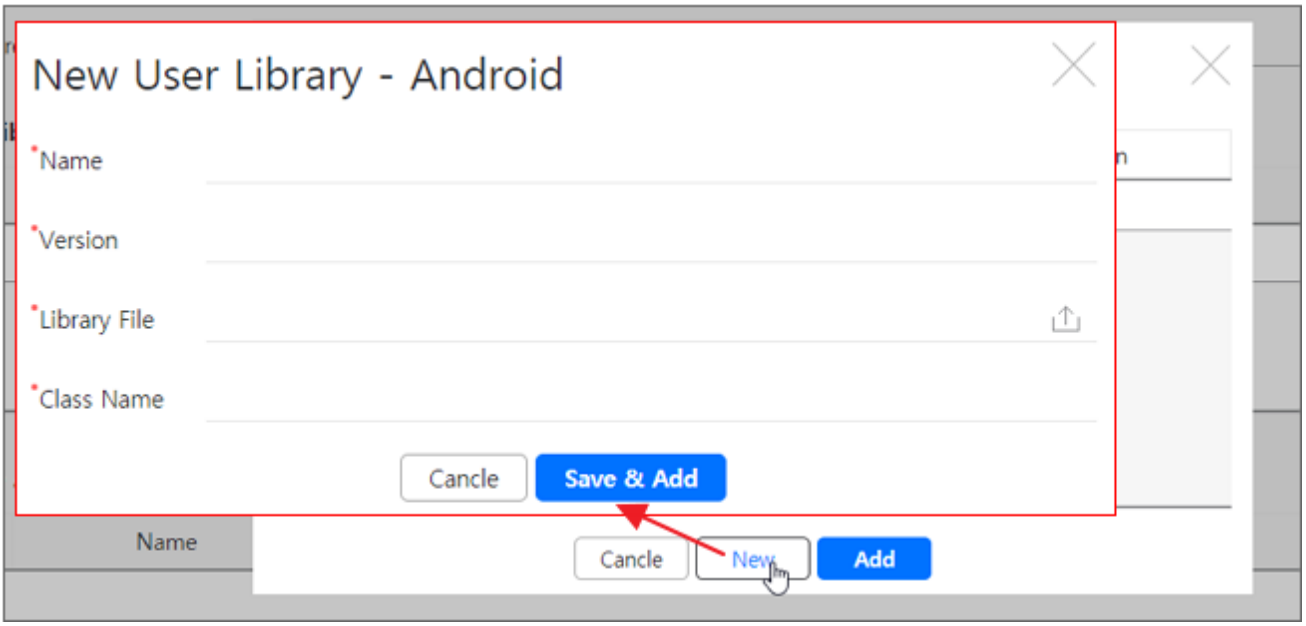
	Selection	Description
1	YES	Maintains the state of the imported project, which has been edited through another tool. - Deactivates the configuration windows of each platform of the project. - While you can change the common configuration, the changed settings will not apply to the platform of the imported project.
2	NO	Does not maintain the state of the imported project, which has been edited through another tool. - Activates the configuration windows of each platform of the project. - Apps are built with the configuration specified on the console screen of App Builder applied to the imported project.

3.2.7 Target OS > User Library

You can select a user library that can be used on Android and iOS operating systems or register a new one. On the Options screen, click the [+] button next to the User Library to display a window where you can select a list. After checking the desired library, click the [Add] button.



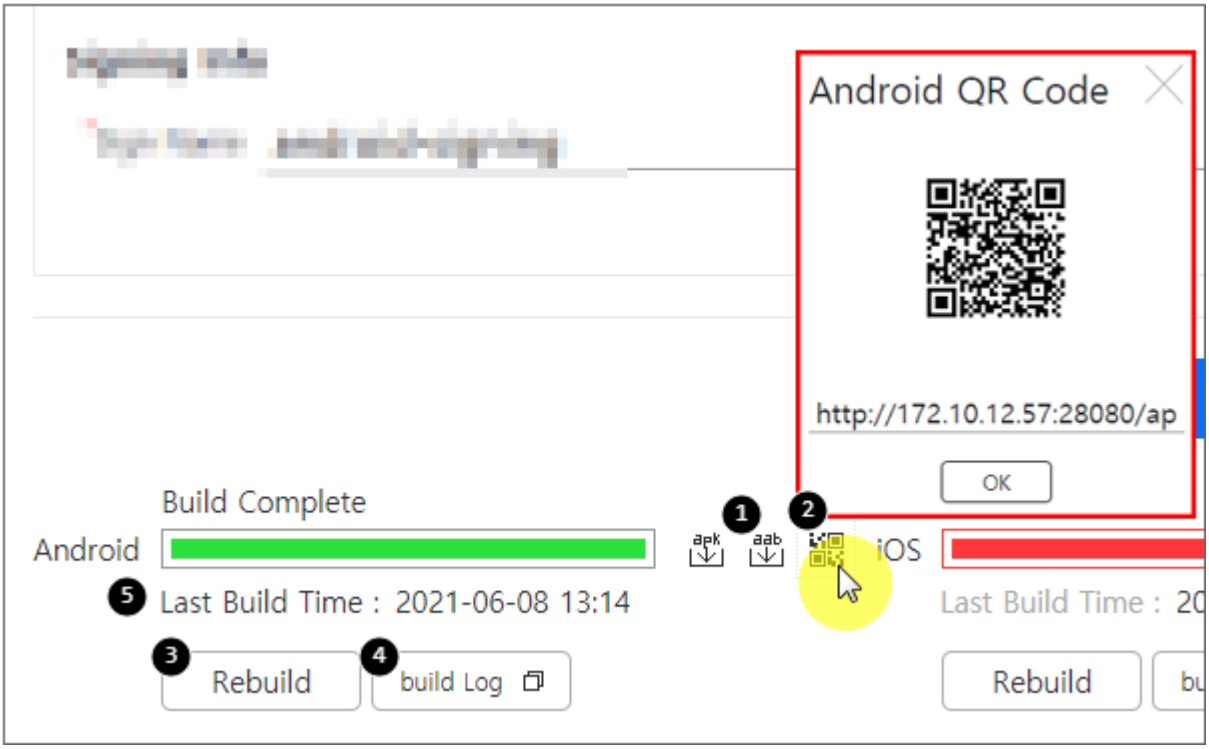
If you need to add a new library, click the [New] button to register a new one.



3.3 Build App

If you try to build apps with a newly created App, the "Start to build" button will appear. When the button is clicked, App Builder starts to build apps for the selected OSes.

When the build operation finishes, results will be displayed. If App Builder successfully builds an app, a link will be created to enable you to download the created app. App Builder allows you to re-build all the apps of selected OSes by clicking the "Rebuild All" button or to re-build a specific app among them.



		Description
1	Download	Allows you to download the installation file of the built app.
2	QRCode	Displays a URL and QR code that allow you to download the installation file of the built app.
3	rebuild	Re-builds the app of the relevant OS.
4	build Log	Allows you to check the logs of the process of building the app.
5	Last Build Time	Displays the time when the app was last built.

If you change settings in the management console, you should click the Save button before building apps. Otherwise, the unsaved change will not be applied when apps are built.

Installation files are named according to the below rules. The "App Tile" and "Build mode" options are set in the App Info stage.
Android: [App Title]-[Build mode].apk, [App Title]-[Build mode].aab
iOS, macOS: [App Title].ipa
eg) sample-debug.apk, sample.ipa

App names displayed on mobile devices follow just the value of the App Title option.

You may face a problem if users try to download your app through Google Chrome by scanning a QR code provided by your app. When users scan the QR code to enter the URL for downloading your app, "downloadFile.do" or "downloadFile.htm" is written in the address bar of Chrome. However, the download will not be performed. The below measures are solutions to such a problem.
- Clear cache in Google Chrome
- Update Google Chrome to the latest

The QR codes are not supported in the following cases.

- macOS
- When the iOS Provisioning profile type is "app-store"
- Android App Bundle(.aab) file

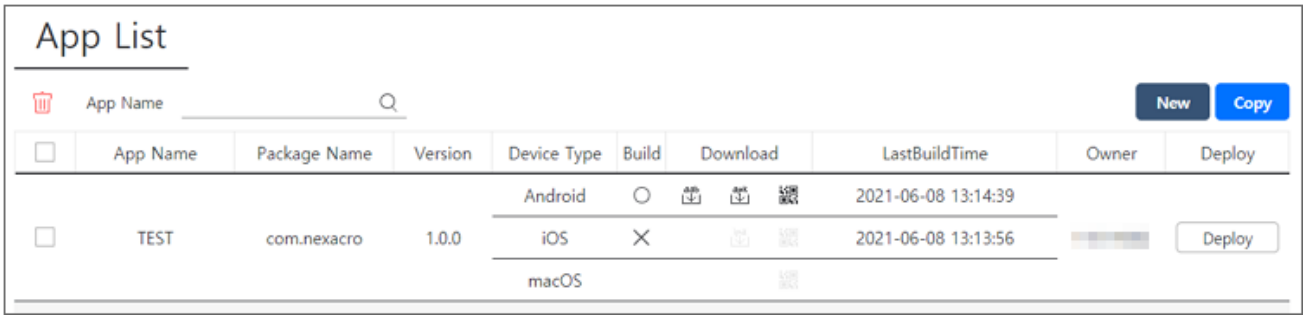
4.

App List

App Builder related articles

- **Server Setup Guide > App Builder**
  App Builder Installation
  Server Configuration
  Deploy Server Setting
  Signature Setting by OS
  User Library Settings
  Nexacro Library Settings
  User Settings
- **Deployment Guide > App Builder**
  Create App
  App List
- **Development Tools Guide > Build App- Create Installation Files**
  Build App- Create Installation Files

You can see the list of created Apps by navigating to the menu [BuildApp > App List] in the management console. The App List presents only the Apps that you are authorized to access as a logged-in user.



While the list indicates whether each app has been built or not, it also provides download links for the successfully-built apps. If you select an App and click the Copy button at the top right side of the console, App Builder will create a copy of the selected App with the name "[App_Name]_copy".



If you copy an App by clicking the Copy button, you cannot change the name of the created App.

Part IV.

iOS/Android/macOS App

5.

App Development and Execution (iOS/iPadOS - Xcode)

5.1 Establish App Development Environment

iOS apps are developed with Xcode and the iOS SDK, which both can run not in Windows computers but in Mac computers. Moreover, you need an Apple developer account to download Xcode, develop an app and test it.

Basically, you should follow the below steps to establish the environment for developing an iOS app. The detailed process may vary according to the updates on the developer site.

1. Create an Apple developer account
2. Install Xcode and the iOS SDK
3. Necessary tasks for distribution



This chapter was written based on the 15.0 version of Xcode.

5.2 Develop App Project

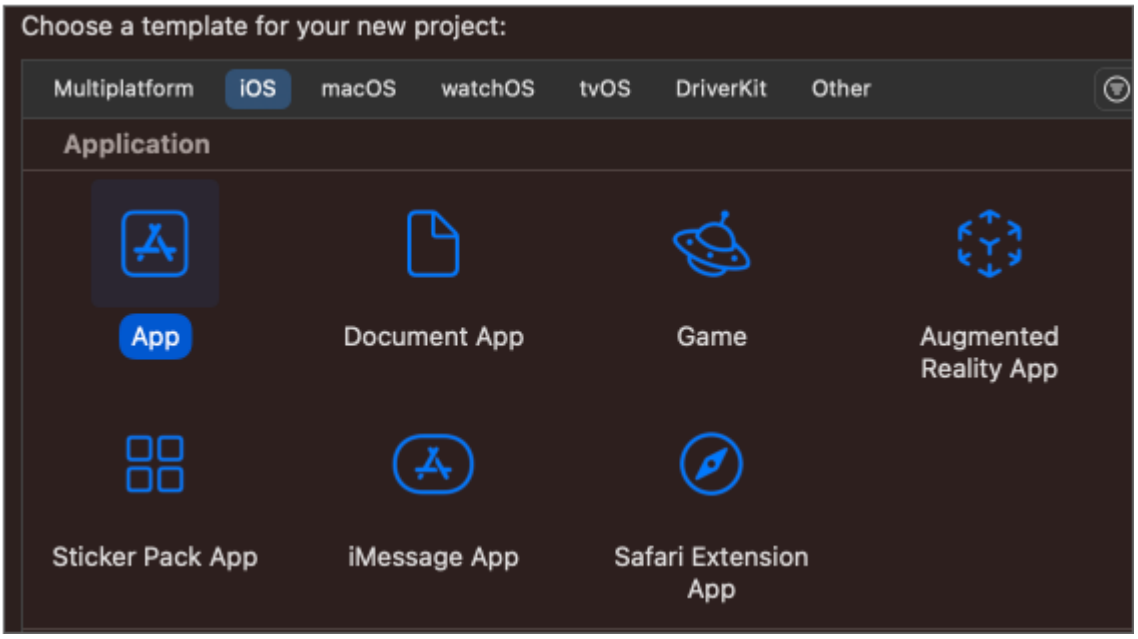
Follow the below steps to work on an iOS app project in Xcode. Before entering an app project, you should find the archive files that you built in Nexacro Studio and locate them in the designated directory.

5.2.1 Create a project

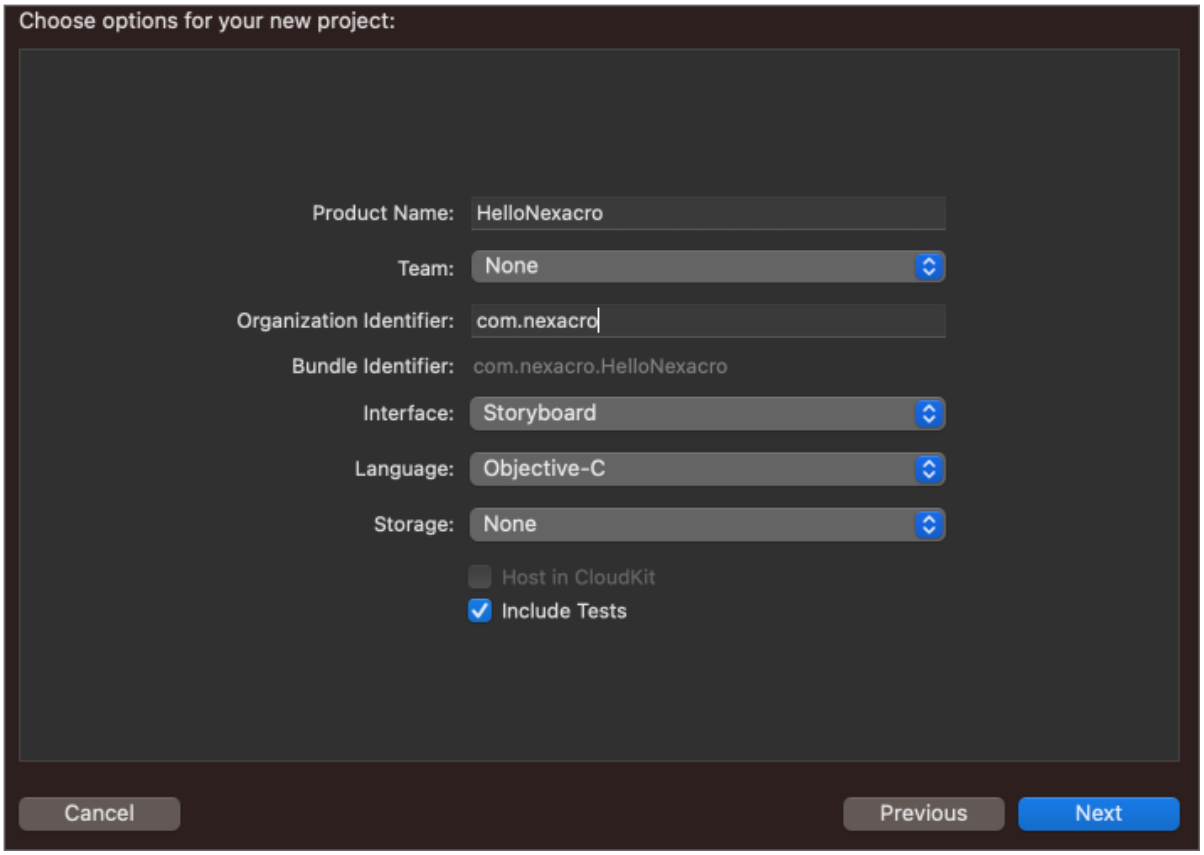
After creating an iOS project that will contain your Nexacro-based app, configure basic settings. You can create a project by accessing the below menu.

```
File > New > Project
```

Select App as a template for the created project.



After deciding on the options for your new project such as the product's name, click the Next button to open a pop-up. Click the Create button in the pop-up after designating the location of the new project and then you will finally see the new project created.



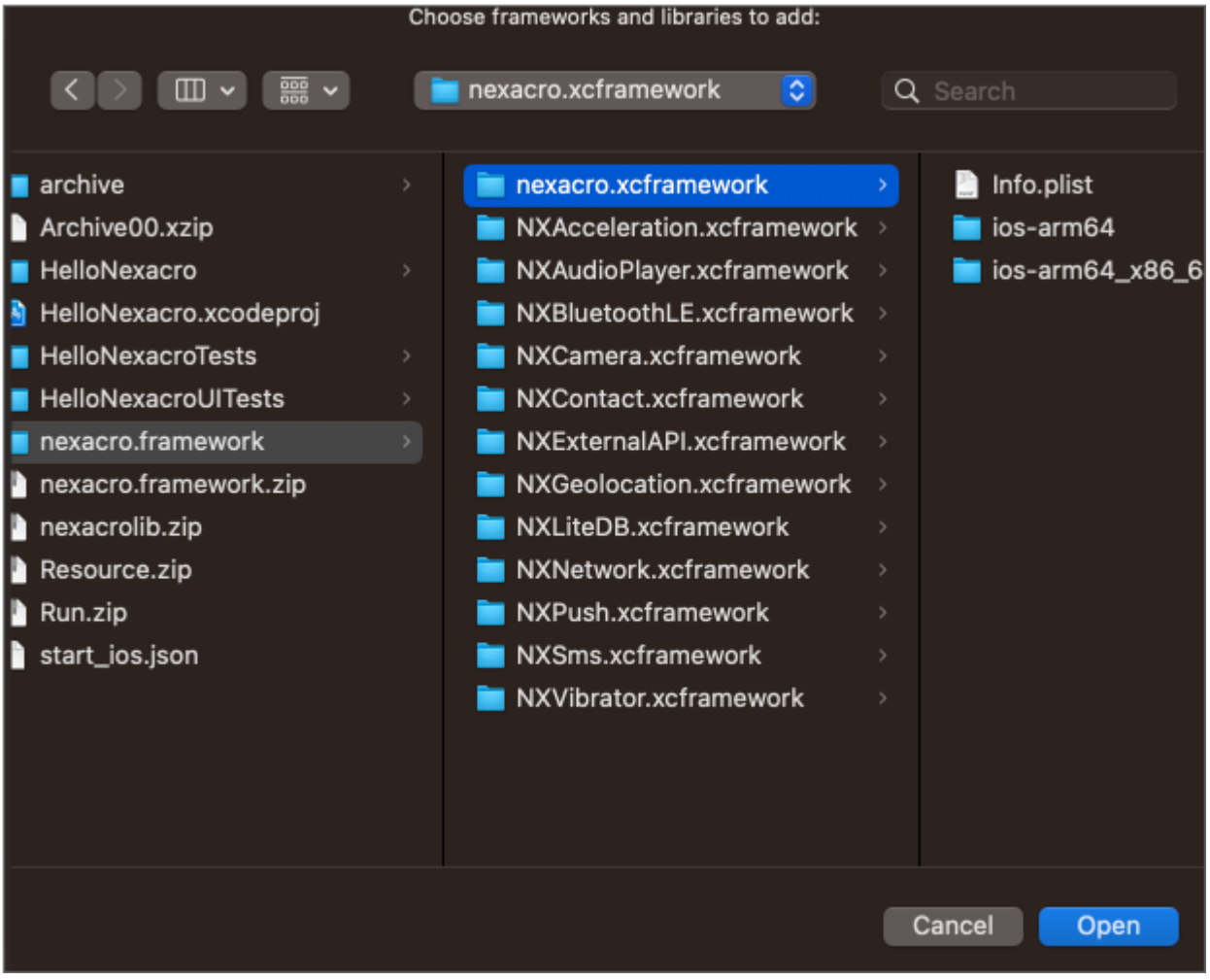
A project may be created in a different way depending on the versions of Xcode.

If you use iPhone X, you should add a Launch Storyboard to your project to prevent blanks at the top and bottom of the screen.
Click on the below links for the detailed information.
https://developer.apple.com/ios/update-apps-for-iphone-x/
https://developer.apple.com/ios/human-interface-guidelines/icons-and-images/launch-screen/

5.2.2 Set Nexacro libraries

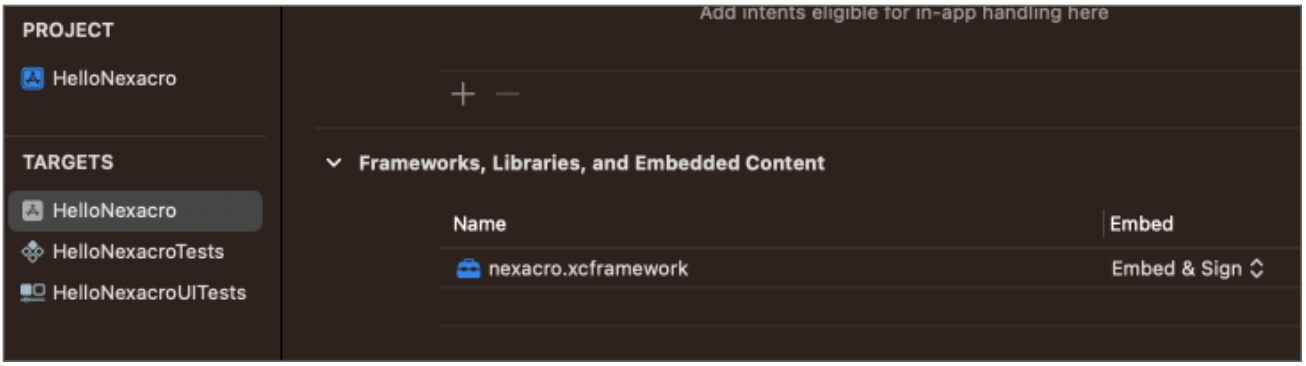
Add Nexacro libraries to your iOS project to establish an optimal environment for Nexacro.

The Nexacro library is provided as a compressed file with the filename, nexacro.framework.zip. First, unzip the file. Then, click the [+] button in Frameworks, Libraries, and Embedded Content and select Add Files. Then, select the file under the nexacro.framework folder to add it.



The default library, nexacro.xcframework, must be configured, while the rest of the items need to be added as needed based on the Device API used by the app.

Library	Required
nexacro.xcframework	Required
NXAcceleration.xcframework NXAudioPlayer.xcframework NXBluetoothLE.xcframework NXCamera.xcframework NXContact.xcframework NXExternalAPI.xcframework NXGeolocation.xcframework NXLiteDB.xcframework NXNetwork.xcframework NXPush.xcframework NXSms.xcframework NXVibrator.xcframework	Added selectively depending on the Device API used by the app



5.2.3 Set resources

In this stage, you can set launch images, icons, messages and layouts applied to your app. You can adjust those options according to projects.

Set images

A project in Xcode created with a template contains an empty asset catalog, which is used to manage resources. You can add app icons, launch images, etc. to the catalog.

You can individually set the app icons and launch screen images used in an iOS app. Moreover, you can bring images that are made already.

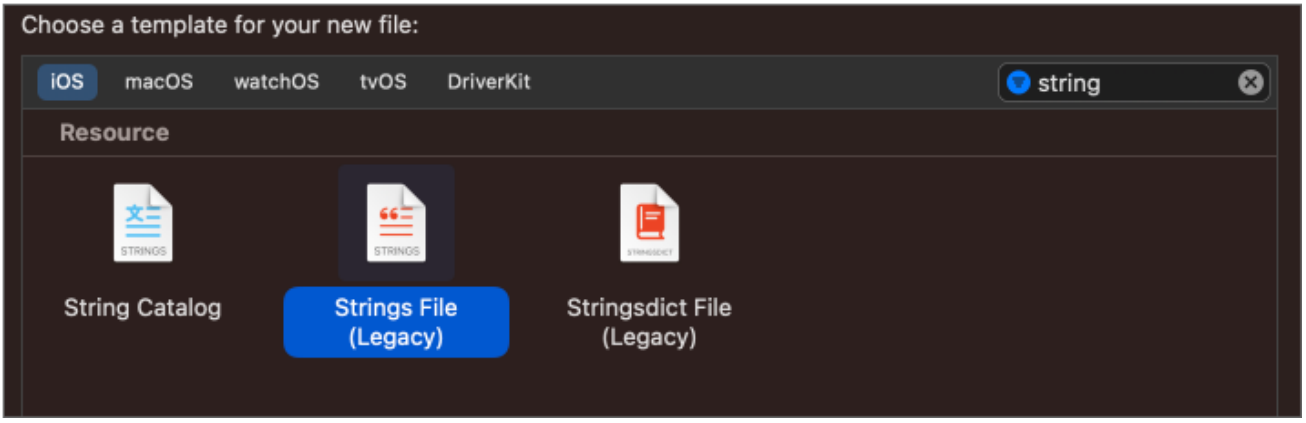
You must use images that are designed appropriately for the iOS devices you target. Assume that you set an image whose size does not fit target devices or set an image with a property incompatible with target devices. In those cases, an error will occur in the build process or your app will not work properly even if it is built.
Click on the below link to visit the Apple developer web page that provides the details on the settings of icons and images.
https://developer.apple.com/documentation/xcode/configuring-your-app-icon

Set messages

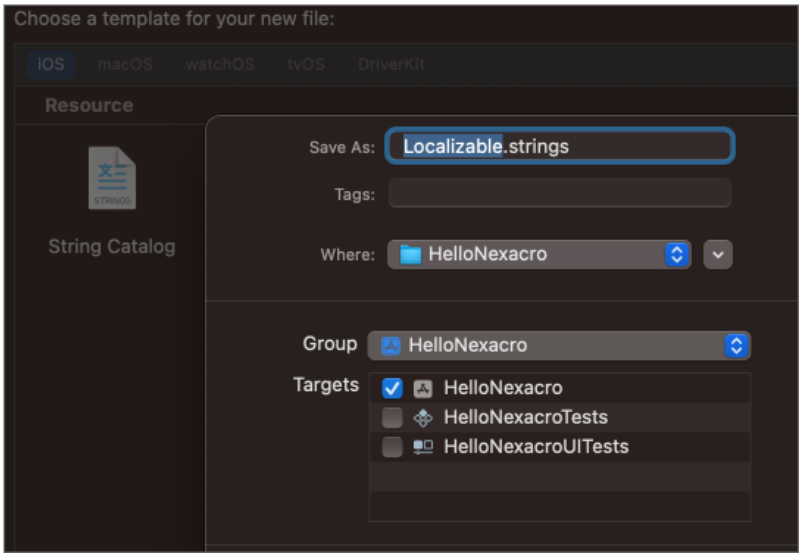
The default messages used by the app are included in the Nexacro library. You can set your own message if you want to redefine the default messages.

To create a new file, select the project folder and select [File > New > File] from the menu.

Choose "Strings Files" from a selection of templates.



In the "Save As" field, make sure the file is named "Localizable.strings."

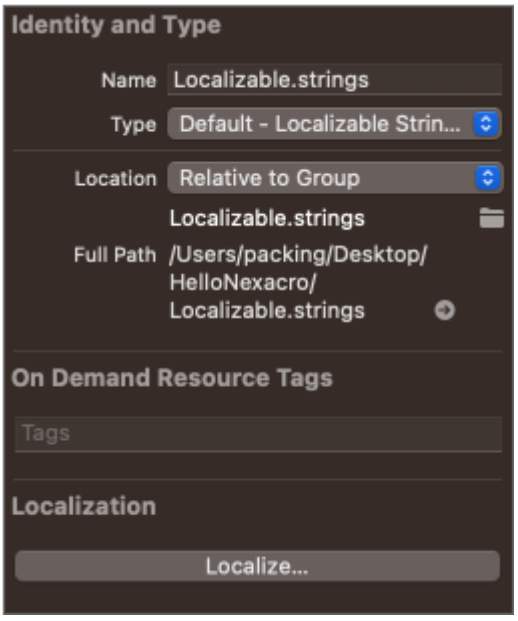


Modify the created *Localizable.strings* file as presented below.

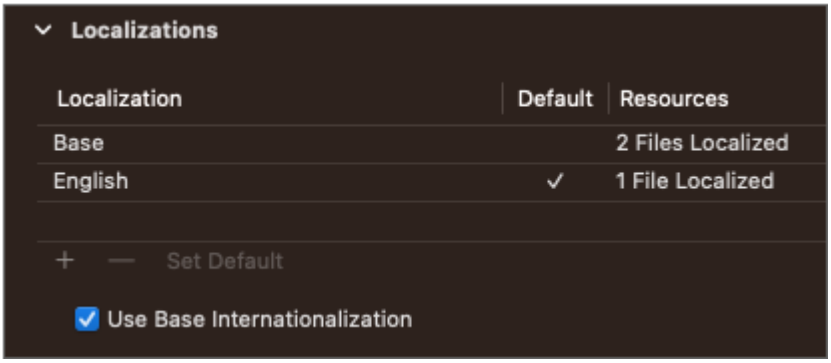
```
/*
 Localizable.strings english
 nexacro
 */

 "needupdate" = "It is need to update. \r\nIt will start after the update is
completed.";
 "loadingFail" = "First loading is fail. \r\nPlease restart.";
 "updateFail" = "Update is fail. \r\nPlease restart.";
 "notexist" = "Start file is NOT exist. \r\nPlease restart.";
 "BeingUpdated" = "Being updated.";
 "ok" = "OK";
 "cancel" = "Cancel";
 "alert_title" = "";
 "checkforupdates" = "Check for updates.";
 "installforupdates" = "Install for updates.";
 "downloadingforupdates" = "Downloading for updates.";
 "new_engine_available" = "Engine update required.";
```

In the Utilities pane on the right side of the Xcode screen, click "Localize..." button in the Localization section to select the language for your app. If you do not add another language in advance in the project's settings, the language will be English by default.



To support languages other than English, navigate to the menu [PROJECT > Info > Localizations] and click + button to add another language.



Click on the below link to visit the Apple developer web page that provides the details on localization.
https://developer.apple.com/documentation/xcode/localization

5.2.4 Set Config

Set the *nexacro_config.xml* file to apply the functionalities provided by Nexacro to your app. You can activate features including app updating, notification and error processing. You should create the *nexacro_config.xml* file by yourself and locate the file under the folder [{Asset} > data]. This file should be included in the project when your app is built in Xcode.

Even if the *nexacro_config.xml* file does not exist, there will be no problem in running your app because the default configuration will be applied instead.

The *nexacro_config.xml* file is in the XML format and written like the below sample.

```
<?xml version="1.0" encoding="utf-8"?>
<nexacro-config>
  <application dialog-position="top" use-wkwebview="true" file-logging="true" /
>
  <notification enable="true" handler="DefaultNotificationManagerHandler"/>
  <updator force="false" cancelable="false" quiet="true" errormsg="true"
failpass="true"/>
  <xpush-server request-missing-message="true" />
</nexacro-config>
```

The below table shows the features that you can set in *nexacro_config.xml*.

Features	Property	Value	Description
application	dialog-position	"top" | "center" | "bottom"	This property sets the position of a pop-up for showing the progress of app updating.
	file-logging	"true" | "false"	This property determines whether to log loading errors in the form of a file. You can check the log file through iTunes.
updator	force	"true" | "false"	This property determines whether to display a pop-up for announcing the existence of an update, contained in *start_ios.json*. If "true", updating will be conducted automatically without pop-up announcement.
	cancelable	"true" | "false"	This property determines whether to display a "Cancel Update" button in a pop-up for announcing the existence of an update. If you click the "Cancel" button, the updating will not be conducted and the app will be closed.
	errormsg	"true" | "false"	This property determines whether to display a pop-up for showing the error information when the loading of the app fails.
	quiet	"true" | "false"	This property determines whether to display a pop-up that asks you whether to update the app. Even if you specify

Features	Property	Value	Description
			"true", the progress of app updating will be displayed when there is an update file.
	failpass	"true" | "false"	Set whether to ignore the update errors or not
xpush-server	request-missing-message	"true" | "false"	This property determines whether to automatically request reliable messaging messages that you have not received from an XPush server.
notification	enable	"true" | "false"	This property determines whether to use the notification feature.
	handler	"[class name]"	This property sets the class to define a handler for processing notification. The default value is “DefaultNotificationManagerHandler”.

5.2.5 Set build environment

You should modify *AppDelegate.h*, *AppDelegate.m* and *main.m*—default files created in the process of creating an iOS project—in a way that suits Nexacro.

AppDelegate.h

Modify the created *AppDelegate.h* file as below so that the file inherits the Nexacro's AppDelegate class. Then, the nexacro framework will work properly.

```
//
//  AppDelegate.h
//  HelloNexacro
//
```

```
#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface AppDelegate : NXAppDelegate


@end
```

AppDelegate.m

In the *AppDelegate.m* file, you should specify the URL of the *start_ios.json* file located in the server. The JSON file is created with Nexacro Studio. Replace the portion written in bold in the below code listings with something appropriate for your settings.

```
//
//  AppDelegate.m
//  HelloNexacro
//

#import "AppDelegate.h"
#import "ViewController.h"

@interface AppDelegate ()

@end

@implementation AppDelegate

- (BOOL)application:(UIApplication *)application
    didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {
    self.viewController = [[ViewController alloc] init];

    // Update Type: Server, Update
    self.viewController.projectUrl = @"http://[URL]/_ios_/";
    self.viewController.bootstrapUrl = @"http://[URL]/_ios_/start_ios.json";

    // Update Type: Local
    // self.viewController.bootstrapUrl =
@"file:///ios_asset/archive/start_ios.json";

    return [super application:application
```

```
didFinishLaunchingWithOptions:launchOptions];
}

@end
```

main.m

UIApplicationMain is a default function provided by the UIKit framework, serving as a starting point of an app. This function gets the app ready to receive inputs from the user by preparing for the operation of the app and setting up the main event loop.

After checking whether the *main.m* file is written as below, modify the file if different.

```
//
//  main.m
//  HelloNexacro
//

#import <UIKit/UIKit.h>
#import "AppDelegate.h"

int main(int argc, char * argv[]) {
    NSString * appDelegateClassName;
    @autoreleasepool {
        // Setup code that might create autoreleased objects goes here.
        appDelegateClassName = NSStringFromClass([AppDelegate class]);
    }
    return UIApplicationMain(argc, argv, nil, appDelegateClassName);
}
```

ViewController.h

```
//
//  ViewController.h
//  HelloNexacro
//

#import <UIKit/UIKit.h>
#import <nexacro/NXViewController.h>
```

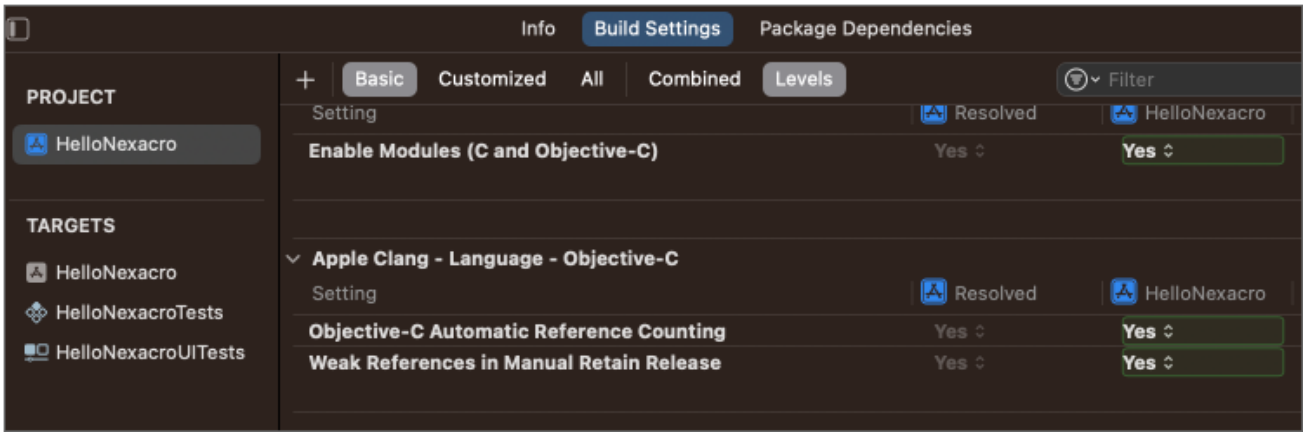
```
@interface ViewController : NXViewController

@end
```

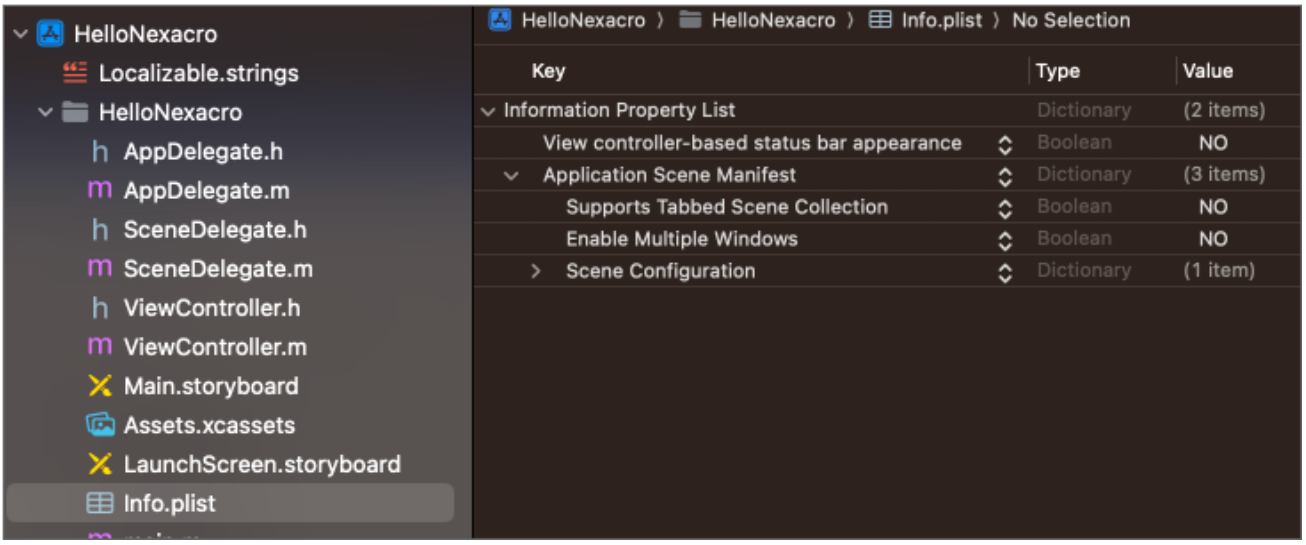
Other settings

- In the "Build Settings" field, check the settings and modify them if necessary.

Build Settings	Item	Value
Architectures > Build Active Architecture Only	Debug	No
	Release	No
Apple Clang - Language - Objective-C	Objective-C Automatic Reference Counting	Yes

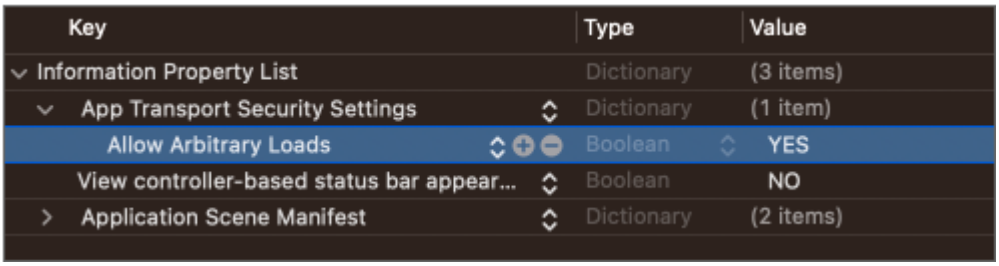


- Select the Info.plist file, add "View controller-based status bar appearance" from the Information Property List, and set the value to "NO."



- In case of setting the projectUrl and bootstrapUrl path as HTTP, select the Info.plist file, add "App

Transport Security Settings," add "Allow Arbitrary Loads" underneath, and set the value to "YES."



5.2.6 Set your app stand-alone in disconnected environments

When packing an app in Nexacro Studio, if the Update Type is set to Local or Update (Local+Server), the app can be run without a network connection. In general, the Local type is used when a network connection is not required, and the Update (Local+Server) type is used when you need to run the app with a local file after running the app while connected to the network. To enable the app to run without a network connection or independently there is a problem with the network, the files necessary to run the app must be loaded and deployed when building the app.

The files necessary for executing an app include modules, sources, resources, *Run.zip* and *start_ios.json*, which are all obtained through the Packing (Archive&Update) process of Nexacro Studio. If you load those files onto the "archive" folder in your Xcode project before building an app, you can run the app without a network connection.



Even if you set the update type to "Local" in the Packing (Archive&Update) process of Nexacro Studio, you must specify a bootstrap URL in the *AppDelegate.m* file of your Xcode project.
For example, you can specify a random URL like the below one, which is either virtual or unused actually.

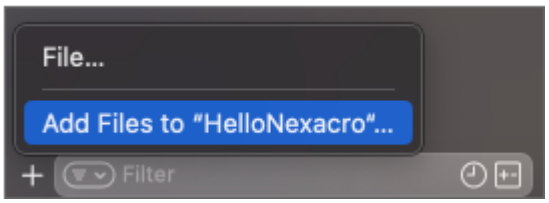
```
NSString *bootstrapUrl = @"http://192.168.0.1/_ios_/start_ios.json";
```

1. Create an "archive" folder in Finder and copy module, source and resource files to the folder.

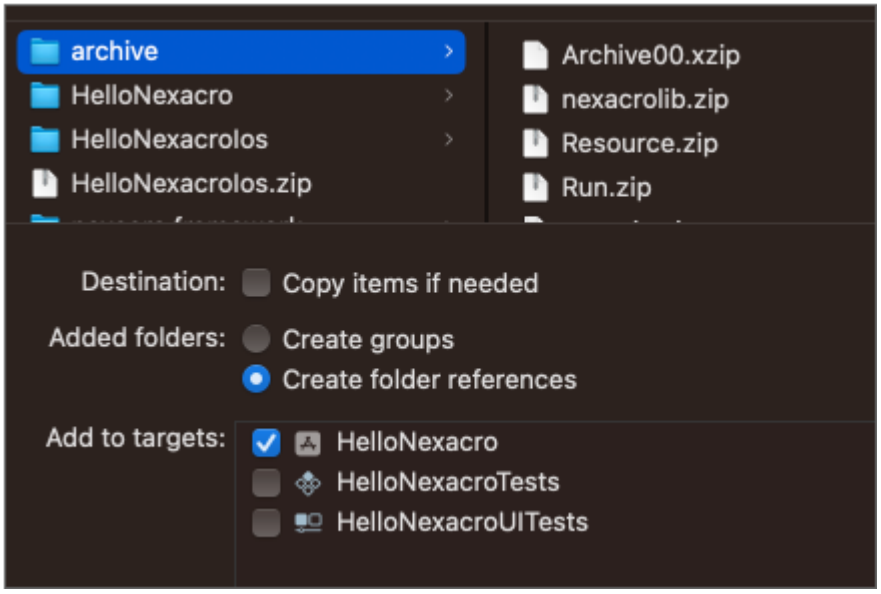
   Launch Finder and create an archive folder at the lowermost part of the Xcode project path. Then, copy the files created by [Packing (Archive&Update)] in Nexacro Studio to the archive folder.

2. Add the archive folder to your Xcode project.

   Click the plus (+) button at the bottom of Xcode's project navigator and select Add Files to "project name".

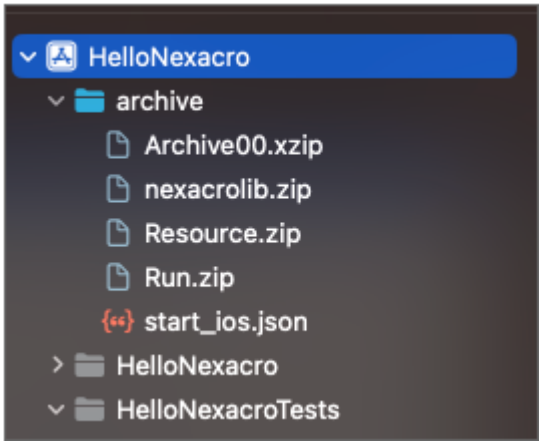


Select the archive folder and choose "Create folder references" as an "Added folders" option. Then, click the Add button.



3 Check whether the folder and files have been added to your project successfully.

Go to the project navigator of Xcode and check whether the archive folder and the files under the folder have been added successfully.



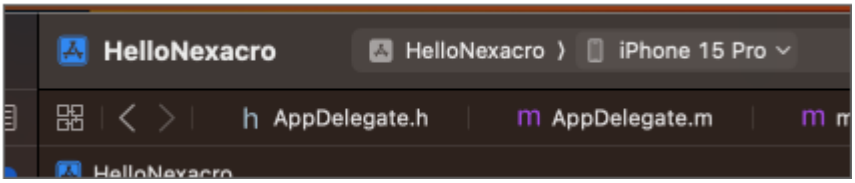
Folders in Xcode's project navigator are displayed in either yellow or blue. Yellow folders are nonexistent since they are just logical groups that serve as an indicator of the project's organization. By contrast, blue ones are actual Finder folders. After an app is built, files in

> yellow folders are included in the app's root directory while files in blue folders are contained in their respective folders with the folder structure unchanged.

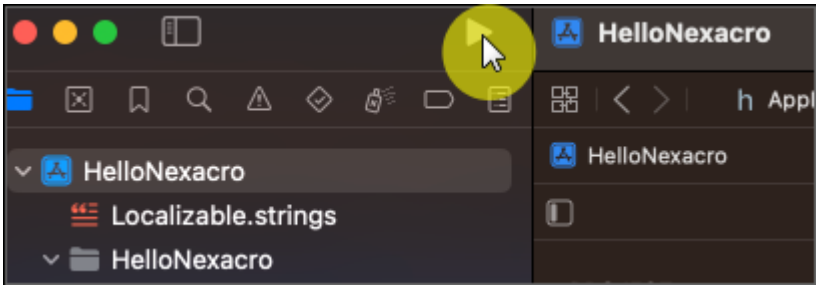
5.3 Test App

You can test your app by connecting an iOS device to your PC using a USB cable or by using a simulator. If you use a physical device, you should install an appropriate profile on the device.

1. As seen below, access the Scheme menu at the top of the Xcode screen and choose a destination—a connected iOS device or an iOS simulator where your app will be tested.



2. Run your app by clicking the Run button at the top of the Xcode screen or accessing the Run menu.



5.4 UIScene Configuration (New)

> Support for UIScene settings was added starting from version 24.0.0.1000.
> The contents of this document function correctly only in versions after 24.0.0.1000.
> To configure UIScene, an environment with iOS 13.0 or higher and Xcode 14.0 or higher is required.

The project creation, framework connection, and build environment setup are conducted in the same manner

as the development of existing app projects.

5.4.1 Configuring SceneDelegate

When a new project is created, files are automatically generated. Edit the generated file as follows.

SceneDelegate.h

```
#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface SceneDelegate : UIResponder <UIWindowSceneDelegate>

@property (strong, nonatomic) UIWindow *window;

@end
```

SceneDelegate.m

```
#import "SceneDelegate.h"

@interface SceneDelegate ()

@end

@implementation SceneDelegate

- (void)scene:(UIScene *)scene willConnectToSession:(UISceneSession *)session
options:(UISceneConnectionOptions *)connectionOptions {
    // Cast to UIWindowScene
    UIWindowScene *windowScene = (UIWindowScene *)scene;

    // Calling configureWindowForScene in the framework
    NXAppDelegate *appDelegate = (NXAppDelegate
*)UIApplication.sharedApplication.delegate;
    [appDelegate configureWindowForScene:windowScene];

    // Setting the window reference in SceneDelegate
```

```
    self.window = appDelegate.window;
}

- (void)sceneDidDisconnect:(UIScene *)scene {
}

- (void)sceneDidBecomeActive:(UIScene *)scene {
}

- (void)sceneWillResignActive:(UIScene *)scene {
}

- (void)sceneWillEnterForeground:(UIScene *)scene {
}

- (void)sceneDidEnterBackground:(UIScene *)scene {
}

@end
```

5.5 UIScene Configuration (Migration)



Support for UIScene settings was added starting from version 24.0.0.1000.
The contents of this document function correctly only in versions after 24.0.0.1000.
To configure UIScene, an environment with iOS 13.0 or higher and Xcode 14.0 or higher is required.

This section describes the necessary steps to transition an existing app project to the UIScene method.

	Existing Method	UIScene Approach
App Lifecycle	UIApplicationDelegate	UIWindowSceneDelegate
Window Management	AppDelegate	SceneDelegate
Multi Window	Not supported	Support (iPad)
iOS Support	Before iOS 13	iOS 13 or later
Future Support	End of Support Planned	Required

5.5.1 Framework Update

Check if the configureWindowForScene method is declared in the NexacroAppDelegate.h file. Updates the framework if the method is not present.

```
- (void)configureWindowForScene:(UIWindowScene *)windowScene API_AVAILABLE(ios(13.0));
```

5.5.2 SceneDelegate Configuration

Create the SceneDelegate.h and SceneDelegate.m files in the project folder. After editing the files, add them to the project as follows.

1. Xcode > File > Add Files to "YourProject"...
2. Select SceneDelegate.h, SceneDelegate.m files
3. Add Click

SceneDelegate.h

```
#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface SceneDelegate : UIResponder <UIWindowSceneDelegate>

@property (strong, nonatomic) UIWindow *window;

@end
```

SceneDelegate.m

```
#import "SceneDelegate.h"

@interface SceneDelegate ()

@end

@implementation SceneDelegate
```

```
- (void)scene:(UIScene *)scene willConnectToSession:(UISceneSession *)session
options:(UISceneConnectionOptions *)connectionOptions {
    // Cast to UIWindowScene
    UIWindowScene *windowScene = (UIWindowScene *)scene;

    // Calling configureWindowForScene in the framework
    NXAppDelegate *appDelegate = (NXAppDelegate
*)UIApplication.sharedApplication.delegate;
    [appDelegate configureWindowForScene:windowScene];

    // Setting the window reference in SceneDelegate
    self.window = appDelegate.window;
}

- (void)sceneDidDisconnect:(UIScene *)scene {
}

- (void)sceneDidBecomeActive:(UIScene *)scene {
}

- (void)sceneWillResignActive:(UIScene *)scene {
}

- (void)sceneWillEnterForeground:(UIScene *)scene {
}

- (void)sceneDidEnterBackground:(UIScene *)scene {
}

@end
```

5.5.3 Add Scene Settings

Open the Info.plist file in a text editor and add the UIApplicationSceneManifest key.

```
...
<key>UIApplicationSceneManifest</key>
<dict>
```

```
    <key>UIApplicationSupportsMultipleScenes</key>
    <false/>
    <key>UISceneConfigurations</key>
    <dict>
        <key>UIWindowSceneSessionRoleApplication</key>
        <array>
            <dict>
                <key>UISceneConfigurationName</key>
                <string>Default Configuration</string>
                <key>UISceneDelegateClassName</key>
                <string>SceneDelegate</string>
            </dict>
        </array>
    </dict>
</dict>
...
```

5.5.4 Check Info.plist Configuration

Key	Action
UIMainStoryboardFile	Delete or Keep (Does not affect scene settings)
UISceneStoryboardFile	Delete
UILaunchStoryboardName	Keep

6.

App Development and Execution (Android - Android Studio)

6.1 Establish App Development Environment

You can develop a Nexacro-based Android app by using Android Studio, the official app development tool of Android. Otherwise, you can use App Builder, which is one of Nexacro products. This chapter explains how to develop an app by using Android Studio. The process of developing a Nexacro-based Android app is basically the same as that of a usual Android app.



If you have established the app development environment already, you can skip this section, Establish App Development Environment.

The installation file of Android Studio includes a software development kit (SDK). Click on the below link to download the installation file appropriate for your running environment.

> https://developer.android.com/studio/index.html



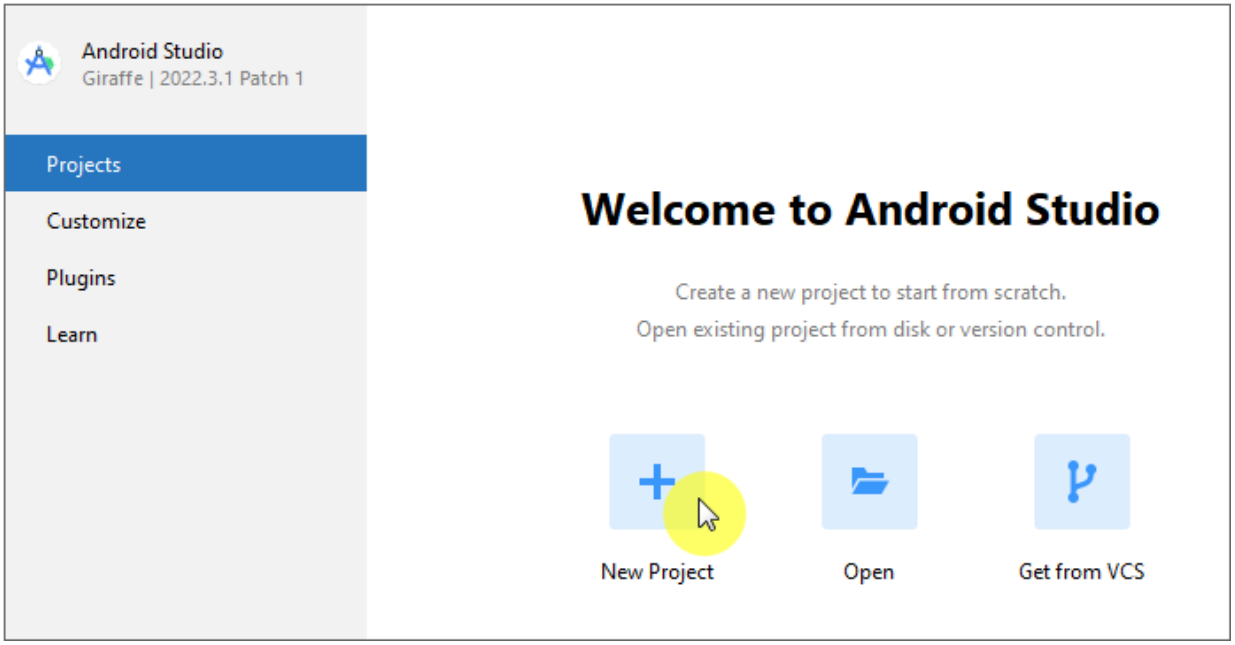
This chapter is written based on the 2023.3.1 version of Android Studio.

6.2 Develop App Project

Once the app development environment is established, open Android Studio to get ready to develop an app running in Android. Before entering an app project, you should find the archive files that you built in Nexacro Studio and locate them in the designated directory.

6.2.1 Create a project

1. Unless you have already opened a project , create a new project in the Welcome to Android Studio window. Click Create New Project as seen in the below picture.

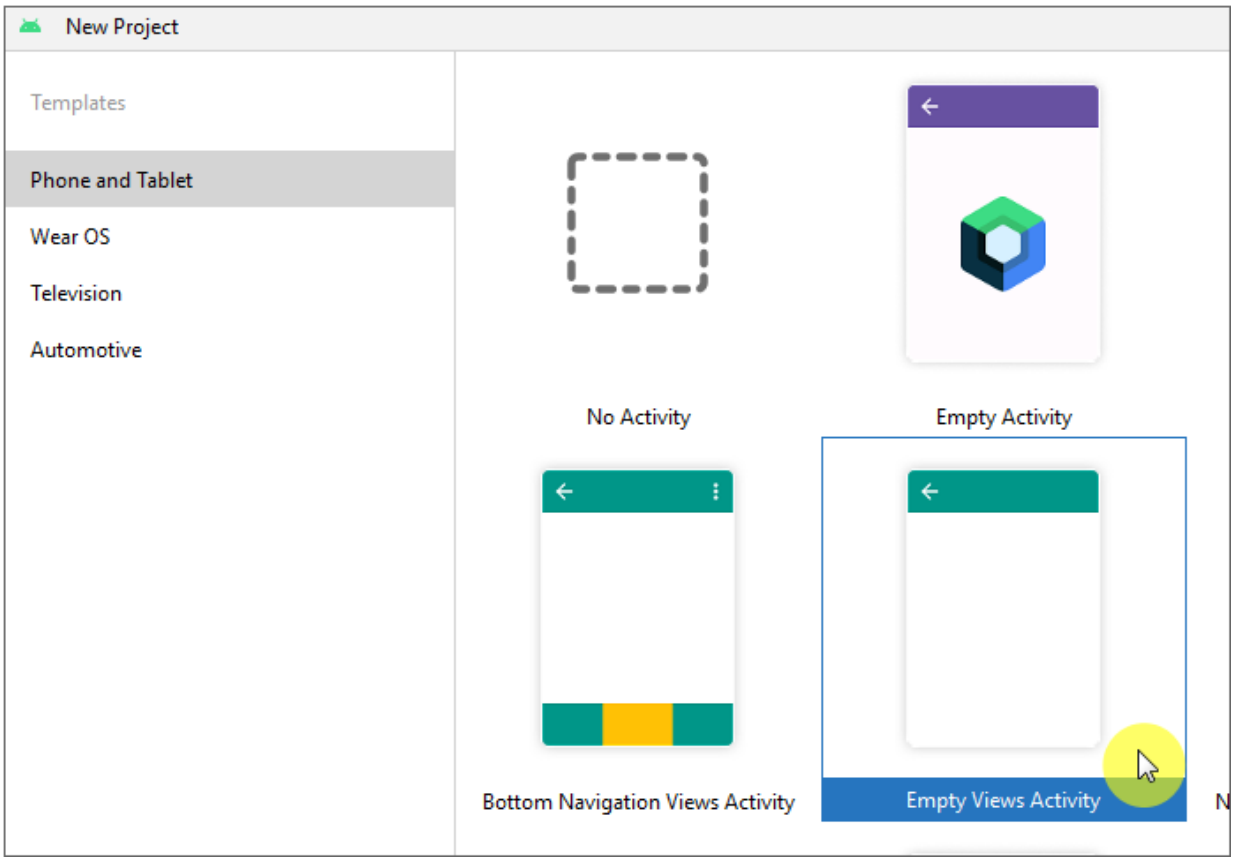


If you want to create a new project with a project already opened, navigate to the below menu.

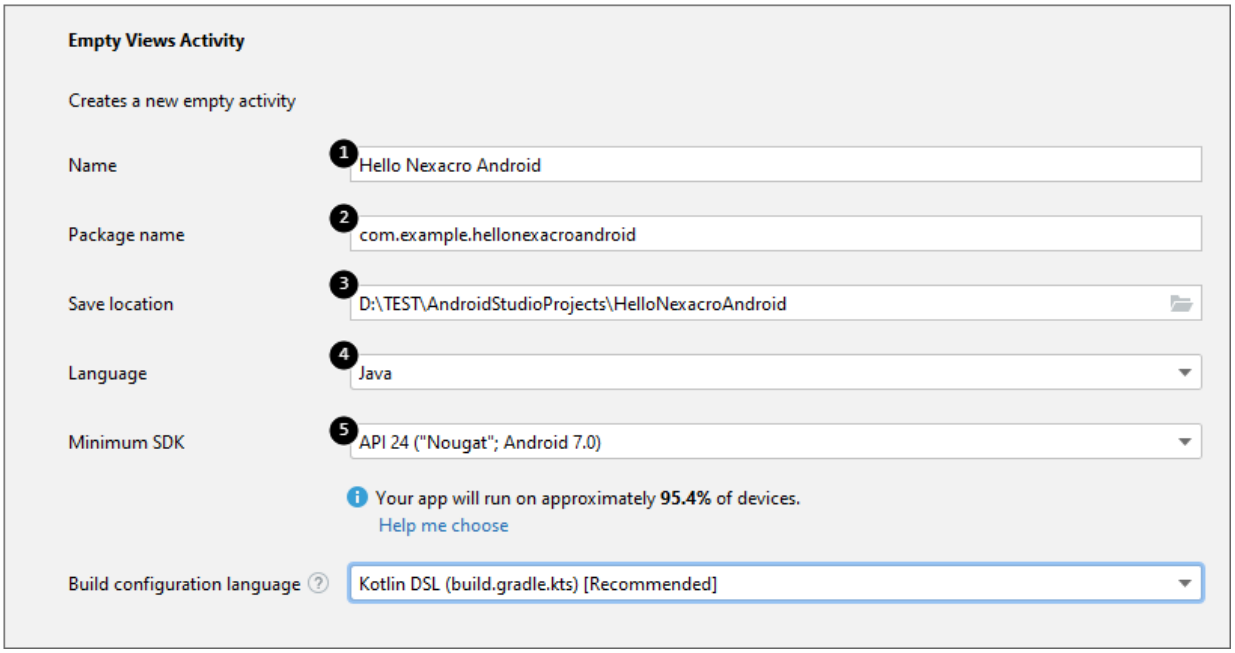
```
File > New > New Project
```

2. Choose a project template provided by Android.

Selection of templates affects an app structure, screen layout as well as the composition of files. Since it is enough for us to have a basic project that just can contain a Nexacro-based app, select Empty Views Activity and click Next.



3 Configure some settings including the app name, package name and project location.



		Description
1	Name	Specify the name of your app.
2	Package Name	Specify the package name.

		Description
❸	Save location	Specify the location where you want to locally store your project.
❹	Language	Select the development language. Although the default language is Kotlin, Java is used in this chapter's sample.
❺	Minimum API	Set the minimum level of an SDK. Whenever you change the SDK level, the information appears below about the device compatibility with the relevant API level.



The minimum level of Android that Nexacro supports is 5.0. Accordingly, you should choose an SDK that supports the API level 21 or higher.

4 As soon as you click the Finish button, the project is created.

As the project window is opened, additional necessary files are downloaded and installed automatically.

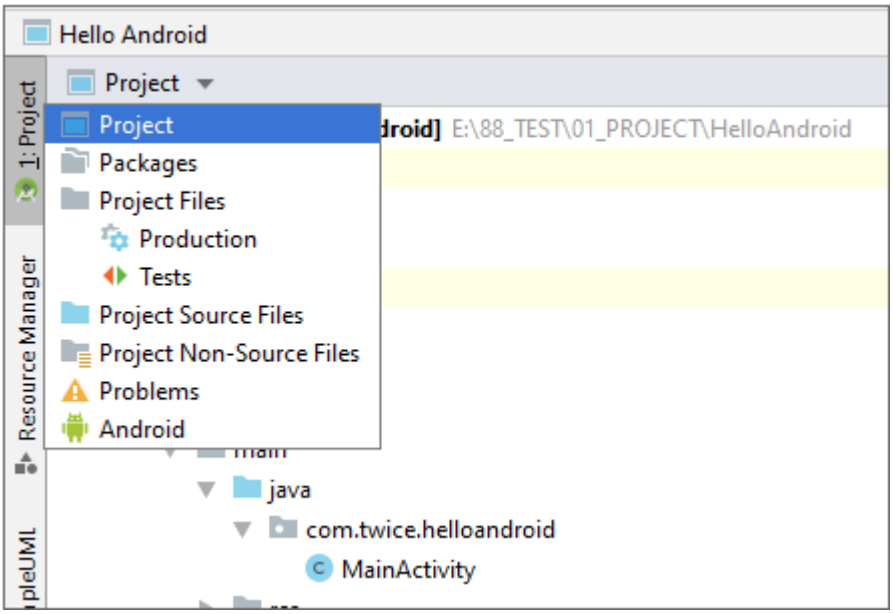
6.2.2 Set Nexacro libraries

Set the Nexacro Android Archive (AAR) file provided for use in app development in the Android project. No setting is required. Just copy it to the designated path.

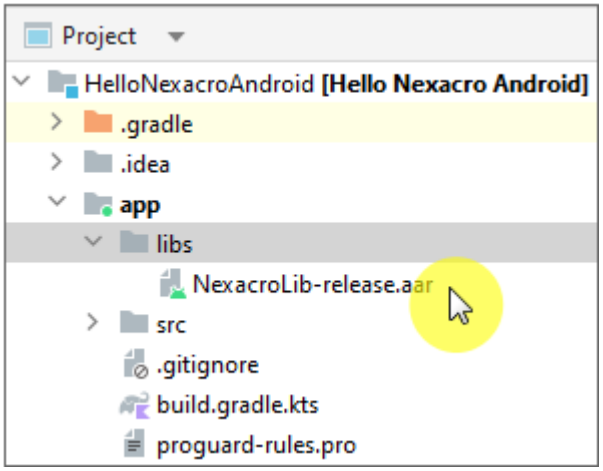
Library File	Path	Requiered
NexacroLib-release.aar	[Project path]\app\libs\	Required
Acceleration-release.aar AudioPlayer-release.aar AudioRecorder-release.aar BluetoothLE-release.aar Camera-release.aar Contact-release.aar ExternalAPI-release.aar Geolocation-release.aar ImagePicker-release.aar LiteDB-release.aar Network-release.aar Phone-release.aar Sms-release.aar Vibrate-release.aar XPush-release.aar	[Project path]]\app\libs\	Added selectively depending on the Device API used by the app

If you want to copy the library files to each designated location in your actual project, you should select the Project view option from the dropdown at the top of the Project window. When you install Android Studio, the view mode is set to Android by default. You should change the view mode to Project to see the actual file

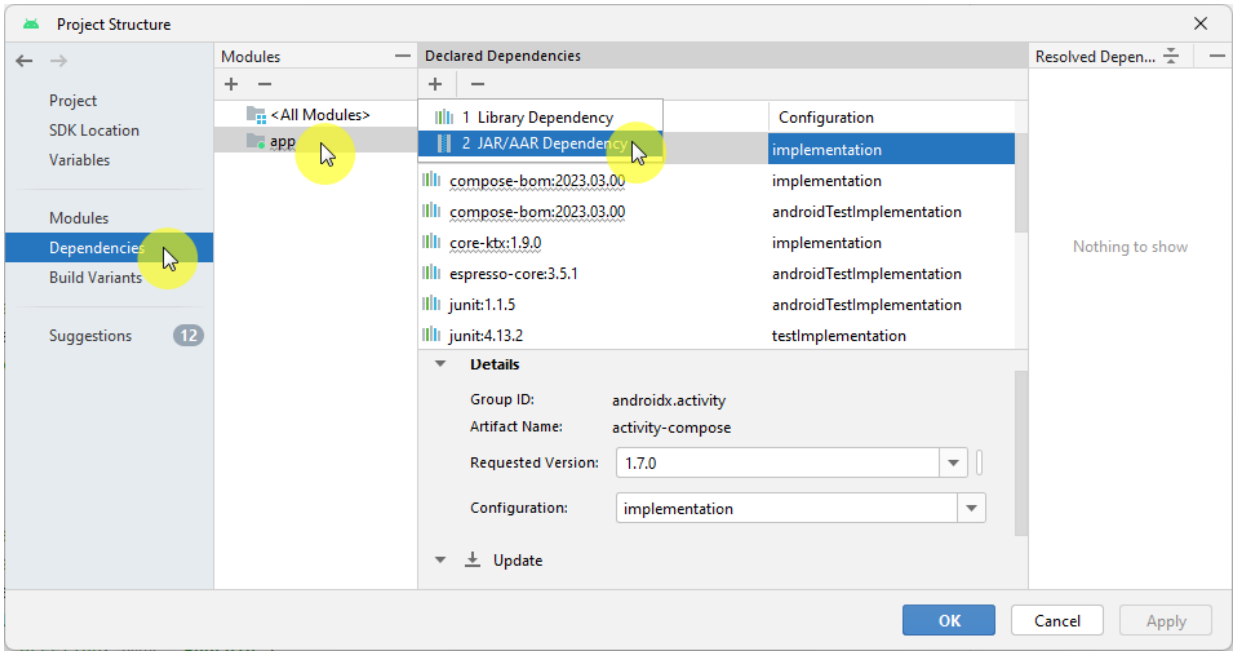
structure of a project.



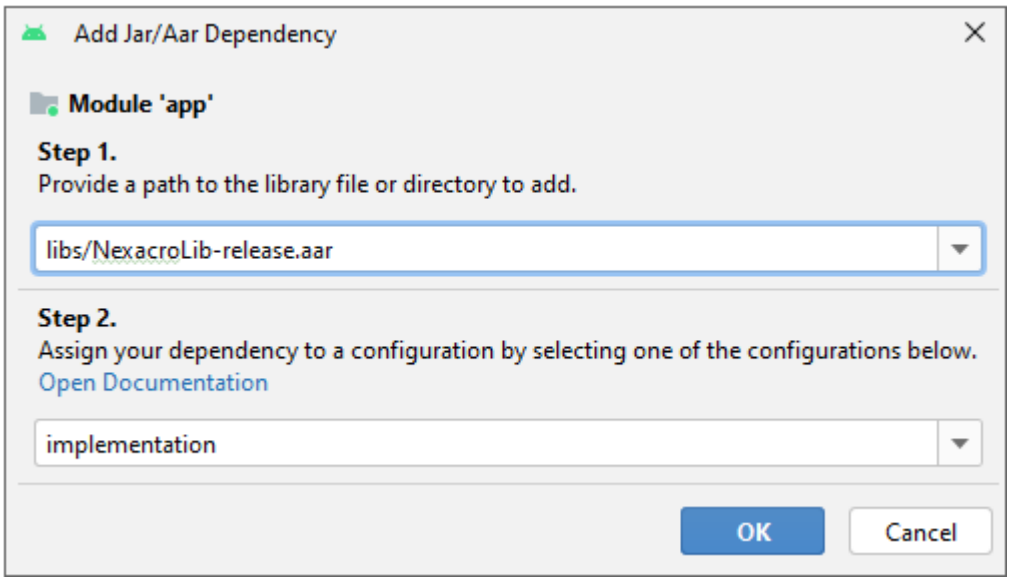
1 Copy the Nexacroib-release.aar file and the required aar files to the [Project path > app > libs] folder.



2 Select "Dependencies" from the menu [File > Project Structure], select "app" from the Modules item, click the "+" button, and select the "JAR/AAR Dependencyy" item.



3 Enter the aar file path in the input window and click the [OK] button.



4 Confirm that the aar file is added to the app > build.gradle.kts file.

```
dependencies { this: DependencyHandlerScope

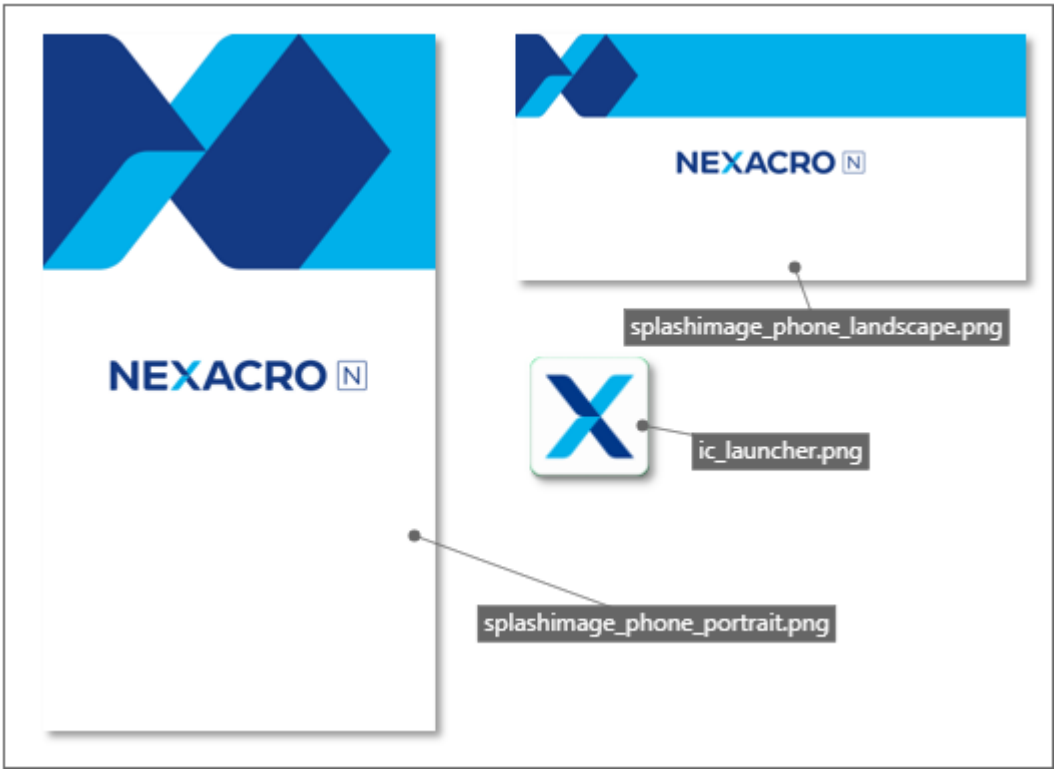
    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.9.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    implementation(files( ...paths: "libs/NexacroLib-release.aar"))
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}
```

For dependency management information, see Dependency Management.

6.2.3 Set resources

In this stage, you can set launch images, icons, messages and layouts applied to your app. The Android system can offer optimal settings for screens if you make a list of resources according to device screen sizes and screen densities and then allocate each of them to appropriate folders.

Basic resources are included in the Nexacro Android Archive (AAR) file. If the default resources are used, no specific setup is required.



See the official Android user guide for further details on resources.

https://developer.android.com/guide/topics/resources/providing-resources

Set images

Images used in an app should be placed in each appropriate directory under the *res* directory according to screen densities. For example, images for a splash screen should be located in *drawable* folders while app icon images are placed in *mipmap* folders. Image files should be put in different folders according to the

resolution levels that are supported by the relevant app.

Images	Description
ic_launcher.png	This is the image of an app icon that appears on the home screen. You can change the name of the image file in the *AndroidManifest.xml* file.
splashimage_phone_landscape.png	This is the image of a splash screen in landscape orientation, applied to an Android phone. (The name must be written in lower case.)
splashimage_phone_portrait.png	This is the image of a splash screen in portrait orientation, applied to an Android phone. (The name must be written in lower case.)
splashimage_pad_landscape.png	This is the image of a splash screen in landscape orientation, applied to an Android tablet. (The name must be written in lower case.)
splashimage_pad_portrait.png	This is the image of a splash screen in portrait orientation, applied to an Android tablet. (The name must be written in lower case.)



Even if you do not set images separately for the launcher icon and splash screen, your app will operate nevertheless because the default settings will be applied instead.

Set strings

The default string settings are applied according to the strings.xml file settings included in the Nexacro Android Archive (AAR) file. To change the default string settings, you can create a strings.xml file in the values, values-ko, values-ja, values-zh folders under the res folder and set only the items you want to change.

The code below is the strings.xml file included in the Nexacro Android archive file. The contents may vary depending on the distributed version

values\strings.xml

```
<?xml version="1.0" encoding="utf-8"?>

<resources>
 <string name="needupdate">It is need to update. \nIt will start after the
update is completed.</string>
 <string name="update">Update</string>
 <string name="exit">Exit</string>
 <string name="updateFail">Update is fail. \nPlease restart.</string>
 <string name="loadingFail">First loading is fail. \nPlease restart.</string>
 <string name="updatedone">Update is Completed.</string>
 <string name="checkforupdates">Check for updates.</string>
 <string name="installforupdates">Install for updates.</string>
 <string name="ok">OK</string>
 <string name="downloadingforupdates">Downloading for updates.</string>
```

```
 <string name="allow_permission">Allow the permission as the permission is
required for the normal function of the engine update.</string>
 <string name="denied_permission">It ends with denial of engine update
permission.</string>
 <string name="cancel">Cancel</string>
 <string name="ssl_invalid">The SSL certificate is invalid. Continue?</string>
</resources>
```

While you can change those strings to suit your situation, you should not change their name attributes because they are predefined for the app.

If you want to add strings written in a non-English language like Korean or Japanese, add a folder under the *res* directory with a name like *values-ko* or *values-ja*. Then, add the *strings.xml* file to the created folder. See the official Android user guide for further details.



https://developer.android.com/training/basics/supporting-devices/languages.html

6.2.4 Set Config

Basic config settings are applied according to the settings of the nexacro_config.xml file included in the Nexacro Android Archive (AAR) file. To change the config settings, create and set the nexacro_config.xml file in the xml folder.

The code below is the content of the nexacro_config.xml file included in the Nexacro Android archive file. The contents may vary depending on the distributed version

xml\nexacro_config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<nexacro-config>
    <application style="default" dialog-position="bottom" file-logging="true"
quiet="false" loglevel="debug" tracemode="none" traceduration="-1"/>
    <notification enable="true" handler=
"com.nexacro.notification.DefaultHandler"/>
    <xpush-server request-missing-message="false" sender-id="1234567890"/>
    <updator cancelable="true" force="true" restart="false" errormsg="true"
quiet="false" failpass="true"/>
    <log filepath="%SD_CARD%nreLog.log" filesize="408960" backupfilecount="5"/>
</nexacro-config>
```

The below table shows the features that you can set in *nexacro_config.xml*.

Features	Property	Value	Description
application	dialog-position	"top" | "center" | "bottom"	This property sets the position of a pop-up for showing the progress of app updating.
	file-logging	"true" | "false"	This property determines whether to log loading errors in the form of a file. If "true", your app will attempt to save a log file first to the external storage (*context.getExternalCacheDir()*). If the external storage is unavailable, the internal storage is the next destination (*context.getCacheDir()*). The directory and file name for saving a log file is as follows: [external/internalstorage] /logs/yyyy_MM_dd.txt.
	quiet	"true" | "false"	This property determines whether to display a pop-up that indicates app launching. If "true", the message "Loading Application" will not be printed.
updator	force	"true" | "false"	This property determines whether to display a pop-up for announcing the existence of an update, contained in *start_android.json*.
	cancelable	"true" | "false"	If "true", updating will be conducted automatically without pop-up announcement.
	errormsg	"true" | "false"	This property

Features	Property	Value	Description
			determines whether to display a pop-up for showing the error information when the loading of the app fails.
	quiet	"true" | "false"	This property determines whether to display a pop-up that asks you whether to update the app. Even if you specify "true", the progress of app updating will be displayed when there is an update file.
	failpass	"true" | "false"	Set whether to ignore the update errors or not
xpush-server	request-missing-message	"true" | "false"	This property determines whether to automatically request messages that you have not received from an XPush server.
notification	enable	"true" | "false"	This property determines whether to use the notification feature.
	handler	“[function name]”	This property sets a function to process the message or data of a notification. The default value is “com.nexacro.notification.DefaultHandler”.



Click on the below links for the details on the storage path defined by the *file-logging* property.

https://developer.android.com/reference/android/content/Context.html#getExternalCacheDir()
https://developer.android.com/reference/android/content/Context.html#getCacheDir()

6.2.5 Set build environment

You should modify *MainActivity.java* and *AndroidManifest.xml* —default files created in the process of creating an Android project—in a way that suits Nexacro.

MainActivity.java

If you do not specify another name for a main activity at the creation of a project, the activity will be created with the name *MainActivity.java*. To find the file *MainActivity.java*, navigate to the folder of the relevant package name under the directory [project > app > src > main > java]. Then, modify the file as below.

MainActivity.java

```
package com.example.hellonexacroandroid;

import android.os.Bundle;
import com.nexacro.NexacroActivity;

public class MainActivity extends NexacroActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Update Type: Server, Update
        setProjectURL("http://[URL]/");
        setBootstrapURL("http://[URL]/start_android.json");

        // Update Type: Local
        // setBootstrapURL("file:///android_asset/archive/start_android.json");
    }
}
```

plugin_config.xml

The plugin_config.xml file is located in [Project > app > src > main > res > xml]. It contains basic information about the Device API and necessary information. Refer to the code below to write the required information.



If you did not set any other libraries aside from Nexacroib-release.aar when setting up Nexacro libraries, the operations will not be affected, even in the absence of the plugin_config.xml file.

plugin_config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin-config>
    <!-- Nexacro Device API -->
    <plugin name="Network" class="com.nexacro.deviceapi.Network" onload="false"
/>
    <plugin name="Geolocation" class="com.nexacro.deviceapi.Geolocation" onload
="false"/>
    <plugin name="Acceleration" class="com.nexacro.deviceapi.Acceleration"
onload="false"/>
    <plugin name="Sound" class="com.nexacro.deviceapi.AudioPlayer" onload=
"false"/>
    <plugin name="AudioRecorder" class="com.nexacro.deviceapi.AudioRecorder"
onload="false"/>
    <plugin name="Camera" class="com.nexacro.deviceapi.Camera" onload="false"/>
    <plugin name="ImagePicker" class="com.nexacro.deviceapi.ImagePicker" onload
="false"/>
    <plugin name="ExternalAPI" class="com.nexacro.deviceapi.ExternalAPI" onload
="false"/>
    <plugin name="Vibrator" class="com.nexacro.deviceapi.Vibrate" onload=
"false"/>
    <plugin name="Call" class="com.nexacro.deviceapi.Phone" onload="false"/>
    <plugin name="Sms" class="com.nexacro.deviceapi.SmsManager" onload="false"/
>
    <plugin name="ContactSet" class="com.nexacro.deviceapi.Contact" onload=
"false"/>
    <plugin name="SQLStatement" class="com.nexacro.deviceapi.LiteDBStatement"
onload="false"/>
    <plugin name="SQLConnection" class="com.nexacro.deviceapi.LiteDBConnection"
onload="false"/>
</plugin-config>
```

AndroidManifest.xml

The *AndroidManifest.xml* file, which is located in the directory [project > app > src > main], contains basic information about your app and necessary information for performing specific functionalities. Modify the file as shown in the below sample.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.HelloNexacroAndroid"
        android:usesCleartextTraffic="true"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:configChanges="orientation|screenSize|keyboardHidden"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <action android:name="OPEN_NEXACRO_ACTIVITY_EXAMPLE" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>

</manifest>
```



If your app targets the SDK version 28 or higher, you must set the *usesCleartextTraffic* attribute to "true" to maintain HTTP traffic.
If you do not set the attribute, HTTP traffic will not be allowed because the attribute defaults to "false".

https://developer.android.com/guide/topics/manifest/application-element#usesCleartextTraffic

```
<application
android:allowBackup="true"
 android:usesCleartextTraffic="true"
```

AndroidManifest.xml > Additional privilege settings

Set necessary permissions and other information according to the features that your app is equipped with. For example, add the below sample to the manifest file if you want your app to provide the camera feature.

```
...
<uses-permission android:name="android.permission.CAMERA" />
<activity android:name="com.nexacro.deviceAPI.CameraListener"
    android:screenOrientation="landscape">
    <intent-filter>
           <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
<uses-feature android:name="android.hardware.camera.autofocus"
    android:required="false"/>
...
```

The below list contains sets of code to set the permissions for frequently-used features. Pick items necessary for your app and add them to *AndroidManifest.xml*. If you want to see all the permissions defined in Android, go to Manifest.permission.



If you execute your app without setting the permissions necessary for certain features of your app, relevant features will not function with a message that you do not have a permission for the feature.

```
<!-- AudioPlayer, AudioRecorder -->
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />
```

```
<!-- Call -->
<uses-permission android:name="android.permission.CALL_PHONE" />
```

```
<!-- Contact -->
<uses-permission android:name="android.permission.READ_CONTACTS" />
<uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

```
<!-- Camera -->
<uses-permission android:name="android.permission.CAMERA" />
<activity android:name="com.nexacro.deviceAPI.CameraListener"
android:screenOrientation="landscape">
    <intent-filter>
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
<uses-feature android:name="android.hardware.camera.autofocus" android:required
="false"/>
```

```
<!-- Geolocation -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name=
"android.permission.ACCESS_LOCATION_EXTRA_COMMANDS" />
```

```
<!-- Acceleration -->
<uses-permission android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS"
tools:ignore="HighSamplingRate" />
```

```
<!-- Map -->
<uses-permission android:name=
"com.google.android.providers.gsf.permission.READ_GSERVICES"/>
<uses-library android:name="com.google.android.maps" />
<meta-data android:name="com.google.android.maps.v2.API_KEY" android:value=
"user value" />
```



If you want to use Google Maps, you need to get an API key and insert it to the relevant configuration file. Click on the below link to see the details about it.
https://developers.google.com/maps/documentation/android/start#obtain_a_google_maps_api_key

```
<!-- SMS -->
<uses-permission android:name="android.permission.SEND_SMS" />
<uses-permission android:name="android.permission.RECEIVE_SMS" />
<uses-permission android:name="android.permission.READ_SMS" />
<uses-permission android:name="android.permission.WRITE_SMS" />

<receiver android:name="com.nexacro.deviceAPI.SmsRecv">
    <intent-filter>
        <action android:name="android.provider.Telephony.SMS_RECEIVED" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.provider.Telephony.WAP_PUSH_RECEIVED" />
        <data android:mimeType="application/vnd.wap.mms-message" />
    </intent-filter>
</receiver>
```

```
<!-- Vibrator -->
<uses-permission android:name="android.permission.VIBRATE" />
```

```
<!-- Bluetooth -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<service
    android:name="com.nexacro.deviceAPI.BluetoothLEAdapterService"
    android:enabled="true"
    android:exported="true">
</service>
```

```
<!-- ExternalAPI -->
<uses-permission android:name="android.permission.GET_TASKS" />
```

```
<!-- Etc -->
<uses-permission android:name="android.permission.GET_ACCOUNTS" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
```

AndroidManifest.xml > FCM settings



About FCM messages
https://firebase.google.com/docs/cloud-messaging/concept-options

```
<!-- Adding NexacroNotificationService -->
<service android:name="com.nexacro.notification.NexacroNotificationService">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>

<!-- Adding NexacroActivity intent-filter -->
<activity android:name="com.nexacro.NexacroActivity"
    android:launchMode="singleTask"
    android:noHistory="false"
    android:hardwareAccelerated="true"
    android:windowSoftInputMode="adjustResize"
    android:configChanges=
"orientation|keyboard|keyboardHidden|screenSize|navigation|uiMode">
    <intent-filter>
        <action android:name="OPEN_NEXACRO_ACTIVITY_EXAMPLE" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

If you use FCM, the below code listings should be added to the *build.gradle* file. Basically, this configuration is automatically performed in Android Studio. If not performed automatically, however, you need to add the below code listings by yourself and download the *google-services.json* file, which is created in the Firebase console. Then, copy the JSON file to the module directory of your app.



Add Firebase to your Android project
https://firebase.google.com/docs/android/setup

AndroidManifest.xml > ImagePicker, Camera, FileUpload, FileDailog settings



If you set targetsdk version to 24 or higher and use ImagePicker, FileDialog, FileUpload, and Camera components (Camera component with the targetsdk version 29 or higher), you need to

create the file_paths.xml file in res/xml folder, add the following items to AndroidManifest.xml file and build.gradle file.



When using the capture property of the input tag in the HTML content connected to the WebBrowser component, an identical setting is required to access the file.

file_paths.xml

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
    <external-cache-path name="cache" path="."/>
    <external-files-path name="files" path="./"/>
</paths>
```

AndroidManifest.xml / FileProvider

```
<provider
    android:name="android.support.v4.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths"/>
</provider>

<!-- AndroidX -->
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths"/>
</provider>
```

build.gradle / exifinterface

```
dependencies {
    implementation 'com.android.support:exifinterface:28.0.0'
```

```
    //AndroidX
    implementation 'androidx.exifinterface:exifinterface:1.3.2'
}
```

Dependency Management

Write the library information in the build.gradle file.

```
implementation 'androidx.appcompat:appcompat:1.5.1'
implementation 'com.google.android.material:material:1.6.1'
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
implementation "androidx.room:room-runtime:2.4.3"
implementation 'androidx.exifinterface:exifinterface:1.3.4'
implementation 'com.squareup.okhttp3:okhttp:4.10.0'
annotationProcessor "androidx.room:room-compiler:2.4.3"
```

When using XPush, add the library information below.

```
implementation 'org.slf4j:slf4j-api:1.7.36'
```

When using Firebase, add the library information below.

```
implementation 'com.google.firebase:firebase-core:21.1.1'
implementation 'com.google.firebase:firebase-messaging:23.0.8'
```

6.2.6 Set your app stand-alone in disconnected environments

You can run your app without a network connection if you set the update type to "Local" or "Update (Local+Server)" when packing the app in Nexacro Studio. Normally, the Local type is appropriate for users who do not need a network connection while the Update (Local+Server) type suits users who usually maintain communications with servers but still need to operate apps even when networking is in trouble. To make your app stand-alone, you should prepare files necessary for execution and load them onto the app in the building process.

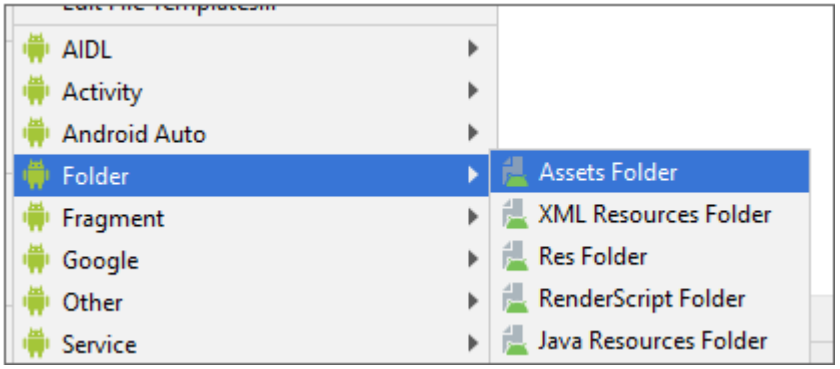
The files necessary for executing an app include modules, sources, resources and *start_android.json*, which are all obtained through the Packing (Archive&Update) process of Nexacro Studio. If you load those files onto the "archive" folder under the "assets" directory in your Android project before building an app, you can run

the app without a network connection.

1 Create the "assets" folder in your project in Android Studio.

> The "assets" directory is a logical storage used in an Android app. After creating the assets folder in your Android project, prepare files to use in the app, copy those files to the assets folder and build the APK of your app. By doing so, you can use those files when you launch the app. Since the assets folder is read-only, the files in this directory should not change during the execution of the app.

Right-click on *app*, the root folder of your project, and select [New > Folder > Assets Folder] from the context menu. Create the assets folder under the directory [project > app > src > main].

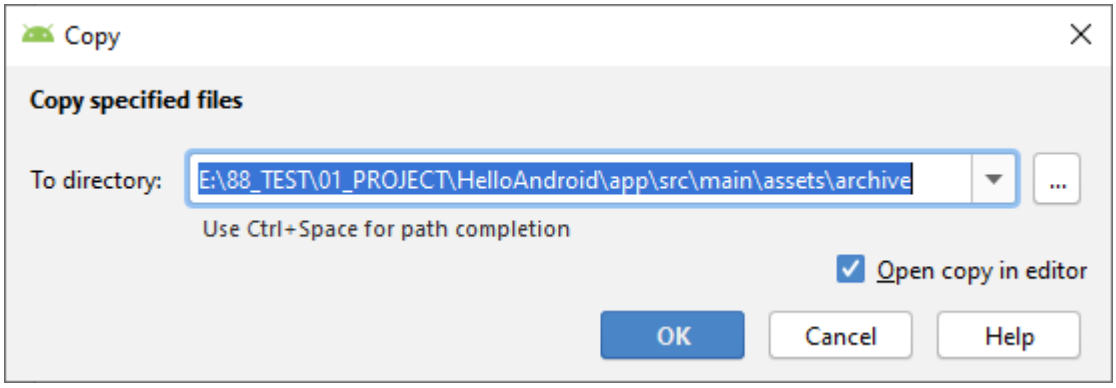


2 Create the "archive" folder under the assets directory.

Right-click the assets folder under the directory [project > app > src > main] and select [New > Directory] from the context menu to create the "archive" folder under the assets folder.

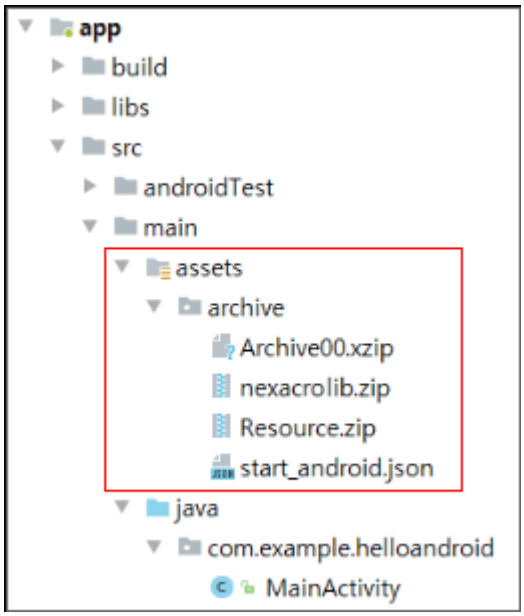
3 Copy module, source and resource files to the assets/archive directory.

Copy module, source and resource files to the [project > app > src > main > assets > archive] directory. You can copy and paste files directly from File Explorer into Android Studio.



4 Check whether the folders and files have been added to the project successfully.

Go to the project view in Android Studio and check whether the added *assets*/*archive* folder and files under the folder are there.



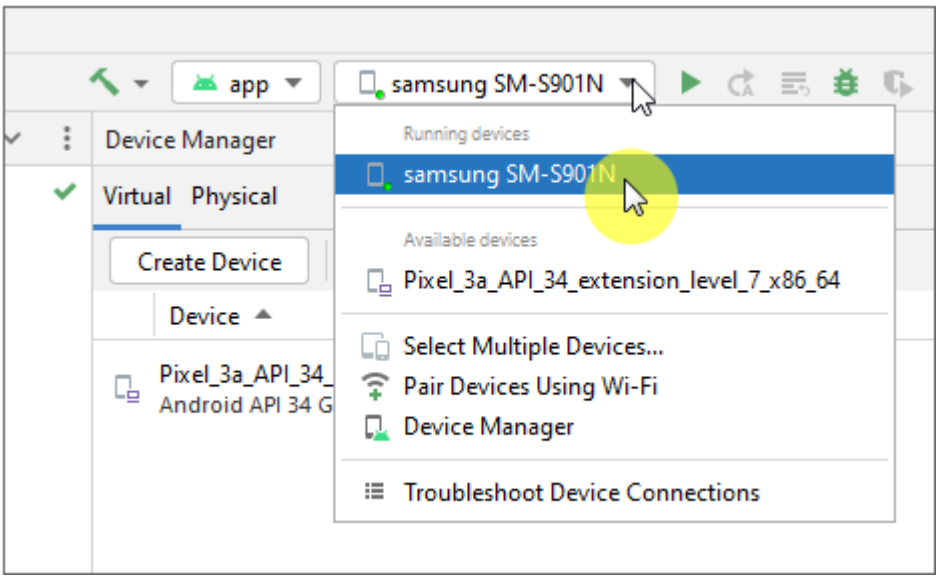
6.3 Build

6.3.1 App test

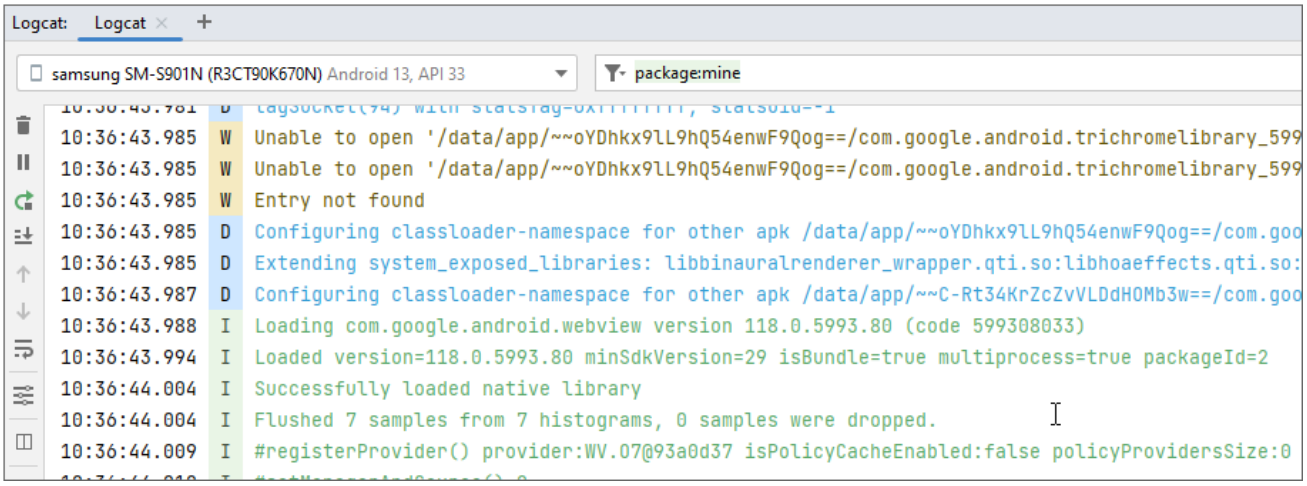
You can test your app by connecting an Android device to your PC using a USB cable. In that case, you need to toggle the USB debugging option in the Developer Options menu in the device and to install a driver for the device.

Click on the below link to learn how to select the USB debugging option.
https://developer.android.com/studio/debug/dev-options

Select a virtual device or connected device from the device list at the top of Android Studio and execute Run.



You can see the log messages for executing the app through the Logcat window, which is situated at the bottom of the Android Studio screen.



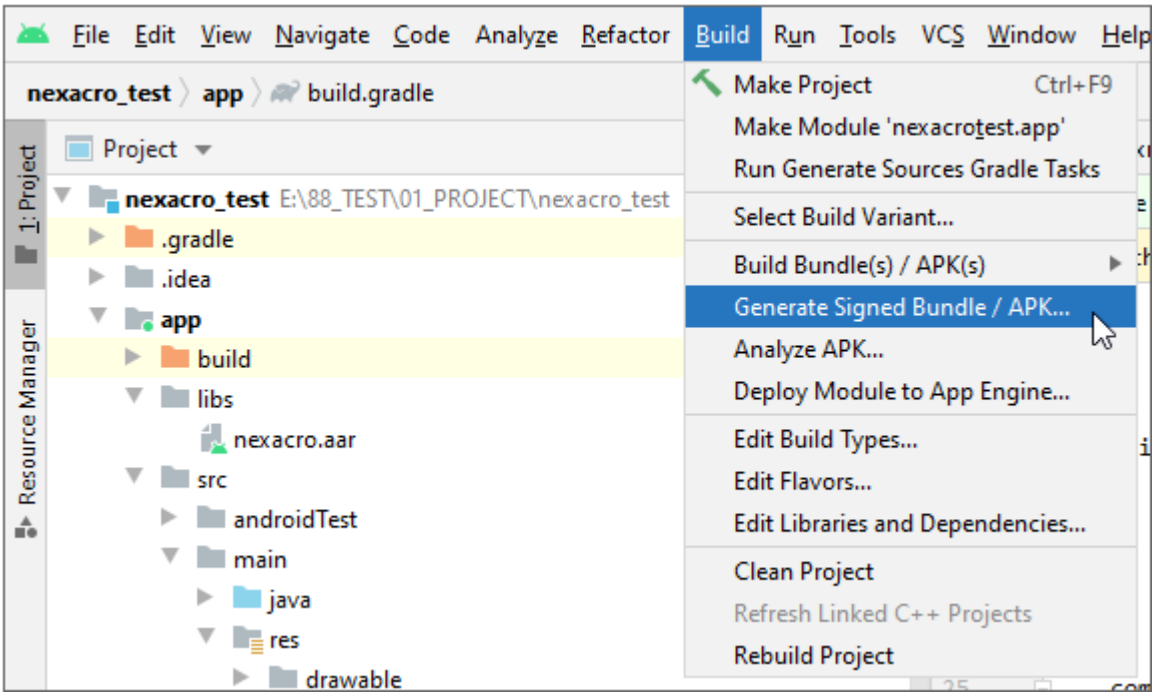
6.3.2 Create installation file

To distribute your app that has been built completely, you should create a keystore that can identify your app and then should create a signed AAB, APK file. You need to create a keystore file just once at the beginning. When the app is updated, you can use the keystore file that you created already.

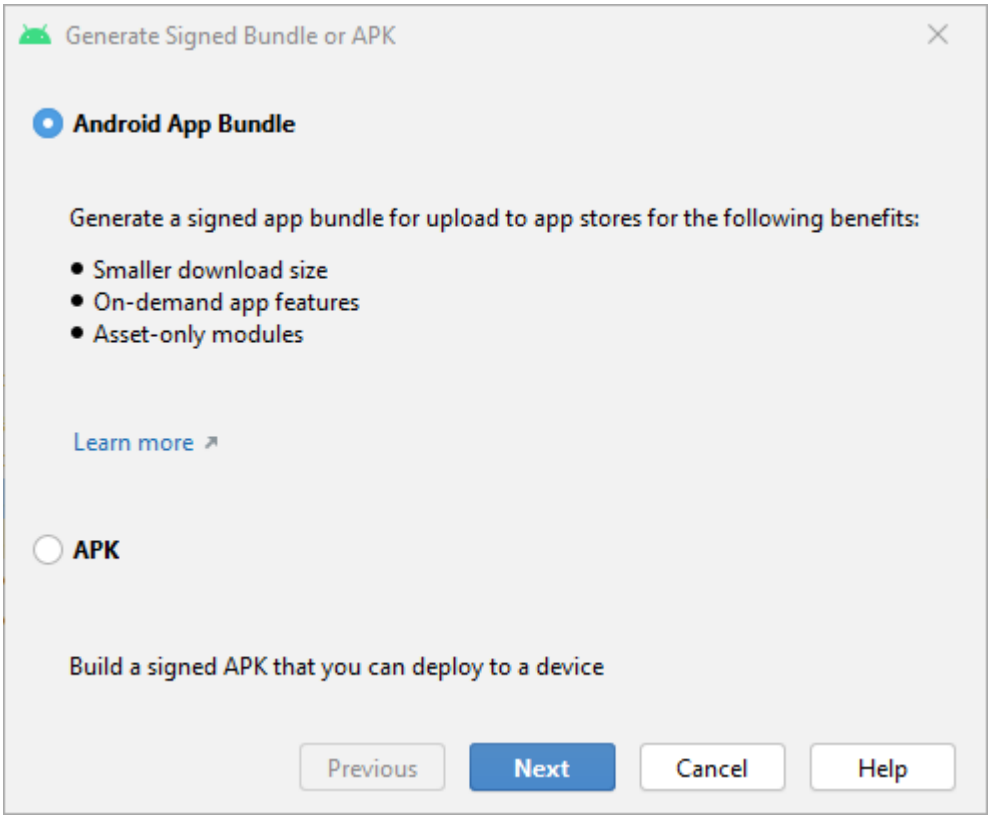
Click on the below link to go to the official Android user guide where you can learn how to create a keystore and signed APK.

http://developer.android.com/guide/publishing/app-signing.html

Select the menu [Build > Generate Singed Bundle/APK].



The default setting is to generate AAB files. If needed, you can select an APK item and generate an APK file.



7.

App Development and Execution (macOS - Xcode)

You can distribute a service developed based on Nexacro to Mac users in the form of a Nexacro Runtime Environment (NRE) app. As an administrator, you need to create deployment files based on a project made by developers. This chapter briefly explains how to create deployment files step by step.

7.1 Preparation

7.1.1 Development environment

You need the Xcode development environment to create an app for macOS. Without addition coding, you just need to go through few steps to make an existing Nexacro-based app applicable to macOS.

The below table shows the basic elements of the development environment for macOS apps.

Elements	Description
Xcode	You may need installation. https://developer.apple.com/xcode/
xcodebuild	This element is installed automatically when Xcode is installed.

7.2 Develop App Project

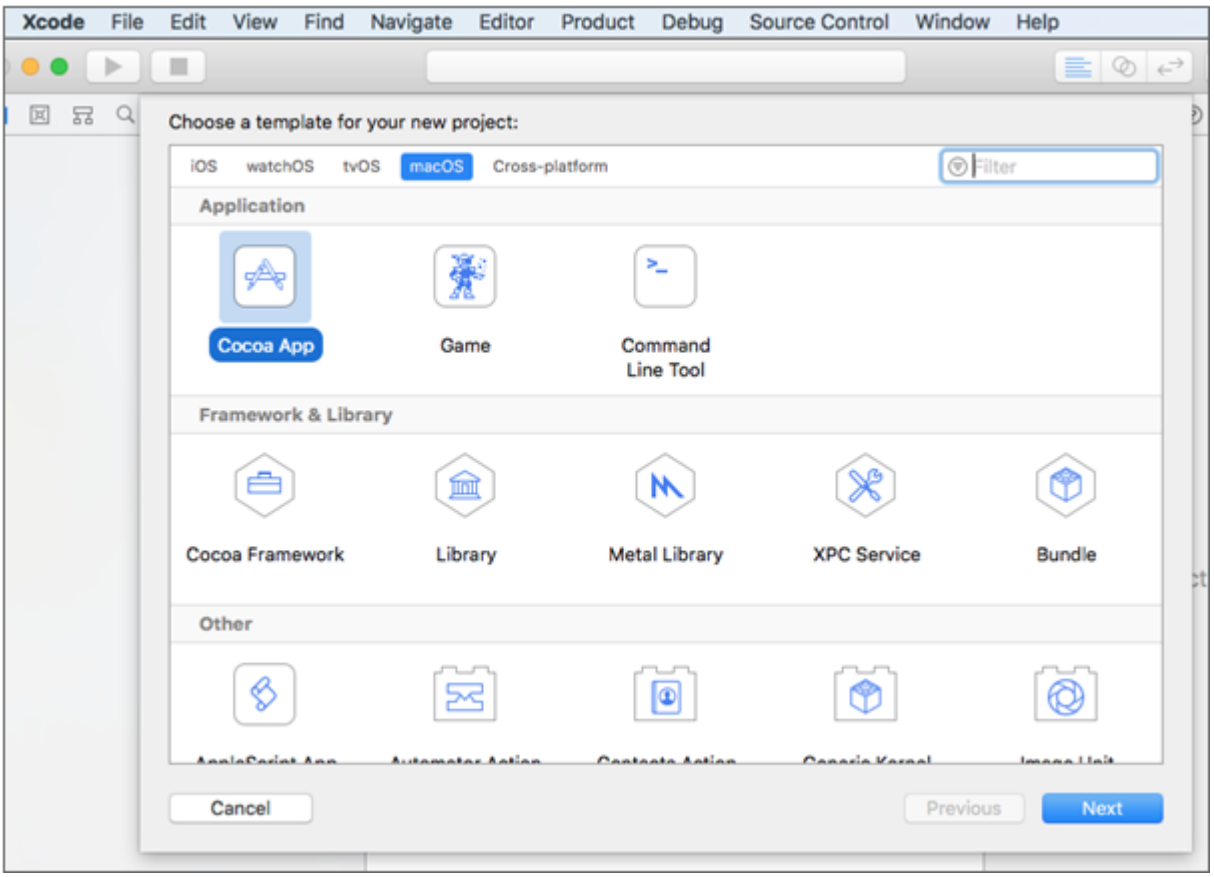
Follow the below steps to work on a macOS app project in Xcode. Before entering an app project, you should find the archive files in an app that you built in Nexacro Studio and locate them in the designated directory.

7.2.1 Create a project

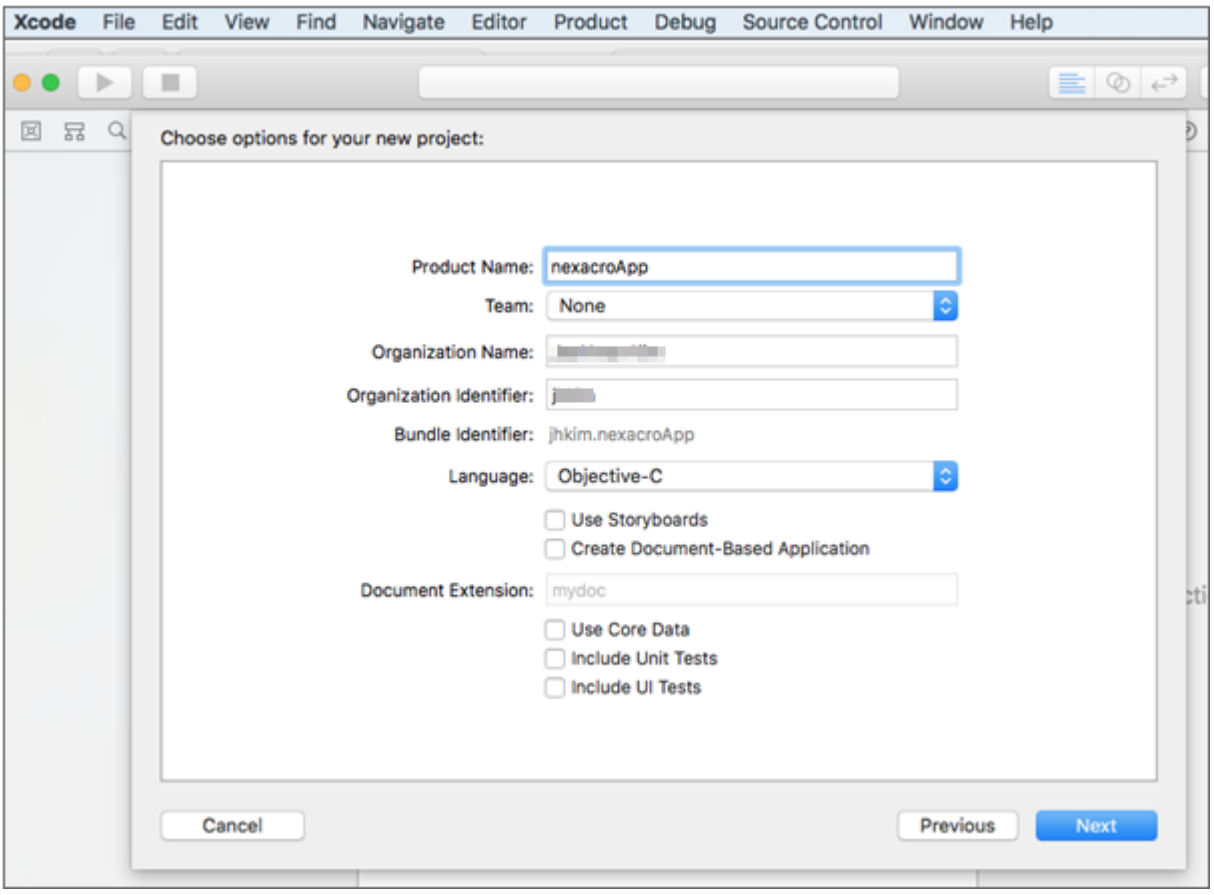
After creating a macOS project that will contain your Nexacro-based app, configure basic settings. You can create a project by accessing the below menu.

```
File > New > Project > macOS
```

Select Cocoa App as a template for the created project.



Set Language to "Objective-C" and set other options including Product Name. Then, click the Create button after designating the folder where the new project is created.

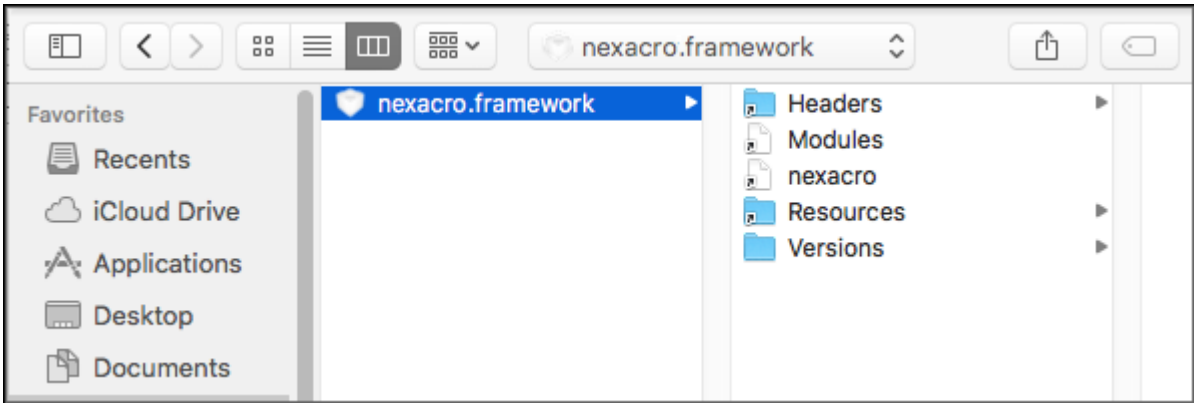


7.2.2 Set Nexacro libraries

Add Nexacro libraries to your macOS project to establish an optimal environment for Nexacro.

Nexacro libraries are provided in the form of a compressed file named *nexacro.framework.zip*. After decompressing the zipped file, bring the libraries to any folder in your project or add them by accessing the menu Add Files to.

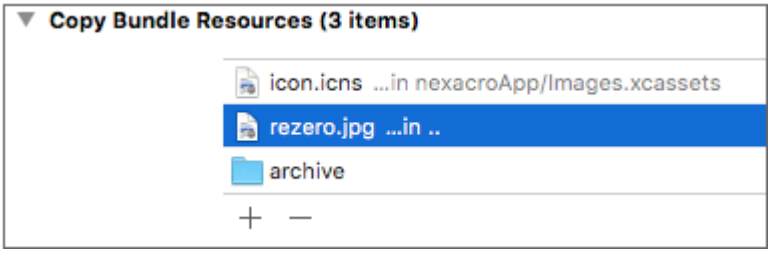
```
File > Add Files to "..."
```



7.2.3 Set resources

In this stage, you can set launch images, icons, messages and layouts applied to your app. You can adjust those options according to projects.

Add image files in the Copy Bundle Resources phase and then modify the *main.m* file as presented under the below picture.




```
startManager.splashImageName = @"rezero.jpg";
```

7.2.4 Set build environment



Delete the *AppDelegate.h*, *AppDelegate.m* and *MainMenu.xib* files, which are created by default but not used.

main.m

Find the *main.m* file located under the Supporting Files folder and modify it as below.

```
//  main.m

#import <Cocoa/Cocoa.h>
#import "nexacro/nexacro.h"

int main(int argc, const char * argv[]) {
    NexacroStartManager *startManager = [NexacroStartManager sharedManager];

    // If omitted, the name of the app is used automatically.
    startManager.applicationKey = @"";

    // This is a required value. If you use the default bootstrap file name,
```

```
you do not need to specify start_macos.json.
    startManager.bootstrapUrl = @
"http://172.10.12.180:8080/Mobile_Test/start_macos.json";

    // If omitted, the URL is the same as the directory where the bootstrap
file is located.
    startManager.projectUrl = @"http://172.10.12.180:8080/Mobile_Test/";

    // Set launch screen (Refers to the resources folder. If omitted, the
default launch screen of Nexacro is used.)
    // startManager.splashImageName = @"sample.png";

    [NSApplication sharedApplication];
    [NSBundle loadNibFile:[startManager getNexacroNibFile]
externalNameTable:nil withZone:nil];
    [NSApp run];

    return EXIT_SUCCESS;
}
```

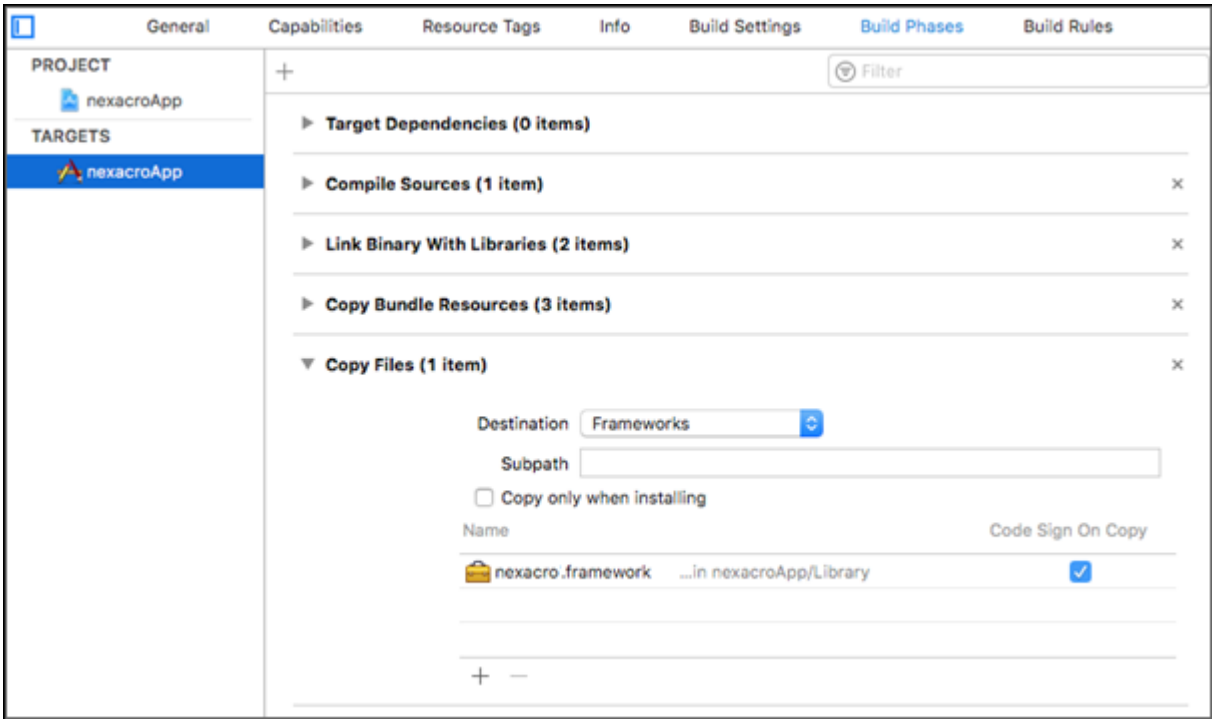
Other settings

General	Deployment Target: 10.7 Main Interface: delete MainMenu
Build Settings	Architectures: $(ARCHS_STANDARD) macOS Deployment Target: macOS 10.7 Objective-C Automatic Reference Counting: Yes



32-bit apps are deprecated in the macOS version released in 2018. From 2019, 32-bit apps are no longer compatible with macOS just like iOS 11.

- Add a framework in the menu [Build Phases > Copy Files].
  Click the Add icon (+) > New Copy Files Phase > Destination: Frameworks
  Add *nexacro.framework*



- Configure settings as below in Information Property List
  App Transport Security Settings > Allow Arbitrary Loads



If you do not set Allow Arbitrary Loads to "YES", a WebBrowser component will not be displayed on the screen.

Appendix A.

Setting Path

A.1 Alias Path

Nexacro uses the following aliases for setting paths on the user PC.

Alias	Actual path
%USERAPP%	%LOCALAPPDATA%LOW\TOBESOFT\Nexacro N\24 (WIndows) /Users/[UserName]/Library/TOBESOFT/Nexacro N/24 (macOS) /data/data/[PackageName]/files/NEXACRO/ (Android) /data/data/[PackageName]/files/[Key] (Android key setting) [AppRoot]/Library/Application Support/NEXACRO/ (iOS)
%UPDATE%	%USERAPP%\Update
%WEBDEPLOY SETUP%	Server Context root
%WEBDEPLOY PROJECT%	Server Context root
%WEBDEPLOY FRAMEWORK%	%WEBDEPLOY PROJECT%\nexacrolib\framework
%WEBDEPLOY COMPONENT%	%WEBDEPLOY PROJECT%\nexacrolib\component
%WEBDEPLOY RESOURCE%	%WEBDEPLOY PROJECT%\nexacrolib\resources
%WEBDEPLOY THEME%	%WEBDEPLOY PROJECT%_resource__theme_
%SD_CARD%	/storage/emulated/0/Android/data/[PackageName]/files/

This alias path can be used when inputting distribution information paths or path information in application programs.



Actual path names can vary according to product versions or license policies of relevant countries.



The actual path of *%NEXACRO%* can differ if you change the installation path by using the Setup Manager when you create an installation file.

A.2 nexacro.xml

The *nexacro.xml* file is XML used to contain information for each user. The file path is as follows.

> %USERAPP%

A.3 Relative Path

A.3.1 Relative Path Support for Paths in Projects

The chart below shows the types of major paths and relative path support, and service usage path support (Service::a.xfdl format path).

File Type	Category	Relative Path		Supports Service Usage Path
		Support	Standard Path	
Typedefinition	Component UpdateURL ❶	Y	TypeDefintion	Y
	Service Path	Y	TypeDefintion	N
GlobalVariable	Image	Y	GlobalVariable	Y
XADL	TypeDefinition ❷	Y	XADL	N
	GlobalVariable	Y	XADL	Y
	EngineUrl ❸	Y	XADL	N
	LiceneceUrl ❹	Y	XADL	N
	Themeid	Y	XADL	Y

1. Component UpdateURL
   A path for the server to update components. It also corresponds to the *TypeDefinition Update Url* in tool. However, if its domain (for rexample, *http://a.b.c/path* > *a.b.c*) isn't matched with the XADL path, update is not available. If XADL has a local path, update is available regardless of the domain information.
2. TypeDefinition
   Paths for services. It also corresponds to *TypeDefinition* Services url in tool.
3. Engine path
   A path to update the engine. It also means ADL *engineurl* in tool. However, if its domain is not matched with the XADL path, update is not available. If XADL has a local path, update is available regardless of the domain information.
4. Licenece path
   A path to check the license before reading the *Typedefinition*.

It means *ADL licenseurl* in Tool.

However, its domain should be not matched with the XADL path. If XADL has a local path, the license will be checked regardless of the domain.

Appendix B.

Exceptions

This chapter is for reference only since the user environment will vary. It describes issues you may encounter while using the Nexacro and presents possible solutions..

B.1 Web Browser Option

Changing web browser options may impact how an app runs as described in this section.

B.1.1 JavaScript Activation

Issue

Setting the web browser to not use JavaScript causes the app to not function normally.

Description

Even though basic options are set to activate JavaScript in general, you need to check the relevant option first when the apps cannot be found on the screen since you can choose other options.

Possible Solution

Depending on the web browser, activate the JavaScript usage option. (The option can vary depending on brower version.) Refer to the following link or each web browser's help for details.

```
http://www.enable-javascript.com
```

B.1.2 File Download Activation

Issue

Even though you restrict to download files from web browsers, files can be downloaded using File download component.

Possible Solution

Even though basic options are set to activate file download, you need to check the relevant option first when the files are not downloaded since you can choose other options.

Depending on the browser, activate the file download option. (The option can vary depending on browser version.) Refer to the web browser's help for details.

Appendix C.

Checking the NRE log

If the app terminates unexpectedly or encounters an error while running in the user's environment, a log file can be created for analysis and to check the problem.



The log function is supported in the NRE of Windows and macOS only.



Errors that occur when running the app before the log manager runs are not logged. When the app is launched, it is displayed to the user as an alert window.

C.1 Create log file settings

The create log file feature is not a default setting and requires additional setup. Log file creation settings can be set in Nexacro Studio or in a config file.

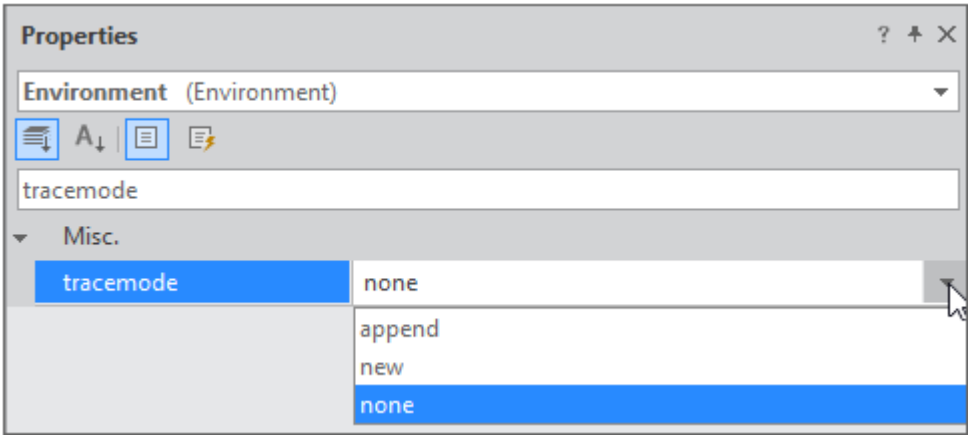
When the app is run, the settings in the config file are applied first. If there is a config file, the value set in the Environment item is ignored.

C.1.1 Create log file settings in Environment

The property values and log functions can be set in Nexacro Studio. Related property values can only be set in Nexacro Studio and cannot be changed while the app is running.

Setting log file creation in the tracemode property

Select Environment in Nexacro Studio and change the tracemode. If the default value is "none", the log function is not processed, and if the property value is set to "new" or "append", a log file is created.



Setting logging level and frequency

You can select Environment in Nexacro Studio and set the loglevel and traceduration properties. The loglevel property sets the situation for the logging and the traceduration property sets the cycle for log file creation.

C.1.2 Config file settings

The log feature can be set in the config file. The Environment property can be set without editing by copying the config file into the user's execution environment or by including it in the build.

Setting -NC options

The "-NC" option can be used when running the nexacro.exe file in the Windows operating system. The config file can be set in the same location as the nexacro.exe file to be run.

For example, if the location of the nexacro.exe file has specified the "log_config.xml" file as the warning -NC value as below, the "log_config.xml" file must exist in the same path.

```
C:\Program Files\TOBESOFT\Nexacro N\21\TEST_LOG\nexacro.exe
C:\Program Files\TOBESOFT\Nexacro N\21\TEST_LOG\log_config.xml
```

If the -NC option is not used, the nexacro_config.xml file will be searched for in the default path and it will be referred to if it exists.

Setting nexacro_config.xml file

For -NC option settings, a file name can be specified, but a separate option must be specified. The nexacro_config.xml file automatically recognizes and processes a file with a given name in the specified location when running the app.

Windows operating systems

In the Windows OS, the feature for distributing the apps by including the nexacro_config.xml file is not included. The nexacro_config.xml file can be created in the path below and referred to it when running the app.

```
%USERAPP%\Config\[Key]\nexacro_config.xml
```



[Key] is the -K option specified in the nexacro.exe execution option or the App Title specified in the Build App stage.

macOS operating systems

App Development and Execution (iOS/iPadOS - Xcode) > Set Config

C.2 nexacro_config.xml file properties

The configuration file (nexacro_config.xml) or the file specified with the -NC option is written in the format shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<nexacro-config os="windows">
    <application
        file-logging="true"
        loglevel="debug"
        tracemode="new"
        traceduration="-1"/>
    <log
        filepath="%USERAPP%\Log\nre_log_test.log"
        filesize="408960"
        backupfilecount="5"/>
</nexacro-config>
```

Among the properties specified in the nexacro_config.xml file, only those related to the log file creation are explained below.

Tags/Properties	Explanation
nexacro-config	
os	This property sets the operating system. - windows: Windows operating system - Android: Android operating system - osx: macOS operating system
application	
file-logging	This property sets whether to save the initial loading error information. This is the same as the information recorded in the nexacro_startError.log file in the %USERAPP% path
loglevel	This property sets the log output level. fatal | error | warn | info | debug Fatal, error, warn, and info are log information processed inside the framework. If the trace method is used or a script error occurs, it is processed as debug.
tracemode	This property sets how log files are processed. new: delete the previous record and record a new log append: record the log after the previous record
traceduration	This property sets options for creating daily log files. The default value is -1.
log	
filepath	This property sets the log file path and file name. If no path is specified, the file is created in the path below. - Windows: %USERAPP% - Android: %SD_CARD%/Android/data/[package name]/files/nexacro/21/Log/ - macOS: %USERAPP%/Log If the file name is not set, the file is created in the format shown below. - nexacro_[protocol+url+port][bootstrapfilename].log ex) nexacro_httplocalhost8080startjson.log
filesize	This property sets the maximum size of the log file. The default value is 204,800 bytes. If you set a value less than the default, the default is applied.
backupfilecount	This property sets the maximum number of the log files. This is the number of files that can be left as backup when the log file size becomes larger than the filesize. The default value is 1.



If the os property value is "windows", the filepath property value may cause security issues, so the following restrictions are applied.

- If this property value is not an allowed path or a subpath of an allowed path, it is changed forcibly to the default path.
  The path that can be specified using Alias in the system.convertRealPath method is the

allowed path.

- If the path is the same as the startup program path, it is changed forcibly to the default path.
- If the file extension is as below, the extension is changed forcibly to "log"
  bat, bin, cmd, com, cpl, exe, gadget, inf1, ins, inx, isu, job, jse, lnk, msc, msi, msp, mst, paf, pif, ps1, reg, rgs, scr, sct, shb, shs, u3p, vb, vbe, vbs, vbscript, js, ws, wsf, wsh, dll, hta

C.3 Example of file creation by property

C.3.1 backupfilecount

If the size of the created log file exceeds the value of the filesize property, backup files are additionally created.

Create files by adding numbers to the end of the file names. When the value specified by the backupfilecount property is reached, the first created file is deleted and the log is recorded.

If the app runs every hour and records a 300KB log run, the log files are created as follows.

```
<log filepath="%USERAPP%nrelog.log" filesize="200" backupfilecount="3"/>
```

Log file name	File size	Time of file creation/change	Task order
2020/05/26 13:00			
nrelog.log	100KB	2020/05/26 13:00	Final log
nrelog.log.1	200KB	2020/05/26 13:00	Start log
2020/05/26 14:00			
nrelog.log	10KB	2020/05/26 14:00	Final log
nrelog.log.1	200KB	2020/05/26 14:00	
nrelog.log.2	200KB	2020/05/26 14:00	
nrelog.log.3	200KB	2020/05/26 13:00	Start log
2020/05/26 15:00			
nrelog.log	180KB	2020/05/26 15:00	Final log
nrelog.log.1	200KB	2020/05/26 14:00	
nrelog.log.2	200KB	2020/05/26 14:00	
nrelog.log.3	200KB	2020/05/26 15:00	Start log / roll over

C.3.2 traceduration

Based on the date of log file creation, files created before the value of the traceduration property are deleted.

A file is created and the date ("_yyyymmdd") is added to the end of the file name. Date comparison is processed according to the operating system system time.

If the app is run once a day and not on weekends, log files are created and deleted as follows. Since the app is not run on weekends (Saturday and Sunday), previously created log files are not deleted. When the app is run on Monday, log files before the traceduration property value are deleted.

```
<nexacro-config os="windows">
    <application file-logging="true" quiet="false" loglevel="debug" tracemode=
"new" traceduration="2"/>
    <log filepath="%USERAPP%nrelog.log" filesize="408960" backupfilecount="3"/>
</nexacro-config>
```

Log file name	Date of file creation	Task
2020/05/21 (Thursday)		
nrelog_20200521.log	2020/05/21	Create file
2020/05/22 (Friday)		
nrelog_20200521.log	2020/05/21	Maintain
nrelog_20200522.log	2020/05/22	Create file
2020/05/23 (Saturday)		
nrelog_20200521.log	2020/05/21	Maintain
nrelog_20200522.log	2020/05/22	Maintain
2020/05/24 (Sunday)		
nrelog_20200521.log	2020/05/21	Maintain
nrelog_20200522.log	2020/05/22	Maintain
2020/05/25 (Monday)		
~~nrelog_20200521.log~~	2020/05/21	Delete file
~~nrelog_20200522.log~~	2020/05/22	Delete file
nrelog_20200525.log	2020/05/25	Create file
2020/05/26 (Tuesday)		
nrelog_20200525.log	2020/05/25	Maintain
nrelog_20200526.log	2020/05/26	Create file
2020/05/27 (Wednesday)		
~~nrelog_20200525.log~~	2020/05/25	Delete file
nrelog_20200526.log	2020/05/26	Maintain
nrelog_20200527.log	2020/05/27	Create file

C.4 Reading log files

The log files can be viewed using the appropriate operating system file explorer.

To implement the feature for viewing the log files within the app, obtain the path of the log file and read the contents of the file using the VirtualFile object.

C.4.1 Reading log files script example

This is an example of getting the first file stored in the corresponding path when the Button is clicked and displaying it in the TextArea component.

```
this.Button01_onclick = function(obj:Button,  e:nexacro.ClickEventInfo)
{
    var strLogpath = nexacro.getLogFilePath();
    this.VirtualFile00.getFileList( strLogpath, ".log",
VirtualFile.findFileOnly );
}

this.VirtualFile00_onsuccess = function(obj:VirtualFile,
e:nexacro.VirtualFileEventInfo)
{
    switch (e.reason)
    {
        case 1:    // open
            this.VirtualFile00.read();
            break;
        case 3:    // read
            this.TextArea00.value = e.textdata;
            break;
        case 8:    // getFileList
            var logfilename = logpath + e.fileattributelist[0].filename;
            this.VirtualFile00.open(logfilename,  VirtualFile.openText);
            break;
    }
}
```

C.4.2 Restrictions in the Windows operating system

To access the log file in the Windows operating system, the environment filesecurelevel property value must be set to "all". The sandbox window is displayed upon the initial run.

Appendix D.

NRE remote debugging

D.1 NRE remote debugging

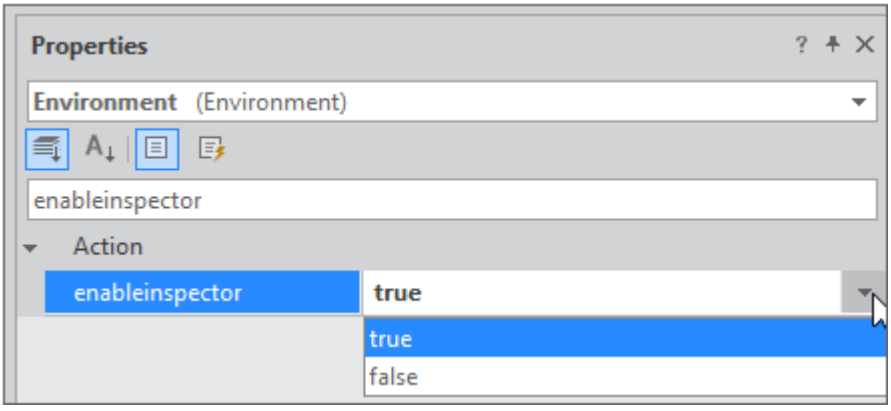
With the NRE, you can debug the running app using the Devices inspect function of the Chrome and Edge browsers. It supported in Windows and macOS operating systems.

Developed scripts and JavaScript libraries can be debugged. It does not support modules installed with DeviceAdaptors or engine internal code debugging.

The Android OS NRE uses the WebView debugging feature. Please refer to the document below.
https://developer.android.com/reference/android/webkit/WebView#setWebContentsDebuggingEnabled(boolean)

D.1.1 Project settings

Change the enableinspector property value of Environment to true.



D.1.2 Run settings

Windows

Add the -RPP option to the nexacro.exe execution option as shown below and run it.

```
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP
127.0.0.1:9223
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 9223
```

If only PORT is set when setting the RPP option, it operates as the localhost.

macOS

When running in the terminal, add the -RPP option as shown below and execute.

```
open nexaApp.app --args -K INSPECTOR_TEST -S
http://localhost:8080/start_macos.json -RPP "127.0.0.1:9223"
open nexaApp.app --args -K INSPECTOR_TEST -S
http://localhost:8080/start_macos.json -RPP "9223"
```

If only PORT is set when setting the RPP option, it operates as the localhost.

D.1.3 Run the Devices inspect feature

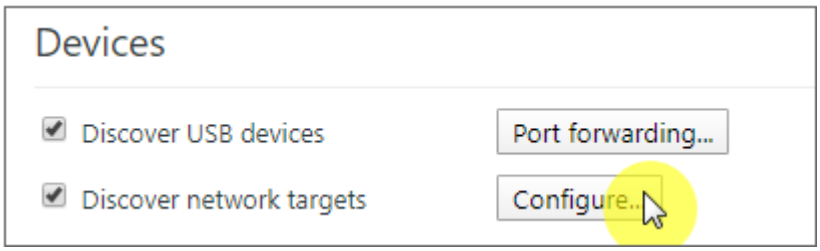
1. Launch the Chrome browser and enter "chrome://inspect" in the address bar.

   ```
   chrome://inspect
   ```

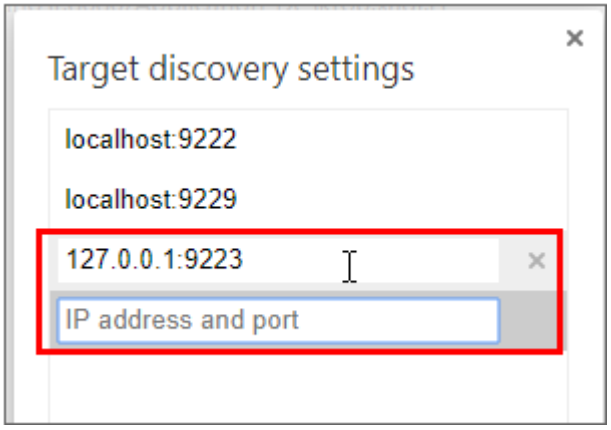
   

2. Click the Configure button to open the Target discovery settings window.

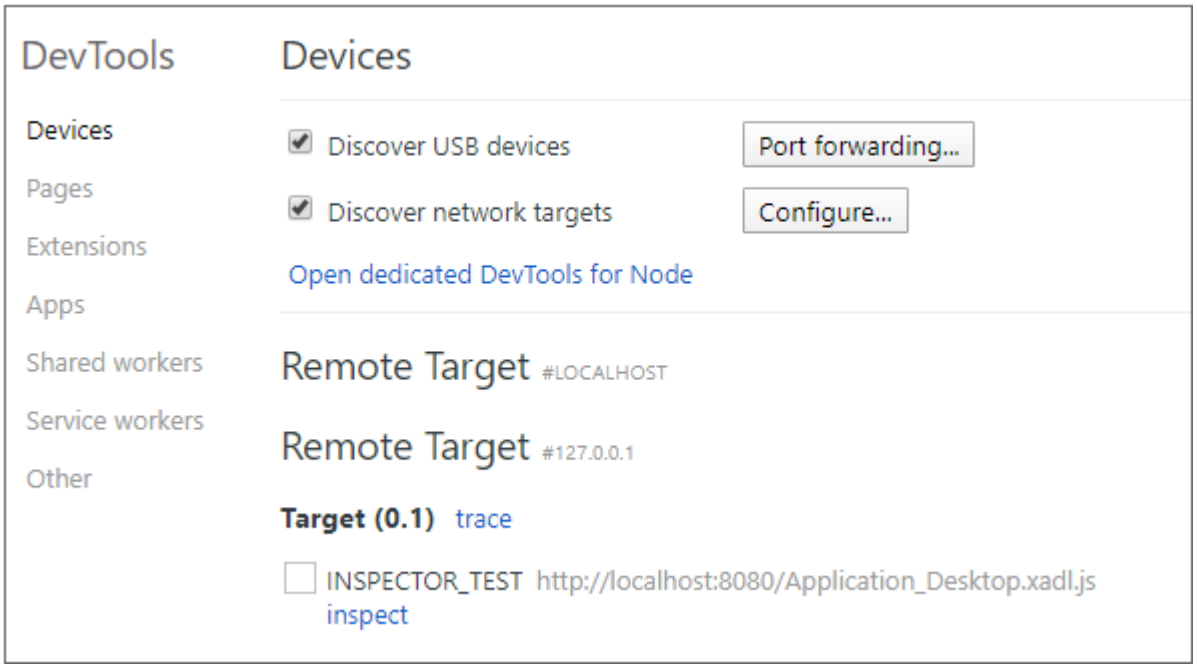
   If the Discover network targets is not checked, check it.



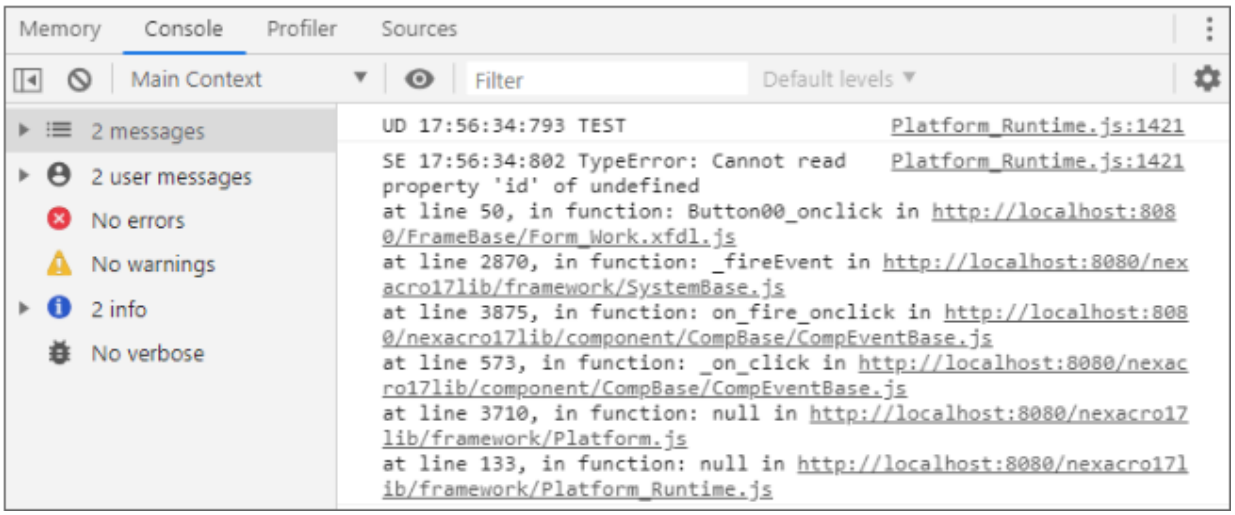
3 Input the RPP address value set earlier to the Target discovery settings window.



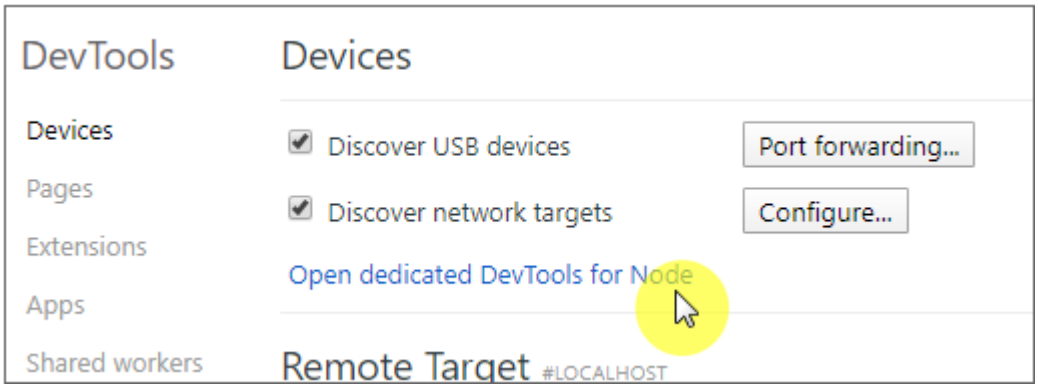
4 Run the app, and the key value of the launched app is displayed in the Remote Target item.



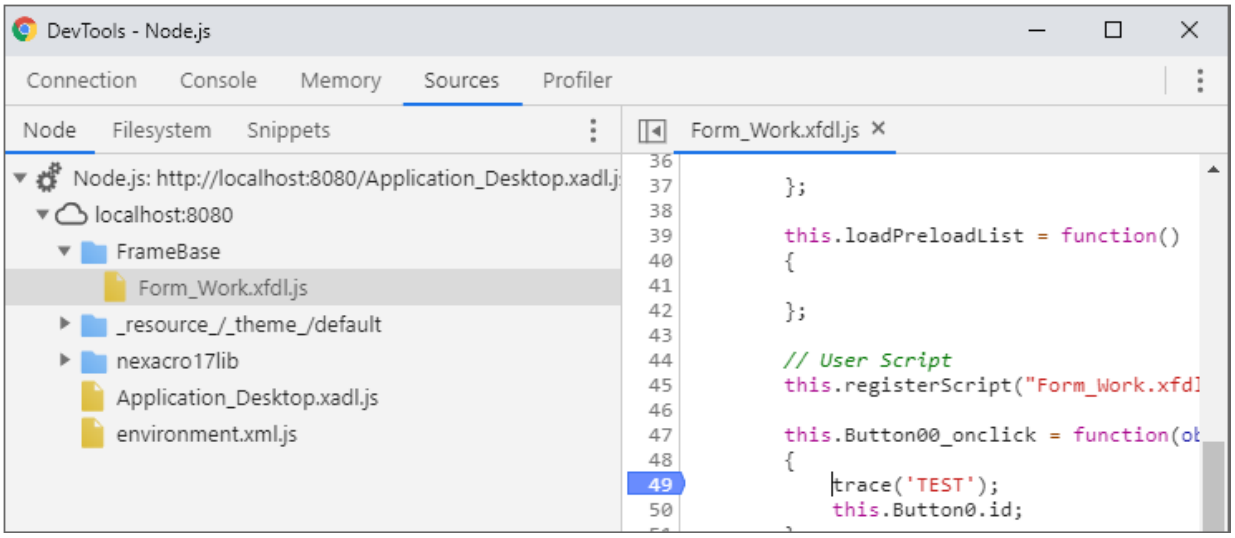
5 Click the inspect URL under the Target item to be redirected to the console window where you can view the results of the trace method execution or error messages.



6 Click the open dedicated devtools for node URL to open a window for exploring the server resources.



In the [Sources > Node] tab, you can select the desired file, add breakpoints, or perform additional debugging tasks.



If the app is not visible in Remote Target

If the app is not displayed for more than 20 seconds after launching, take the following measures.

1. Close the browser and reestablish the connection.

2. Check that the inspect feature is running properly in the app.

   After running the app, accessing the following URL in your browser should display the information. If there is no response, there may be a problem with your project settings or other issues.

   ```
   [Target discovery settings RPP Address]/json/list
   http://127.0.0.1:9223/json/list
   ```



D.1.4 Option settings for Pause While Loading feature

Because NRE remote debugging connects to the running app and executes debugging, it is not possible to debug errors or issues that occur during loading. A setting to forcibly pause during loading can be added through the additional options as follows.



The pause while loading feature pauses upon the completion of receiving the theme file (theme.map.js), then waits for the Devices inspect connection. Depending on the project loading method or network conditions, some files may not be downloaded at this time, and the files not downloaded cannot be debugged.

Windows, macOS

Add the -brk "true" option to the nexacro.exe execution option as shown below and execute it.

```
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP
127.0.0.1:9223 -brk "true"
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 9223
-brk "true"
```

macOS

When running in the terminal, add the -brk "true" option as shown below and execute.

```
open nexaApp.app --args -K INSPECTOR_TEST -S
http://localhost:8080/start_macos.json -RPP "127.0.0.1:9223" -brk "true"
open nexaApp.app --args -K INSPECTOR_TEST -S
http://localhost:8080/start_macos.json -RPP "9223" -brk "true"
```

Appendix E.

Applying the Cordova Plugin

You can build an app with the Cordova plugin applied.

E.1 Nexacro Studio

1. Install the Cordova plugin in Nexacro Studio.
2. Install the Cordova plugin and proceed with the packing process.
3. In the Project Explorer, select TypeDefinition > CordovaPlugins and execute Export from the Context menu.

E.2 Android - Android Studio

1. Copy the file folder exported from Nexacro Studio to the path of your app project.
2. Register the plugin in the plugin_config.xml file by referring to the plugin plugin.xml file and the documented guide.

   For example, for cordova-plugin-statusbar, the plugin.xml file will be as follows:

```
<platform name="android">
    <source-file src="src/android/StatusBar.java" target-dir="src/org/apache/cordova/statusbar" />

    <config-file target="res/xml/config.xml" parent="/*">
        <feature name="StatusBar">
            <param name="android-package" value="org.apache.cordova.statusbar.StatusBar" />
            <param name="onload" value="true" />
        </feature>
        <preference name="StatusBarOverlaysWebView" value="true" />
    </config-file>
</platform>
```

For name and class, refer to the feature name and android-package items in the plugin.xml file.

```
...
<plugin name="StatusBar" class="org.apache.cordova.statusbar.StatusBar"
onload="true"/>
<preference name="StatusBarBackgroundColor" value="#FFFF00" />
...
```

For preference, refer to the documentation provided by each plugin. The documentation provided by cordova-plugin-statusbar is as shown below.

> https://github.com/apache/cordova-plugin-statusbar/tree/master#preferences

E.3 iOS/iPadOS - Xcode

1. Compress the file folder exported from Nexacro Studio, copy it to the device where Xcode is installed, and then unzip it.

2. Register the plugin in the plugin_config.xml file by referring to the plugin plugin.xml file and the documented guide.

   For example, for cordova-plugin-statusbar, the plugin.xml file will be as follows:

```
<platform name="ios">
    <config-file target="config.xml" parent="/*">
        <feature name="StatusBar">
            <param name="ios-package" value="CDVStatusBar" />
            <param name="onload" value="true" />
        </feature>
        <preference name="StatusBarOverlaysWebView" value="true" />
        <preference name="StatusBarStyle" value="lightcontent" />
    </config-file>

    <header-file src="src/ios/CDVStatusBar.h" />
    <source-file src="src/ios/CDVStatusBar.m" />
</platform>
```

For name and class, refer to the feature name and ios-package items in the plugin.xml file.

```
...
<plugin name="StatusBar" class="CDVStatusBar" onload="true"/>
<preference name="StatusBarOverlaysWebView" value="false" />
<preference name="StatusBarBackgroundColor" value="#FFFF00" />
...
```

For preference, refer to the documentation provided by each plugin. The documentation provided by cordova-plugin-statusbar is as shown below.

> https://github.com/apache/cordova-plugin-statusbar/tree/master#preferences

3. In Xcode, select the project, select [Add Files to] from the Context menu, and add the files inside the file folder that has been exported from Nexaro Studio.

   For example, cordova-plugin-statusbar will be as shown below.

   > CDVStatusBar.h
   > CDVStatusBar.m

4. Modify the import syntax path in the copied file by adding "nexacro/" in front of it.

```
#import <Cordova/CDVPlugin.h>
>
#import <nexacro/Cordova/CDVPlugin.h>
```